

Security and Decentralization of Blockchains in the presence of Rational Players

Thesis submitted in partial
fulfillment of the requirements of the degree of

Master of Science

in

Computer Science and Engineering by Research

by

Varul Srivastava

2019111015

`varul.srivastava@research.iiit.ac.in`

Advised by Dr. Sujit Prakash Gujar



International Institute of Information Technology

Hyderabad - 500 032, INDIA

May, 2025

Copyright © Varul Srivastava, 2025
All Rights Reserved

International Institute of Information Technology
Hyderabad, India

CERTIFICATE

It is certified that the work contained in this thesis, titled “Security and Decentralization of Blockchains in presence of Rational Players” by Varul Srivastava, has been carried out under my supervision and is not submitted elsewhere for a degree.

Date

Adviser: Prof. Sujit P Gujar

To my *family*.

Acknowledgments

I wish to express my sincere gratitude to everyone who has contributed to this thesis.

First, I thank IIIT Hyderabad and the Machine Learning Lab (MLL) for providing an inspiring research environment. My advisor, Professor Sujit Gujar, has been instrumental in shaping my mindset and fostering my independent thinking as a researcher. His high expectations helped me push myself and unwavering support—especially during the rejections I faced in my early years—taught me resilience and mental discipline. I am deeply grateful for his encouragement throughout my academic journey. I also thank Professor Kannan Srinathan for his valuable insights on cryptography, research methodology, and life.

Professor Gujar has cultivated a strong sense of community within the lab where we bonded through tea breaks, outings, cricket matches, and festival celebrations. I extend my sincerest thanks to my labmates Sankarshan Damle, Manisha Padala, Sanjay Chandlekar, Anurag Jain, Sanidhay Arora, Shantanu Das, Shaily Mishra, Samhita Kanaparthi, Debojit Das, Sambhav Solanki, Bhaskar, Tejas, Srivatsa, Rasheed, Avyukta, Parth, and Pronoy. I have had the chance to collaborate with some, help some more and discuss about research with all. It has helped me gain a broader vision of how to approach problems in various disciplines.

I am grateful to my coauthors over the years, Sankarshan Damle, Manisha Padala, Sanjay, Uday Bhaskar, and Avyukta, for their ideas, suggestions, and collaboration. In particular, Sankarshan Damle has been my guide and elder brother since the beginning of my research; his honest feedback and counsel has been invaluable to my growth.

I have also been fortunate to share this journey with my friends Rojjha, Atman, Zishan, Akshett, Shivang, Keshav, Trinadh, Ishu and Harshit. Their unwavering support in good and difficult times means more than words can express. I also thank all my teachers, colleagues, friends, staff and the administration for their help throughout my (long) journey at IIIT, Hyderabad.

Finally, I am indebted to Krishna Vasudev, my parents, and my brother Tejas. Their unconditional love, patience, and constant belief in me has been the basis of all my achievements.

This thesis is a testament of the hardwork, dedication and sacrifices made by the above-mentioned people, upon whose shoulders I stand.

Abstract

In this thesis, we study the problem of Security and Decentralization in distributed systems when incentive driven players are present. The challenge of security is crucial in distributed systems to ensure agreement of the current state among different players distributed across the globe. Distributed systems is used for critical infrastructures such as communication, finance and banking, data warehousing and storage. Therefore, the presence of security vulnerabilities allows for an attacker to exploit such infrastructure and cause disruptions — which can indirectly lead to severe consequences.

Similarly, decentralization is another critical requirement from democratized distributed systems such as Blockchains, to ensure that no single player or a small set of like-minded players hold disproportionate control over the system. Centralization of a democratized distributed system such as blockchains can cause challenges such as censorship, reversing transactions, double payment, unilateral changes in the protocol, etc. Therefore, the challenges of security and decentralization of distributed systems are of critical importance.

Recent literature has tackled this problem against a set of completely disruptive player (called Byzantine player) and a set of honest (altruistic) players that follow the protocol honestly. However, a more realistic model of the system would involve a discussion on incentive-driven players, which will follow a profit-maximizing strategy. This is the behavior of real-world players involved in such distributed protocols. Decentralization in Proof-of-Work (PoW) based blockchain protocols (which is a type of distributed consensus protocol) is challenged by Rational players – who aim to reduce risk in rewards through the formation of mining pools — where a group of miners give their mining power to a

single pool manager in exchange for more frequent payments. This causes centralization of such PoW blockchains.

Among existing works that address the problem of security in the presence of rational players, we cover the gaps and show that these bounds can be improved. We also show that a higher level of security is achievable through a clever design of the protocol. On the decentralization front, existing work aims to reduce centralization in PoW blockchains through protocol design. However, we propose a change in the class of reward mechanisms used in PoW blockchains that eliminates any incentives of centralization through pool formation.

Overall, this thesis studies and solves the problem of security and decentralization in distributed systems. We identify that accountability and penalty are crucial components in designing such a mechanism, and show that we can improve the adversarial tolerance bounds by designing a consensus/agreement protocol using these two properties, while also guaranteeing security. In decentralization of PoW blockchains, we categorize the Block Reward Mechanisms (BRMs) into two categories — Memoryless and Retentive BRMs. We show that most existing blockchains use Memoryless BRMs and it is impossible to achieve true decentralization using such BRMs. We then propose our own retentive BRM — decentBRM which achieves true decentralization. This proves through construction that true decentralization can be achieved using Retentive BRMs. Our work shows changes in reward/penalty schemes can achieve a higher level of security and decentralization in the presence of rational players. The goal of this thesis is to demonstrate the possibility of solving challenges in cryptographic protocols involving rational players through changes in reward/penalty schemes.

Research Papers

Research Papers part of the thesis

Conference Papers

1. Varul Srivastava, Sujit Gujar. “DecentBRM: Decentralization Through Block Reward Mechanisms” In The 23rd Autonomous Agents and Multi Agent Systems ([AA-MAS 2024](#))
2. Varul Srivastava, Sujit Gujar. “Towards Rational Consensus in Honest Majority” In The 44th IEEE International Conference on Distributed Computing Systems ([IEEE ICDCS 2024](#))

Other Research Papers (not included in the thesis)

Conference Papers

1. Sankarshan Damle, Varul Srivastava, Sujit Gujar. “No Transaction Fees? No Problem! Achieving Fairness in Transaction Fee Mechanism Design”. In The 23rd Autonomous Agents and Multi Agent Systems ([AAMAS 2024](#))
2. Sankarshan Damle, Varul Srivastava, Sujit Gujar. “No Transaction Fees? No Problem! Achieving Fairness in Transaction Fee Mechanism Design” (extension of the previous work). In the 27th European Conference on Artificial Intelligence ([ECAI 2024](#))
3. Shoeb Siddiqui, Varul Srivastava, Raj Maheshwari, Sujit Gujar “Quicksync: A quickly synchronizing pos-based blockchain protocol”. In the 5th IEEE International Conference on Blockchain and Cryptocurrency ([IEEE ICBC 2023](#))
4. Uday Bhaskar, Varul Srivastava, Avyukta Manjunatha Vummintala, Naresh Manwani, Sujit Gujar “Optimal Strategies for Federated Learning Maintaining Client Privacy” ([under review](#)) appeared as poster at Privacy Enhancing Technologies Symposium ([PETS 2024](#))

Workshop Papers

1. Varul Srivastava, Sankarshan Damle, Sujit Gujar “Centralization in Proof of Stake Blockchains: A Game-Theoretic Analysis of Bootstrapping Protocols” In the 6th Games, Agents and Incentives Workshop ([GAIW@AAMAS 2024](#))

Contents

Chapter	Page
1 Introduction	1
1.1 Rationality versus Altruism	3
1.2 Distributed Systems	3
1.3 Challenge 1: Security in Distributed Consensus	5
1.3.1 Types of security threats	6
1.3.1.1 Liveness Attack	6
1.3.1.2 Censorship Attacks	7
1.3.1.3 Forking Attacks	9
1.3.2 Solution: Identification and Accountability	11
1.4 Decentralization of Blockchains	12
1.4.1 Problem: Mining pools and centralization	13
1.4.2 Solution: Change the Incentive Scheme	14
1.5 Thesis Contributions	15
1.6 Outline of the Thesis	16
2 Preliminaries	17
2.1 Consensus and Blockchain	17
2.1.1 Consensus in Distributed Systems	18
2.1.1.1 Byzantine General's Problem	18
2.1.1.2 Rational General's Problem	18
2.1.2 Threat Model	21
2.2 Impossibilities	22
2.3 Cryptographic Preliminaries	24
2.3.1 Modelling the Adversary	24
2.3.2 One Way Functions	26
2.3.3 Signature and Hashing	27
2.3.3.1 Hashing Schemes	27
2.3.3.2 Signature Schemes	28
2.3.4 Merkle Trees	29

2.3.5	Network Preliminaries	29
2.4	Consensus Protocols: A Formal Treatment	30
2.5	Blockchains: Incentives and Randomizations in Consensus	36
2.5.1	Blockchains: A Distributed Ledger	37
2.5.2	Blockchain Protocols	37
2.5.2.1	Proof-of-Work (PoW) Blockchains	37
2.5.2.2	Proof-of-Stake (PoS) Blockchains	41
2.5.2.3	Proof-of-Burn (PoB) Blockchains	41
2.5.2.4	BFT based blockchains	42
2.5.3	Blockchain Properties	42
2.5.3.1	Blockchain Structure	42
2.5.4	Required Properties	43
2.5.4.1	Common Prefix	44
2.5.4.2	Chain Quality	44
2.5.4.3	Chain Growth	45
2.5.4.4	Decentralization	45
2.6	Game Theory Preliminaries	46
2.6.1	Stackelberg Games	46
2.6.2	Incentive Compatibility	47
2.6.2.1	Dominant Strategy Equilibrium	47
2.6.2.2	Nash Equilibrium	48
2.6.2.3	Bayesian Nash Equilibrium	48
2.7	Other related works	49
3	Consensus in the Rational Setting	51
3.1	Introduction	51
3.2	Problem Description	52
3.2.1	Solution Approach	53
3.2.2	Chapter's Contributions	54
3.3	Additional Preliminaries	56
3.3.1	Blockchain and Atomic Broadcast Protocol	56
3.3.1.1	Player Types	56
3.3.2	Flavours of Consensus	57
3.3.2.1	Byzantine Broadcast & Agreement	57
3.3.2.2	Atomic Broadcast	57
3.3.2.3	Rational Consensus	57
3.3.3	Cryptographic and Network Preliminaries	59
3.3.3.1	Trusted Setup	59
3.3.3.2	Network Models	59
3.3.4	Baiting-Based Consensus Protocols	59
3.4	Our Model	60

3.4.1	The Game	60
3.4.1.1	Player Categories	60
3.4.1.1.1	System States	61
3.4.1.1.2	Player Types	61
3.4.1.2	Utility Structure	62
3.4.1.2.1	Strategy Space	62
3.4.1.2.2	Penalty	62
3.4.2	Threat Model	63
3.4.3	Equilibrium & Incentive Compatibility	64
3.4.3.0.1	Nash Equilibrium (NE)	64
3.4.3.0.2	Focal Point	64
3.4.3.0.3	Challenges with multiple Nash Equilibria in a Security Game	65
3.4.3.0.4	Dominant Strategy Equilibrium (DSE).	65
3.5	Impossibilities	66
3.6	pRFT: Rational Consensus Protocol	71
3.6.1	Protocol	71
3.6.1.1	Propose Phase	72
3.6.1.2	Vote Phase	72
3.6.1.3	Commit Phase	73
3.6.1.4	Reveal Phase	73
3.6.2	View Change	73
3.6.3	Discussion	75
3.6.3.1	Accountability and Proof-of-Fraud (PoF)	76
3.6.3.2	Tentative and Final Consensus	76
3.6.3.2.1	Effectiveness of Tentative Consensus	76
3.6.3.3	Message Complexity	77
3.7	Theoretical Analysis of pRFT	81
3.8	Conclusion	84
4	Study of Centralization in PoW Blockchains	85
4.1	Introduction	86
4.2	Related Work	90
4.3	Preliminaries	91
4.3.1	Blockchain Preliminaries	92
4.3.1.1	Proof-of-Work (PoW)	92
4.3.1.2	Block Reward Mechanism	92
4.3.2	Mining Pools	93
4.3.2.1	Message Distribution	94
4.3.2.2	Reward Sharing Scheme [154]	94
4.4	The Game	95

4.4.1	Players	95
4.4.1.1	Risk Tolerance	96
4.4.1.2	Strategy Space and Environment	96
4.4.1.2.1	Discrete Strategy Space	97
4.4.2	Utility of a Player	97
4.4.2.1	Message Distribution	98
4.4.2.1.1	Pool Hopping / Switching Cost	98
4.4.3	Game Progression and Properties	99
4.4.3.1	Game	99
4.4.3.2	Fairness	100
4.4.3.3	Decentralization	102
4.5	Theoretical Analysis	104
4.5.1	Memoryless BRMs ($\Gamma_{memoryless}$)	104
4.5.2	Retentive BRMs ($\Gamma_{fruitchain}$)	109
4.5.2.1	Fruitchains [162]	110
4.5.2.2	Game Formulation	110
4.5.2.3	Analysis	111
4.6	decentBRM : Optimal BRM	114
4.6.1	decentBRM	114
4.6.2	Game Formulation	115
4.6.3	Analysis	115
4.7	Conclusion	117
5	Conclusion and Future Work	119
5.1	Summary of Contributions	119
5.2	Future Directions	120

List of Figures

Figure		Page
1.1	Rational Threat Model	4
1.2	Real-world applications of Distributed Systems where Liveness property is a critical requirement for proper functioning and avoiding critical failures. .	6
1.3	Centralization due to Mining pools	14
2.1	Byzantine Generals Problem	19
2.2	Rational Generals Problem	19
2.3	Types of players based on their capacity to deviate from the protocol	25
2.4	Ideal functionality of State Machine	31
2.5	Random Oracle for PoW	38
2.6	Block Structure	43
3.1	pRFT Protocol	78
3.2	Normal execution of pRFT protocol	79
3.3	Messages sent in pRFT	79
3.4	Construction of Proof-of-Fraud (PoF)	80
3.5	Message Complexity for different consensus protocols compared with pRFT	80
4.1	Centralization in Bitcoin over 4 years [172]	89
4.2	Bitcoin Hashrate Distribution	90
4.3	Bounds on switching cost parameter for $\Gamma_{memoryless}$	109

List of Tables

Table		Page
2.1	Results in different consensus protocols	22
2.2	Comparison of Consensus Protocols under various communication and threat models.	23
3.1	Bounds for consensus in different threat models.	54
3.2	Payoff function $f(\sigma, \theta)$	61
3.3	Example of two equilibria in a 3-player game (Here, utility is in the order (A, a, α))	65

Chapter 1

Introduction

“It is not from the benevolence of the butcher, the brewer, or the baker that we expect our dinner, but from their regard to their own interest.”
— Adam Smith, *The Wealth of Nations* (1776)

A system of independent and individual players should not rely on the hopes of cooperation and goodwill, but rather the belief that individuals act to maximize their self-interest. This idea, borrowed from the *father of capitalism* has been almost as old as the idea of community. A system of independently thinking individuals function properly as a society to achieve a common goal if the mechanism is designed so that self-interest is maximized by following the actions which achieve the common societal goal (thereby also maximizing collective social utility). This challenge is sometimes known as protocol design for rational players or — rational protocol design.

Example 1.1: The Burning Building

Imagine a high-rise building. There are two evacuation systems:

- **Altruism-Based System:** Everyone is told, “Just help each other. Be kind. Stick together.”
- **Rationality-Based System:** Everyone is told, “Follow the fire evacuation plan. Take the marked routes. Don’t use elevators. Prioritize speed and spacing.”

Normal Day

- The altruistic system feels great. People help elders down the stairs. They chat. No pressure. Everyone gets out, eventually.
- The rational system feels cold — people move with purpose, don’t talk, follow orders. It works, but doesn’t feel as human. Result? Both systems “work” when it doesn’t matter.

Crisis Day The building is on fire, smoke is thick, and seconds count.

- In the altruistic system, people hesitate: “Should I go back for someone?... You go first — no, you”. Emotional decisions. Confusion. Delays. People die because well-meaning choices become deadly mistakes.
- In the rational system: People follow the plan. No guilt, no hesitation. The system is built to optimize for survival — not feelings. Hence, rationality saves more lives, even if it looks less compassionate in the moment.

1.1 Rationality versus Altruism

As demonstrated through Example 1.1, relying on altruism results in failure of expected outcome from the system. Therefore, incentive based protocols should be set in place and reliance on rationality is essential for a functional system. Similar to this, incentive based elicitation of altruistic behavior is achieved through Mechanism Design to ensure alignment of incentives with altruistic behavior — commonly called as *Incentive Compatibility*. Through this example, we have demonstrated the necessity of reducing reliance on the altruism of players and treating them as rational for a more functional system.

In this thesis, we study one particular system — distributed system of agents which are instructed to follow a protocol. In such a case, traditional modeling of agents is either altruistic or byzantine. Both of them have fixed behavior and cannot be deviated through incentives. However, an actual system has rational players who follow “profit-maximizing strategies”. This thesis aims to study consensus protocols (used in blockchains) under the presence of Rational players and aims to use mechanism design to propose protocols which guarantee security and decentralization. We briefly explain distributed systems, after which we discuss the two problems — security and decentralization; which we address under our more general and realistic adversary modelling (which includes all — altruistic, rational and byzantine adversaries).

1.2 Distributed Systems

Distributed consensus is a problem of reaching agreement among multiple players on a single value among one (or more) alternatives. Historically, mechanisms/protocols which solve this problem have considered two types of players — altruistic (complacent) and byzantine (maximally disruptive) [1, 2]. The currently most secure result for solving this problem in a system of n players require $> \frac{2n}{3}$ to be altruistic players in partially synchronous setting. This problem has found immense application in the field of cloud-

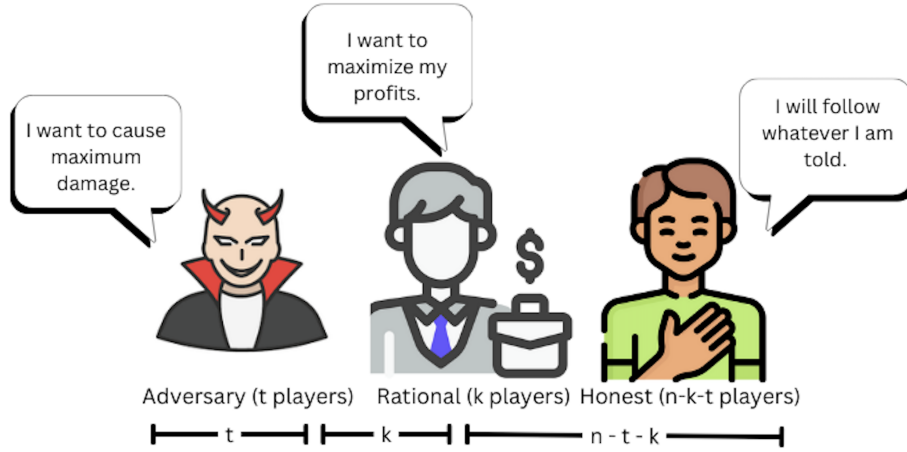


Figure 1.1: Rational Threat Model

computing and storage — where same data and transactions are replicated on servers across the globe for (1) avoiding single point of failure and (2) faster access and higher uptime. In addition, more recent application includes to blockchains, a distributed ledger of transactions which simulate a trustless virtual machine [3] and is most popularly known by decentralized payment networks such as Bitcoin [4] and Ethereum [5].

These systems are maintained by multiple (n) participants, each of which fall into one of three sets:

- *Byzantine* — Adversarial set of t players which aim to maximally disrupt the system irrespective of their individual utility in doing so.
- *Altruistic* — Honest set of h players which aim to follow the protocol as long as participating is incentivized¹.
- *Rational* — Rational set of k players which aim to follow the *utility maximizing strategy* irrespective of it's impact on the system.

¹this condition is popularly known as Individual Rationality in Mechanism Design literature

Distributed systems face three major challenges (called the trilemma) — satisfying *decentralization, security and scalability*. However, traditional works such as [6, 7] focus on systems with byzantine and altruistic players. We generalize the system to include rational players; and propose incentive-driven protocols to improve the security and decentralization of such systems. Before going into the solutions, we briefly discuss the problems of security and decentralization in distributed systems.

1.3 Challenge 1: Security in Distributed Consensus

Distributed consensus protocols for partially synchronous network models achieve consensus when over two-third majority is altruistic. Although such an assumption was mildly realistic for distributed servers [8], the problem becomes much more severe for public distributed systems (where anyone can join).

“Public Distributed Systems face the challenge of reducing their reliance on altruistic players to avoid system failure.”

Previous works which have reduced this reliance on altruistic players popularly include Bitcoin [9] which reduces this threshold to $\frac{n}{2}$. However, they rely on strong network assumptions and achieve *extremely* low transaction throughput. Further, their solution is *probabilistic consensus* i.e. agreement is finalized with certain probability at any point. Multiple attacks have been identified on such probabilistic consensus schemes [10, 11, 12]. In addition to being susceptible to such attacks, there is also a general lack of *accountability* i.e. on identification of an attack, pointing out the attacker is not possible. Existing approaches to solve the problem for deterministic consensus protocol in partially synchronous settings [13] have the problem of focal equilibria — existence of multiple utility maximizing points some of which are adversarial.

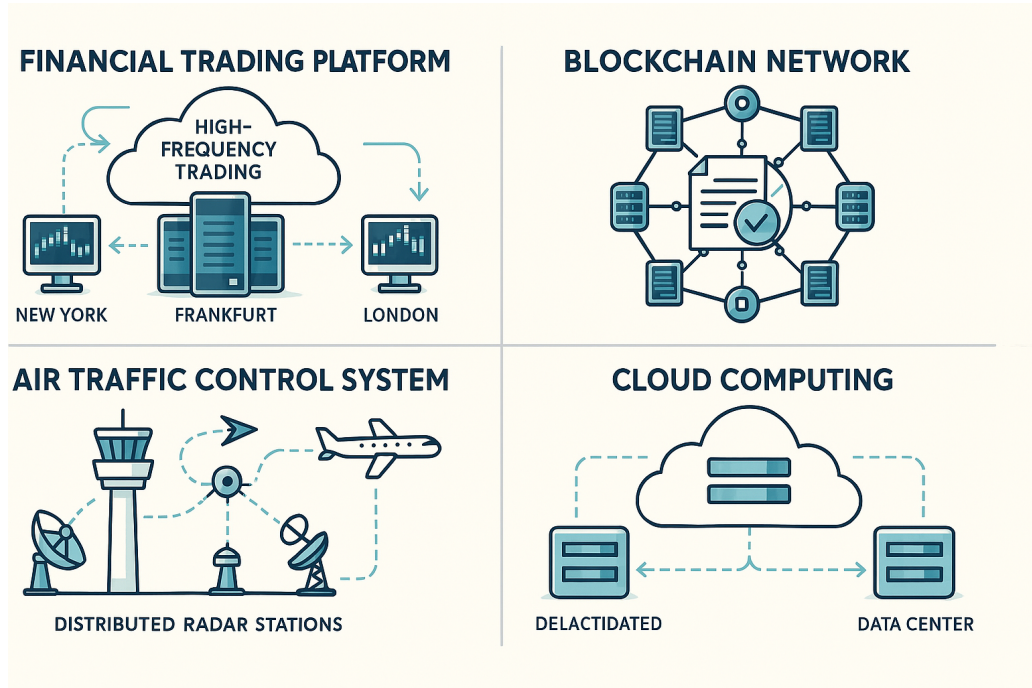


Figure 1.2: Real-world applications of Distributed Systems where Liveness property is a critical requirement for proper functioning and avoiding critical failures.

1.3.1 Types of security threats

Distributed systems face various types of security threats, each influencing one aspect of the system. These threats are broadly classified into: (1) Liveness attack, (2) Censorship attack, and (3) Forking attack.

1.3.1.1 Liveness Attack

Liveness-critical distributed systems are those where the system's ability to make progress and respond to inputs in a timely manner is essential. In such systems, liveness properties ensure that operations eventually complete, maintaining the system's responsiveness and availability. As depicted in Figure 1.2, the examples include:

1. **Financial Trading Platforms:** Systems like the New York Stock Exchange (NYSE) rely on distributed architectures to process and disseminate trade information. Any disruption can lead to significant financial losses and market instability.
2. **Air Traffic Control Systems:** Distributed systems manage real-time aircraft tracking and routing. The French Air Traffic Control System, for instance, uses such systems to ensure continuous and reliable communication.
3. **Cloud Computing Services:** Platforms like Amazon Web Services (AWS) provide distributed computing resources. Ensuring liveness is crucial to maintain service availability and meet service-level agreements.
4. **Blockchain Networks:** Cryptocurrencies like Bitcoin and platforms like Ethereum depend on distributed consensus protocols to validate transactions. Liveness ensures that transactions are processed without undue delay.

Liveness attacks disrupt the normal operation of distributed systems, leading to delays or complete service outages. Notable instances include:

1. **Distributed Denial-of-Service (DDoS) Attacks:** These attacks overwhelm systems with excessive requests, hindering legitimate operations. For example, in February 2022, Ukraine experienced its largest DDoS attack, disrupting government and financial services.
2. **Sybil Attacks on the Tor Network (2014):** Malicious entities created numerous fake identities within the Tor network to deanonymize users, compromising the network's integrity and user privacy.

1.3.1.2 Censorship Attacks

The redeeming quality of distributed systems has been their decentralization and non-controlling nature. For example, international payment systems like Swift [14] face not only

single points of failure, but also censorships like recently faced by Russia and Belarus over their international policies. However, there are instances where blockchains and distributed systems have also been susceptible to censorship attacks.

- 1 **Selective Transaction Censorship:** Miners may refuse to include transactions to comply with external sanctions or bias, leading to delayed or omitted transactions. After the 2022 OFAC sanctions, 46% of Ethereum blocks were produced by actors deliberately censoring transactions [15].
- 2 **Eclipse and Network Partitioning Attacks:** Adversaries can monopolize all peer connections of a node (eclipse), or stealthily partition the network by influencing peering decisions (EREBUS), isolating victims to censor or manipulate their view of the ledger [16, 17].
- 3 **DHT-Based Content Censorship:** In peer-to-peer storage networks like IPFS, attackers can poison the Distributed Hash Table to blacklist specific content identifiers, preventing retrieval of censored data [18].
- 4 **Bribery-Based Blacklisting:** On-chain bribery contracts incentivize miners to temporarily blacklist a targeted account’s transactions without any direct mining power, exploiting game-theoretic incentives [19].
- 5 **Regulatory Sanctions and Compliance Censorship:** Legal sanctions against protocols (e.g., Mixer bans) force nodes and service providers to blacklist smart contracts or addresses to avoid penalties, effectively censoring sanctioned entities [15].
- 6 **Smart Contract-Level Censorship:** In rollups and layer-2 designs, adversaries can censor challenge or dispute transactions by blocking inclusion until after deadlines, delaying or nullifying fraud proofs [20].

7 Routing-Level Hijacking Censorship: Network-level attackers hijack BGP prefixes to reroute or intercept blockchain traffic, blacklisting nodes by filtering or dropping their messages en route [21].

There has been little work to address censorship challenges in blockchains. However, frameworks like Tithonus embed covert communications into the blockchain gossip layer, achieving two orders of magnitude lower cost for censorship-resilient data transfer than on-chain writing [22]. This thesis studies the problem of censorship in distributed systems.

1.3.1.3 Forking Attacks

Forking is when two players / systems in a distributed system network has different image of the system state. Forking attacks are consistency violations where a malicious server creates multiple instances of a trusted execution context (TEE) and supplies each with a valid but divergent state, causing different clients to observe inconsistent operation histories [23, 24]. These attacks generalize rollback attacks by allowing an adversary to run multiple concurrent epochs of the context, thereby violating state continuity and causing responses from different instances to diverge [23]. Because TEEs cannot verify freshness or the number of instances, they are inherently vulnerable to such attacks unless additional mechanisms such as trusted monotonic counters or client-to-client synchronization (fork-linearizability) are employed [23, 24].

In blockchain networks, forking attacks manifest as adversarially induced chain splits, where attackers intentionally create alternative branches to manipulate consensus, execute double-spend attacks, or undermine trust in the network [25, 26]. These attacks exploit the decentralized nature of consensus algorithms and the longest-chain rule, which selects the branch with the most accumulated work or stake as the valid chain, allowing attackers to outpace honest participants under certain conditions [25, 26].

Some notable instances of such forking attacks in distributed systems and blockchains, along with their impact are as provided below. This also explains the criticality of the attack and the massive disruptions it has caused in the system historically.

- 1 **Selfish mining (2013):** Colluding miners can increase their revenue by withholding newly found blocks and selectively releasing them, causing rational miners to join the selfish pool and undermining Bitcoin’s decentralization [27]
- 2 **Feather forking (2013):** Miners threaten to orphan any block containing black-listed transactions by refusing to mine on chains that include them, enforcing censorship with less than 50% hash power [28]
- 3 **TEE forking formalization (2017):** Brandenburger et al. [23] introduced rollback and forking attacks on Intel-SGX, showing that a malicious host can instantiate multiple enclave epochs to feed divergent states to clients, and proposed Lightweight Collective Memory to detect such forks [23]
- 4 **Ethereum Classic 51% double-spend (January 2019):** Attackers reorganized the chain in a deep reorganization, double-spending ≈ 219500 ETC ($\approx \$1.1M$) by renting hash power [29]
- 5 **Another Ethereum Classic 51% attack (July–August 2020):** Two major reorganizations (3693 and 4000 blocks) led to losses over $\$5M$ and prompted exchanges to pause ETC deposits [30, 31]
- 6 **Empirical SGX forking attacks (2023):** Wilde et al. [32] demonstrated that real-world Intel-SGX applications are susceptible to forking and rollback attacks by launching multiple enclave instances or feeding stale states [32]
- 7 **CloneBuster — detecting enclave-cloning forks (2023):** Briongos et al. [33] presented CloneBuster, a mechanism to detect cloning-based forking attacks on SGX applications via a cache-based covert channel [33]

8 **Bribe & fork in payment channel networks (2024):** Avarikioti et al. [34] introduced a bribery-based forking threat that reduces the cost of censoring transactions in PCNs to $\approx \$125$ by leveraging feather-fork threats [34]

1.3.2 Solution: Identification and Accountability

We see in this thesis that for the problem of agreement / consensus, reliance on the altruism of players can be shifted to rational players if we can make the protocol detect attacks and introduce accountability.

We address this challenge in this thesis, through our work [35] which improves the threshold by reducing requirement of $> \frac{2n}{3}$ players to be rational to $> \frac{n}{2}$. Therefore, for agreement protocols, as long as majority is honest and majority of attackers is rational, agreement can be achieved under certain assumptions on the utility structure of the rational players.

We solve this problem by using a protocol which can identify when system is not in correct state (there is an attack) — called *identification* property. Using this with a penalty mechanism which penalizes players responsible (captured through *accountability* property) for deviations, we are able to prohibit rational players deviating from the protocol. Such a construction is generalizable to build a class of **“rationally secure accountable protocols”**. We demonstrate the sufficiency of the identification and accountability through Example 1.2 below.

Example 1.2: Cops and Criminals

Imagine a society where each crime is *identified* by the cops. Furthermore, for any crime a non-zero subset of parties to be held *accountable* are apprehended. In such a society, the following features are observed:

- 1 No *crime will go unpunished*. If a crime is committed, a combination of identification and accountability will result in apprehension. Furthermore, no innocent citizen will be apprehended for the crimes of a criminal.
- 2 More interestingly, *no criminal will commit crime*. Given the guarantee of apprehension, any rational criminal will behave honestly.

Clearly, any rational players capable of deviations will be strictly disincentivized against it, if the protocol satisfies the properties of identification and accountability. We do so by proposing pRFT, and discuss this in detail in Chapter 3. The second head of the three-headed dragon is decentralization. In the general sense of the word, decentralization is the prevention of the accumulation of resources (money, power or influence) in the hands of a selective set of players, disproportionately. We describe the problem of centralization in distributed systems (focusing on blockchains) below.

1.4 Decentralization of Blockchains

Another challenge is the struggle for control of distributed systems. Consensus algorithm by design run on *whoever controls the majority, controls the system*. Majority here refers to a “resource” used by consensus protocol. For example, in Proof-of-Work (PoW) blockchains, this refers to mining/compute power. Similarly, in Proof-of-Stake blockchains this refers to Stake or money in circulation in the system.

A natural motivation for an individual player would be to increase their influence over the system — thereby making the system more centralized. However, this brings a unique

challenge of *single point of failures*, where critical players deviating or unavailability can disrupt the system. Further, these critical players can influence decisions unilaterally, thereby causing censorships, forking etc.

Decentralization redistributes control across network participants, enhancing security, transparency, and autonomy by eliminating reliance on a central authority [36, 37]. In proof-of-work blockchains, decentralization ensures that no single entity can manipulate consensus by controlling a majority of the hash rate—an attack vector that would enable censorship or double-spend exploits and undermine the trustless, permissionless nature of the system [27, 38]. Historical incidents such as the GHash.io pool briefly commanding over 51% of Bitcoin’s hashing power in June 2014, and the January 2019 51% double-spend attacks on Ethereum Classic, demonstrate the real-world dangers of concentration of power in ostensibly decentralized networks [39, 40].

Mining pools were introduced to reduce reward variance for individual miners by aggregating hashing power and distributing payouts proportionally, thus stabilizing income without affecting expected returns [41, 42]. However, the rise of a few dominant pools poses a significant threat to decentralization: large pools can collude to censor transactions, engage in selfish-mining strategies, or even coordinate 51% attacks, undermining network security and fairness [27, 42]. To counter these risks, decentralized pool protocols such as P2Pool and enhanced communication standards like Stratum-V2 empower miners with greater autonomy over block construction and governance, helping to restore the balance of power in proof-of-work ecosystems [43, 44].

1.4.1 Problem: Mining pools and centralization

In PoW blockchains, maintaining most recent system state, mempool transactions as well history of the blockchain is both network-bandwidth and compute intensive process. Therefore, solo-miners usually tend to join mining facilities called *mining pools* which provide them all — verification, mempool and other facilities in exchange for their compute.

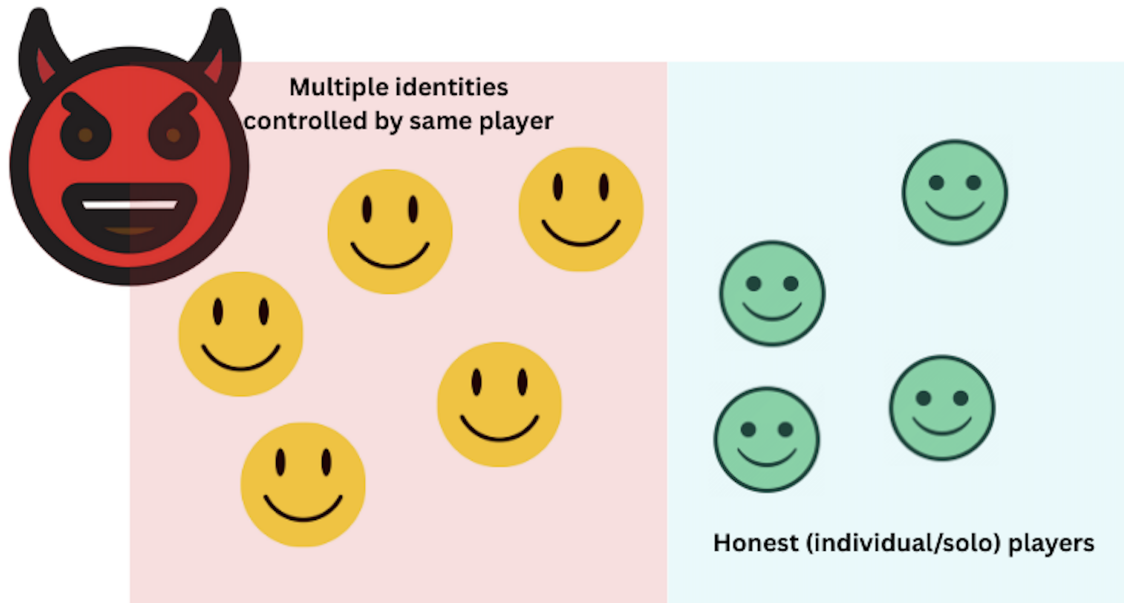


Figure 1.3: Centralization due to Mining pools

It is like a partnership business model, where each partner / solo miner of the business / mining pool contributes some amount (compute) to the system. If a block is mined, the reward is distributed proportionally among the partners / miners according to their contributed mining power.

While this system is convenient and allows miners to join the system at a *lesser bootstrap cost*; there is an inherent challenge of centralization; because the block is proposed by the mining pool manager, who thereby holds high compute power to decide which transactions can/cannot be included in the chain. Additionally, if one or more of major mining pool deviates, that can potentially cause disruption of service or even security threats.

1.4.2 Solution: Change the Incentive Scheme

This challenge to decentralization in of PoW blockchains goes against the idea of rationality. For an individual miner, the expected reward in joining the mining pool is the same,

however, the variance (signifying risk) is much less in joining mining pools than mining solo [45]. Therefore, *mining pools reduce risk at the cost of centralization* for solo miners.

To solve a problem where the goal goes opposite to the incentive scheme, the solution often is to reverse the incentive scheme. So is the case with our proposal [46] of **decent-BRM** protocol reduces the gap of risk between solo-mining and joining mining pools by changing the block reward incentive scheme.

1.5 Thesis Contributions

We discuss the role of rational protocol design and incentive schemes in both *security* and *decentralization* of distributed systems.

- We propose *pRFT* — a distributed consensus protocol under Rational threat model which achieves consensus under honest majority; under the condition that majority adversary is rational. This bound is an improvement over previous secure protocols which required adversaries $< \frac{n}{3}$. However, pRFT guarantees security and liveness under certain types of utility functions for $< \frac{n}{2}$ adversaries. Through this protocol, we address the problem of security under rational players in distributed systems and improve the bounds on adversarial tolerance.
- We also propose *decentBRM* — a decentralized block reward mechanism for Proof-of-Work (PoW) blockchains. DecentBRM reduces risk of solo mining to the extent that it becomes equivalent to joining mining pools. Under this reduced risk, a miner is incentivized to perform solo mining due to marginal advantage in utility over joining mining pools. Through decentBRM, we aim to solve the problem of centralization in PoW blockchains caused due to existence of mining pools.

1.6 Outline of the Thesis

This thesis first discusses the relevant technical background on Distributed Systems, Blockchains and Networks following which it discuss relevant background on Game Theory. Following this, the thesis discusses the problem of rational consensus in honest majority and then decentralization in Proof-of-Work blockchains using mechanism design. Finally, we conclude this thesis summarizing the major takeaways and potential future extensions.

- Chapter 2 discusses the related background literature on Distributed Systems, Blockchains, Networks and Game Theory.
- Chapter 3 discusses the problem of security in distributed systems. In this chapter, we introduce pRFT which solves the problem of Rational Consensus with honest majority $t + k < \frac{n}{2}, t < k$.
- Chapter 4 addresses the problem of centralization in Proof-of-Work (PoW) blockchains due to mining pools. In this chapter we introduce decentBRM to solve the problem of centralization through Block-Reward Mechanism Design.
- We conclude the thesis with Chapter 5, discussing major takeaways and potential future directions which can be pursued through contributions of this thesis.

Chapter 2

Preliminaries

In this chapter, we go through the background required to this thesis. We first discuss the literature on consensus and blockchains followed by the Game Theoretic preliminaries required to model incentive-driven players in the distributed system setting. We also go through some popular centralization metrics employed in quantification of Network/Blockchain centralization. Finally, we look at some existing work related to our contributions.

2.1 Consensus and Blockchain

Consensus is the problem of achieving agreement among different players in a distributed setting. The goals of consensus are *consistency* — all non-adversarial players agree on the same value and *liveness* — the system reaches agreement and protocol terminates. Blockchains are distributed ledgers which store state-transitions in the form of transactions in a block. Blockchains employ consensus protocols to achieve agreement on the next block in the chain. This ensures consistency across the network on the state of the blockchain. In this section, we first look at the cryptographic preliminaries employed in consensus protocols used in blockchains. Following this, we explore the traditional problem of consensus in distributed systems. We then study how consensus can be achieved

in a scalable public network, i.e. blockchains. Finally, we go through properties of the blockchain system and the network model.

2.1.1 Consensus in Distributed Systems

We now look at the problem of consensus in distributed system. This problem has different variants based on the nature of the adversary, each motivated by a variant of the byzantine generals' problem. We will first discuss the different variants of the problem and corresponding challenges and broader impossibilities. Following this we discuss protocols which are used to achieve consensus in this setting.

2.1.1.1 Byzantine General's Problem

Traditionally, the Generals' problem has been used to motivate the problem statement. We motivate Byzantine Agreement and Rational Agreement problems through the corresponding two variants of the problem. A protocol which solves the Byzantine Generals Problem also solves Byzantine Agreement. Another variant of this is if the adversarial generals can abort the campaign at any given time (crash). Agreement in this (weaker) adversary model of only Crash Faults is called Crash Fault Tolerant Agreement.

2.1.1.2 Rational General's Problem

Another form of agreement is called Rational Agreement. This is when in addition to Byzantine Generals, there is also presence of Rational Generals. Both these generals can deviate from following the agreement honestly, but Rational generals deviate only if they are incentivized to do so. Their incentives and utility function depends on their types and valuations. This variant of the problem is called Rational Generals' Problem.

The Byzantine Adversaries are stronger as they follow a strategy irrespective of it's utility. However, Rational players are weaker form of adversaries because they are incentive

Byzantine Generals' Problem [47]

Setting: Consider n Ottoman generals want to decide if they should attack or not attack the kingdom of Byzantine. However, an offensive will be successful only if $> \frac{2n}{3}$ generals attack, otherwise the offensive fails. The goal is for this majority $> \frac{2n}{3}$ to agree on same value **Attack/Retreat**.

Adversary: Consider there are $t \leq t_0$ generals who are adversarial. These adversarial generals can disrupt communication (delay message passing among other generals), send conflicting messages about their own decision to different generals. This variant of the problem is known as the *Byzantine Generals' Problem*.

Figure 2.1: Byzantine Generals Problem

Rational Generals' Problem [13]

Setting: Consider n Ottoman generals want to decide if they should attack or not attack the kingdom of Byzantine. However, an offensive will be successful only if majority $> \frac{n}{2}$ generals attack, otherwise the offensive fails. The goal is for this majority $> \frac{n}{2}$ to agree on same value **Attack/Retreat**.

Adversary: Consider there are $t \leq t_0$ generals who are adversarial of the byzantine type and k generals are rational. These generals can disrupt communication (delay message passing among other generals), send conflicting messages about their own decision to different generals. However, the Rational generals will deviate only if they obtain higher utility from deviating than following the agreement protocol.

This variant of the problem is known as the *Rational Generals' Problem*, and a protocol which obtains agreement in such a threat model achieves Rational Agreement.

Figure 2.2: Rational Generals Problem

driven. They will follow the protocol honestly, unless there exists a deviation which gives them higher payoff according to their utility function.

Definition 2.1: Byzantine Agreement (BA)

A protocol Π is said to achieve Byzantine Agreement among the set of players $P = \{p_1, p_2, \dots, p_n\}$ ($|P| = n$) with set of byzantine adversaries $T \subset P$ ($|T| \leq t_0$) and set H of honest players ($H = P \setminus T$) if the following conditions are satisfied:

1. All honest players reach agreement on a common value v .
2. If each honest player starts with the same input value v_{in} then they reach agreement on the same value v_{in} .
3. All honest players reach on agreement (i.e. protocol Π terminates).

Definition 2.2: Rational Agreement (RA)

A protocol Π among a set of players $P = \{p_1, p_2, \dots, p_n\}$ ($|P| = n$) with set K ($|K| = k$) of rational players, T ($|T| = t \leq t_0$) byzantine players and H ($|H| = h$) altruistic players is said to achieve Rational Agreement if:

1. All honest players reach agreement on common value v
2. If each honest player starts with some input value v_{in} then they reach agreement on the same value v_{in} .
3. All honest players reach agreement (protocol Π eventually terminates).

Agreement is also called Authenticated Agreement if the players are computationally bounded and we can make use of PKI and Digital Signatures.

2.1.2 Threat Model

The problem of agreement is discussed in context with a threat model. This entails details about the set of players, and the types of adversary¹. Based on that, we define three threat models as below:

- **Crash Fault Tolerant** $CFT(n, t)$ is a threat model with n players out of which t are faulty. These faulty/adversarial players can crash (i.e. stop responding to messages in the protocol) at any time during the protocol execution. Remaining $n - t$ players are altruistic and follow the protocol honestly.
- **Byzantine Fault Tolerant** $BFT(n, t)$ is a threat model with n players out of which t are adversarial. These are byzantine players and can cause disruption by following arbitrary strategy.
- **Rational Fault Tolerant** $RFT(n, k, t)$ is a threat model where out of total n players, some t are adversaries of the byzantine type; i.e. cause arbitrary disruption to the protocol. Another k players are of rational type; i.e. deviate from the protocol only if they have incentive to do so. Remaining $n - k - t$ players are altruistic and follow the protocol honestly.

For the same value of t , we notice that $CFT(n, t) \subset BFT(n, t) \subseteq RFT(n, k, t)$. In addition, we also make a simple observation that $RFT(n, k, t) \subseteq BFT(n, k + t)$ due to the more general nature of byzantine adversary than rational. We summarize the existing results on consensus protocols in Table 2.2.

¹We assume Dolev-Yao [48] level of network access to the adversary through backdoor access to the Network Simulator.

Network	Threat Model			
	$CFT(n, c)$		$BFT(n, t)$	
	Unbounded	Poly-time	Unbounded	Poly-time
Synchronous	$t < n$	$t < n$	$t < \frac{n}{3}$	$t < n$
Partially-Synchronous	$t < \frac{n}{2}$	$t < \frac{n}{2}$	$t < \frac{n}{3}$	$t < \frac{n}{3}$
Asynchronous	$t < \frac{n}{2}$	$t < \frac{n}{2}$	$t < \frac{n}{3}^\dagger$	$t < \frac{n}{3}^\dagger$

† : Randomized Consensus Protocols

Table 2.1: Results in different consensus protocols

2.2 Impossibilities

Having understood the Byzantine Generals' problem, we now look at some impossibility results for agreement under different model assumptions and different fraction of adversarial control over the players.

Under synchronous network model, consensus is impossible for $t_0 \geq \frac{n}{3}$ computationally unbounded byzantine adversaries [49]. The proof trivially follows from the fact that a case with t_0 adversaries and $n - t_0$ adversaries is indistinguishable in the synchronous setting. However, if the players are computationally bounded, agreement is possible for any $t_0 < n$ in $CFT(n, t_0)$ threat model and for any $t_0 < \frac{n}{2}$ for any $BFT(n, t_0)$ threat model.

For asynchronous networks and computationally unbounded adversary, the famous impossibility by Fischer et al. [61] which states that there is no deterministic protocol which achieves Distributed Consensus if there is at least one faulty process (crash fault or byzantine fault). Randomized consensus protocols [62, 56, 63] circumvent this impossibility. However despite randomized protocols, consensus is impossible for $t_0 \geq \frac{n}{3}$.

Dwork et al. [64] stated that consensus is impossible for $t_0 \geq \frac{n}{3}$ in partially synchronous setting for $BFT(n, t_0)$ threat model. Fox et al. [65] proposed the CAP theorem through which consensus with $t_0 \geq \frac{n}{2}$ is impossible in $CFT(n, t_0)$ threat model in partially syn-

Work	Threat Model	Network Model	Security	Communication
Lamport et al. [50]	$BFT(n, t)$	Synchronous	$t < n/3$	$O(n^{t+1})$
Dolev et al. [51]	$BFT(n, t)^\dagger$	Synchronous	$t < n^\dagger$	$O(n \cdot t)$
Castro et al. [52]	$BFT(n, t)$	Partial-Sync	$t < n/3$	$O(n^2)$
Yin et al. [53]	$BFT(n, t)$	Partial-Sync	$t < n/3$	$O(n)$
Gilad et al. [54]	$BFT(n, t)$	Partial-Sync	$t < n/3$	$O(n)$
David et al. [55]	$BFT(n, t)$	Partial-Sync	$t < n/3$	$O(n^2)$
Miller et al. [56]	$BFT(n, t)$	Asynchronous	$t < n/3$	$O(n^2)$
Amoussou et al. [57]	$RFT(n, k, t)$	Synchronous	$t < \kappa < k + 1$	$O(n^2)$
Abraham et al. [58]	$RFT(n, k, t)$	Partial-Sync	$k + t < 1/3$	$O(n^2)$
Eyal et al. [59]	$RFT(n, k, 0)$	Partial-Sync	$k < n/2$	$O(n)$
Aiyer et al. [60]	$RFT(n, k, t)^\wp$	Asynchronous	$t < (n - 2)/3$	$O(n^2)$

$\dagger$: authenticated / with signatures

$\wp$: no collusion between rational players

κ : quorum threshold

Table 2.2: Comparison of Consensus Protocols under various communication and threat models.

chronous network. We tabulate the bounds on consensus protocols in $CFT(n, c)$ and $BFT(n, t)$ threat model under different settings in Table 2.1.

2.3 Cryptographic Preliminaries

We explore the cryptographic preliminaries which are required for our work. Throughout the thesis, we assume each player is polynomially bounded. This means that any player can be simulated through a polynomial time turing machine. In addition, all security guarantees we provide under this setting further relies on the assumption $P \neq NP$; that is, the class of algorithms simulated by a Polynomial Time turing machine is a strict subset of the class of algorithms simulated by a Non-Deterministic Polynomial time algorithm.

2.3.1 Modelling the Adversary

We model adversaries based on three broad categories. First is their capacity of deviating from the protocol, based on which they can be classified into five types of players as depicted in Figure 2.3 and described below.²

- **Honest** players follow the prescribed protocol Π without any deviation.
- **Semi-Honest** players also follow the protocol Π but the adversary can learn all inferrable information from this player’s “view”. Such players are also called *honest-but-curious* or *passive*. They are similar to honest players when we consider correctness of the protocol.
- **Crash** players can stop participating in the protocol (message passing and any other obligation) at any time during the protocol execution.
- **Rational** players may deviate from the protocol to follow any strategy conditioned that the new strategy should be incentivized over the current (honest) strategy.

²Note that in Figure 2.3 $\text{Semi-Honest} \subseteq \text{Crash}$ assuming Crash Turing Machines can also read the messages sent through the (same) network simulator. However, this is not true in general because Semi-Honest Turing Machines can read the messages while Crash Turing Machines cannot do so. For the purpose of this thesis, this fact is unessential and therefore for the sake of simplicity we assume $\text{Semi-Honest} \subseteq \text{Crash}$.

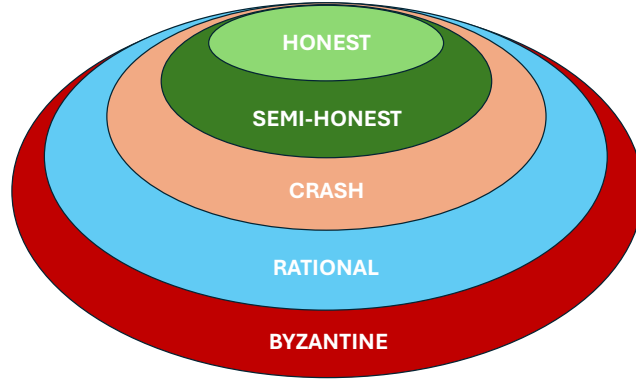


Figure 2.3: Types of players based on their capacity to deviate from the protocol

- **Byzantine** players deviate arbitrarily from the protocol and may cause disruption indifferent to their individual utility from following any given strategy.

Players are also categorized based on their computational power into three broad types.

- **Unbounded** players are such that they can follow any strategy which can be simulated by any Turing Machine. Security guarantees against such adversaries can be of the type *perfect security* or *statistical security*. Such adversaries are not bounded by any cryptographic or complexity-theoretic assumptions.
- **Computationally Bounded** players follow any strategy which can be simulated by probabilistic polynomial time turing machine (PPTM). Such players can generally simulate any strategy present in complexity class BPP but are assumed unable to solve NP hard problems (by assuming $BPP \neq NP$).
- **Fractionally Bounded** players can perform finite and bounded number of operations per unit interval. Such a model is used to model hashrate in Proof-of-Work blockchains [66].

2.3.2 One Way Functions

One way functions (OWF) are a function which is computationally easy, but the inverse of this function is computationally hard. If function f is an OWF, then a player which can execute any polynomial time algorithm (belongs to complexity class P) can compute $f(\cdot)$ but cannot compute $f^{-1}(\cdot)$ except with a negligible probability.

Definition 2.3: One Way Functions (OWF) [67]

A function $f : \{0,1\}^* \rightarrow \{0,1\}^\lambda$ is called a one way function if the following two conditions hold:

1. There exists a deterministic polynomial-time algorithm A such that any polynomial time turing machine can compute $f(x)$ by invoking algorithm A on input x .
2. For any algorithm B which can be executed on a polynomial time turing machine, such that input is $f(x)$, the probability of output $\in f^{-1}(f(x))$ is negligible in λ . That is, for any polynomial function $p(\cdot)$

$$Pr[B(f(x), 1^\lambda) \in f^{-1}(f(x))] \leq \frac{1}{p(\lambda)}$$

The parameter λ is called the security parameter of the function f .

One way functions are realized in practicality through hash functions³. These functions give us the following properties, which are of use to ensure security and tamper-proofness in blockchains:

- *Pre-Image Resistance*: Given a binary string $y \in \{0,1\}^\lambda$, it is difficult to compute $x \in \{0,1\}^*$ such that $f(x) = y$.

³There are other methods to realize one way functions such as encryption algorithms, but we focus on Hashing algorithms such as SHA, because they are most widely employed in practise.

- *Second Pre-Image Resistance*: It is computationally difficult to find two strings $x_1, x_2 \in \{0, 1\}^*$ such that $f(x_1) = f(x_2)$.

2.3.3 Signature and Hashing

The construction of digital **signature schemes** from a generic one-way function (OWF) can be achieved in a fully *black-box* manner:

- **Signature Schemes**: First, build *one-time signatures* via Lamport’s scheme by revealing preimages of OWF outputs for each message bit, then aggregate multiple OTS public keys efficiently using a Merkle tree to form a reusable signature framework [68, 69, 70].
- **Hash Functions**: Next, derive *collision-resistant* or *universal one-way* hash functions (UOWHFs) from any OWF via Rompel’s black-box transformation or by employing an iterative Merkle–Damgård construction that embeds OWF calls in the compression step [70, 71, 72, 73].

Throughout, the OWF acts as an opaque oracle whose inversion hardness underpins both signature unforgeability and hash collision resistance, abstracting away implementation details while preserving provable security [74, 75].

2.3.3.1 Hashing Schemes

- **SHA-256** 256-bit Merkle–Damgård hash used in Bitcoin’s proof-of-work, Merkle trees, and transaction IDs (double-SHA-256) [76, 77].
- **Keccak-256** Sponge-based SHA-3 submission variant producing 256-bit outputs; used by Ethereum for block hashing, address derivation, and state roots [78, 79].
- **RIPEMD-160** 160-bit hash applied after SHA-256 in Bitcoin’s `hash160` address scheme to yield compact wallet addresses [80, 81].

- **BLAKE2b** High-performance hash selected for Zcash’s Equihash POW and Polkadot’s block hashing, offering keyed hashing and efficient parallelism [82, 83].
- **SHA3-256** NIST-standardized 256-bit sponge hash (different padding from Keccak-256), available in modern crypto libraries but less common on public blockchains [84].

2.3.3.2 Signature Schemes

- **ECDSA (secp256k1)** Elliptic Curve Digital Signature Algorithm over the SECG secp256k1 curve, used by Bitcoin and Ethereum for transaction authentication and non-repudiation [85, 86].
- **EdDSA (Ed25519)** Deterministic Schnorr-variant signature on the twisted Edwards curve Curve25519, offering fast signing and small signatures; adopted by Cardano, Stellar, Monero, and more [87, 88].
- **Schnorr** Linear signature scheme enabling signature aggregation and improved privacy; introduced to Bitcoin in Taproot via BIP-340 [89, 90].
- **BLS (BLS12-381)** Pairing-based Boneh–Lynn–Shacham signatures with short, aggregatable signatures over the BLS12-381 curve; used extensively in Ethereum 2.0 and Chia [91, 92].
- **RSA-PSS** Probabilistic RSA signature scheme with security proofs under RSA assumptions; sometimes found in permissioned or legacy blockchain deployments [93].
- **Threshold and Multi-signature Schemes** Protocols such as MuSig2 for Schnorr and threshold EdDSA/ECDSA enabling distributed key-generation and signing among n -parties.

2.3.4 Merkle Trees

Merkle trees are n -ary tree data structure (usually $n = 2$ so they are binary trees) employed to store tamper-proofness record of data. Consider a binary merkle tree of depth d where each leaf node i has value v_i equal to hash of some data point x which we want to store. The merkle tree is constructed in a bottom-up manner from the leaf to the root node. Each node i with children $ci1$ and $ci2$ has value $v_i = f(v_{ci1}||v_{ci2})$. For a n -ary merkle tree, this value is $v_i = f(v_{ci1}||v_{ci2}||\dots||v_{cin})$.

2.3.5 Network Preliminaries

Distributed consensus protocols run over a peer-to-peer network. The network can be represented as a random graph, which makes it difficult for the adversary to compromise the network by exploiting certain structures. However, based on the connectivity between different players and the delay in message passing, we can categorize the network into three broad network models.

Definition 2.4: Synchronous Model

A network model is synchronous if for any message sent at time t (according to either one of sender's or receiver's clock) by sender A to recipient B reaches B within time $< t + \Delta$ (according to the same clock) for some known Δ .

Synchronous model of communication is too optimistic. Consider if we want a protocol to work in the synchronous network and we want to set a Δ . Too large Δ will reduce the throughput while too small Δ can cause timeouts before messages can be transferred. Even if an ideal Δ is found, consider a sender sends messages to two receivers which arrive at times $\Delta + \epsilon$ and $\Delta - \epsilon$. In this case, while one receiver has received the message on time, the former suffers timeout and has a different “view” of the system. Thus, such a model

is not practically feasible. The most general model of communication is the Asynchronous network model which is defined below.

Definition 2.5: Asynchronous Model

A network model is asynchronous if for any message sent at time t by sender A to recipient B eventually reaches B . The delay Δ is unbounded, but finite.

Asynchronous network model is too general and impractical for any distributed protocol to provide reasonable guarantees. Therefore, Partial-synchronous network model was introduced by Dwork et al. [64] as a middle-ground between Synchronous and Asynchronous network model.

Definition 2.6: Partial Synchronous Model [64]

A network model is partially synchronous if there exists a known, finite bound Δ and an event GST (Global Stabilization Time) such that:

1. After some unknown delay, GST to eventually happen (say at time t_{GST}).
2. A message sent at t reaches by time $< \Delta + \max(t, t_{GST})$

In the partial synchronous model, the network behaves as asynchronous till before GST and synchronous after that. We next discuss the problem of consensus in distributed systems under the above mentioned network models. We observe the relation between the three network model is that $Synchronous \subseteq PartialSynchronous \subset Asynchronous$.

2.4 Consensus Protocols: A Formal Treatment

We now look at consensus protocols which solve the problem of agreement in different network and adversary setting. The goal of these protocols is to behave in such a way that each altruistic player's view is same as when they interact with an ideal state-machine

$\mathcal{F}_{state-machine}^{ideal}$, which is described in Figure 2.4. Any agreement protocol aims at realizing $\mathcal{F}_{state-machine}^{ideal}$, and this problem is called *state-machine replication*.

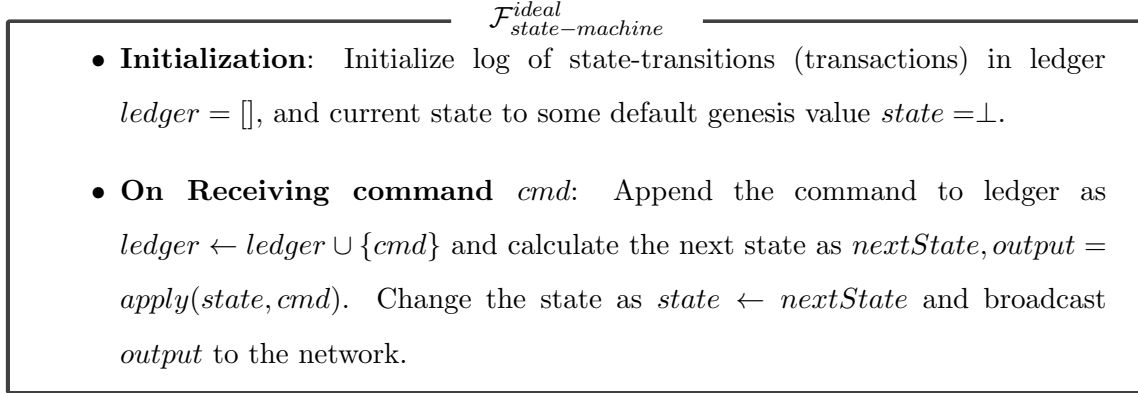


Figure 2.4: Ideal functionality of State Machine

We first notice the more trivial protocols such as agreement in synchronous setting under $CFT(n, t)$ threat model. In this case, we notice for any $t < n$ a simple protocol with $t + 1$ rounds of all-to-all broadcast of the supposed value suffices. If we get $t + 1$ messages with some (same) value v , then we agree on that value (but continue broadcasting this value v till $t + 1$ rounds). If till round t this scenario doesnot happen, then we agree on the majority value broadcasted in round $t + 1$. This satisfies the three properties of agreement (1) agreement: all players agree on a common value, (2) validity: if all non-faulty players have initial input v , then final agreed value is v and (3) termination: the protocol terminates deterministically in v rounds.

Consensus under synchronous setting under $BFT(n, t)$ threat model against computationally unbounded adversary is possible through the *Phase-King Protocol* (Algorithm 1). This protocol achieves consensus in synchronous setting for $t < \frac{n}{2}$.

Algorithm 1 Phase-King Protocol [94]

1: 1. *Initialization:*

- Set of players $P = \{p_1, p_2, \dots, p_n\}$ participate in the protocol.
- Protocol proceeds in $t+1$ rounds with p_i assuming the role of leader/king in round r .

2. *For round $r : 1 \rightarrow t + 1$, with leader/king p_r*

- a Each player p_i broadcasts their value v_i
 - b Each player sets $C_i^b = 1$ iff atleast $n - t$ players send b
 - c Each player p_i broadcasts (C_i^0, C_i^1) . Consider $C_{i \rightarrow j}^b$ is the value of C_j^b received by p_i .
 - d Each p_i calculates $D_i^b := |\{j | C_{j \rightarrow i}^b = 1\}|$. Set $v_i = 1$ if $D_i^1 > t$ otherwise set $v_i = 0$.
 - e King/Leader p_r sends v_r to all p_i . Each p_i sets $v_i = v_r$ if $D_i^{v_i} < n - t$, and remains unchanged otherwise.
-

A modification of this protocol for the same $BFT(n, t)$ threat model under synchronous network for computationally bounded adversary to achieve authenticated agreement was proposed by [95]. This protocol is described as Algorithm 2. For $t < n$, we consider two cases for the Dolev-strong protocol. When (1) leader is honest, it is not possible to introduce any other value because duplicating signature is not possible for computationally bounded adversary. In addition, due to network synchrony each honest player gets the value proposed by the leader and the protocol terminates in 1 round with $n - t$ votes; while if (2)

leader is adversary and proposes duplicate values to two different honest players, the set V will be of size > 2 for all honest players (as all messages get delivered due to synchronous network). Hence the agreed value is $\perp$. In any case, (i) consensus i.e. each honest player agrees on the same value, (ii) validity, if the leader is honest, then all players agree on the value proposed by the leader and (iii) termination (in $t + 1$ rounds) is satisfied. Other protocols solving authenticated byzantine agreement in synchronous setting include [2, 96, 97, 98, 99].

Algorithm 2 Dolev-Strong Protocol [94]

1. *Initialization:* Set of players $P = \{p_1, p_2, \dots, p_n\}$ with public keys $\{pk_1, pk_2, \dots, pk_n\}$ participate in the protocol.
 2. *For round $r : 1 \rightarrow t + 1$,* with leader/king p_r
 - a Leader p_r broadcasts value v_r and signature $s_r^{v_r}$
 - b Each player p_i receives $v_{r \rightarrow i}$ and corresponding signature $s_r^{r \rightarrow i}$ from p_r
 - c Each player p_i broadcasts message, signature pair if $s_r^{v_{r \rightarrow i}}$ is a valid signature on the proposed value $v_{r \rightarrow i}$.
 - c Each player p_r constructs set $V = \{j | v_{j \rightarrow i} = v_{r \rightarrow i}\}$
 - d Each player $|V| > 1$ then output $\perp$ else output $v_{r \rightarrow i}$
-

Consensus in partially synchronous setting is much more difficult and is achieved through a class of protocols [100, 101, 102] motivated from the *pBFT* protocol proposed by Castro et al. [1]. We describe the protocol in Algorithm 3. The protocol reaches agreement in T rounds with probability $1 - \frac{1}{2^T}$.

Algorithm 3 Practical Byzantine Fault Tolerance (pBFT) Protocol

1: 1. *Initialization:*

- Set of nodes $N = \{n_1, n_2, \dots, n_k\}$ participate in the protocol.
- Protocol proceeds in 3 phases: Pre-prepare, Prepare, and Commit.
- A leader node n_L is chosen to initiate the protocol.

2. *Pre-prepare phase:*

- a The leader node n_L broadcasts a Pre-prepare message containing the proposed message m to all other nodes.

3. *Prepare phase:*

- a Each node n_i (except the leader) verifies the Pre-prepare message and broadcasts a Prepare message if the message is valid.
- b Each node n_i waits to receive Prepare messages from at least $2f + 1$ nodes (including itself) to ensure sufficient agreement.

4. *Commit phase:*

- a Once a node n_i has received at least $2f + 1$ Prepare messages, it broadcasts a Commit message.
- b After receiving at least $2f + 1$ Commit messages, node n_i finalizes the decision and commits to the message m .
- c Node n_i broadcasts the decision to all nodes.

5. *Final decision:*

- a After receiving Commit messages from at least $2f + 1$ nodes, the message m is considered committed.
-

In asynchronous network setting, the FLP Impossibility [61] states that no deterministic protocol achieves consensus in presence of a single faulty node if the network is asynchronous. We circumvent this impossibility through randomized protocols [103, 104, 105, 106, 56, 107]. These protocols exploit the randomized coin toss procedure to achieve consensus. Global random coin toss procedure ideal functionality is defined by $\mathcal{F}_{\text{coin-toss}}^{\text{ideal}}$. $\mathcal{F}_{\text{coin-toss}}^{\text{ideal}}$ for round r gives a random bit $b \in \{0, 1\}$ which is common for each player in the distributed system. That is, in round r , invocation of global random coin toss gives a random value same to each player. We show randomized consensus protocol which uses common coin-toss procedure in Algorithm 4.

Algorithm 4 Randomized Consensus Protocol [103, 56]

```

1: Initialization: Set of players  $P = \{p_1, p_2, \dots, p_n\}$  and initial values
    $\{v_1^0, v_2^0, \dots, v_n^0\}$ . Process for player  $p_i$ 
2: for  $r \in 1 \dots T$  do
3:    $S_i^0 = \{\}, S_i^1 = \{\}, decided = 0$ 
4:   Broadcast  $v_i^{r-1}$ 
5:   On Recv  $\langle v_j^{r-1} \rangle$ 
6:      $S_i^{v_j^{r-1}} \leftarrow S_i^{v_j^{r-1}} \cup \{v_j^{r-1}\}$ 
7:   if  $|S_i^b| \geq n - t$  for  $b \in \{0, 1\}$  then
8:      $v_i^r \leftarrow b, decided \leftarrow 1$ 
9:     Output  $v_i^r$ 
10:  else
11:     $b \leftarrow \mathcal{F}_{\text{coin-toss}}(r)$ 
12:     $v_i^r \leftarrow b$ 
13:  end if
14: end for

```

Asynchronous consensus protocols [104, 108, 109, 56, 107] leverage this Randomized consensus protocol that uses common coin procedure for agreement. Note that this protocol works in asynchronous setting and achieves agreement with overwhelming probability.

Gaps and Challenges

Agreement protocols based on the byzantine consensus problem faces the limitation of scalability. The protocol becomes extremely inefficient if the number of players scale. This is the case with distributed ledgers (blockchains) which operate over a peer-to-peer connected network comprising of thousands of players. In addition, Authenticated byzantine agreement also relies on the assumption that public keys of participating players is known. This is not possible for public blockchains, as observed in Bitcoin, Ethereum etc. Therefore, there is the need of a class of scalable consensus protocols which can operate in a public network where players can join and leave network at any time during the protocol execution. We next discuss how blockchains solve this challenge by leveraging two factors: (1) randomization and (2) incentives.

2.5 Blockchains: Incentives and Randomizations in Consensus

Public blockchains has an added requirement that the network is public i.e. players join and leave the network at any time due to which a player does not know other players participating in the protocol. To achieve consensus in this setting, protocols of the type Proof-of-X (such as proof-of-work (PoW), proof-of-stake (PoS) etc.) which use *incentives* — players spends some money (like computational cost in PoW or collateral in PoS). Upon honest behavior (correct execution) of the protocol, the responsible players get compensated through rewards which compensate for the spent cost. In addition, many protocols [110, 9] also leverage randomization to achieve consensus. Such protocols has (1) Block-proposer

selection rule (BPSR) and (2) chain selection rule (CSR). Block proposer selection rule decided which player is responsible for proposing the next block to the blockchain. In addition, chain selection rule resolves any forks due to multiple/conflicting blocks proposed at the same level in a chain.

2.5.1 Blockchains: A Distributed Ledger

Blockchain builds an append-only ledger which realizes the problem of *State Machine Replication* (SMR). State Machine is an automaton system which has a defined state, which is changed through *transactions*. The SMR is maintained in a distributed manner by multiple players, each has their own version of the current state. Blockchains aim to maintain a general consensus on the current state as well as relative ordering between transactions happening on the SMR.

This is achieved by maintaining a chain of blocks (to confirm relative ordering) and each block contains a batch of transactions. This form of “linked-list type of ledger” ensures *agreement* on current state as well as ordering of transactions through randomized consensus protocol, which differs based on the protocol being used (See Section 2.5.2 for details). The relative ordering is agreed upon, after which it becomes *immutable* due to pre-image resistance property of one way functions (Hash functions) used in blockchains (See Section 2.3.2).

2.5.2 Blockchain Protocols

In this section, we go through some blockchain protocols such as Proof-of-Work, Proof-of-Stake, Proof-of-Burn and BFT based blockchain protocols.

2.5.2.1 Proof-of-Work (PoW) Blockchains

Proof-of-Work (PoW) blockchains [111, 9] uses “work” as a substitute for penalty. The protocol block proposer selection rule is to select the miner that first solves the crypto-

Initialization: The functionality initializes the set of players $P \leftarrow \emptyset$ and hash-map $H \leftarrow \emptyset$. For any x , initially $H(x) = \perp$.

Registration:

- On receiving (register, sid, p_i), $P \leftarrow P \cup \{p_i\}$ and return (register, sid) to p_i
- On receiving (de-register, sid, p_i), $P \leftarrow P / \{p_i\}$. Send (de-register, sid) to p_i
- On receiving (get-registered, sid) from $\mathcal{A}$ (adversary) send (P , sid) to $\mathcal{A}$

Random Oracle queries: On recv (eval, sid, x) from p_i :

1. If $H[x] = \perp$, allocate $H[x] = y$ for some $y \in_U \{0, 1\}^\lambda$.
2. Return (eval, sid, x , $H[x]$) to the requestor

Figure 2.5: Random Oracle for PoW

graphic puzzle. The chain selection rule is to select the longest chain. Cryptographic puzzle can be of any type but generally consists of some constraint $\text{Constraint}(\lambda, y, T)$ with security parameter λ , input y and target T . The goal for a player is to find pre-image x such that for some one way function $f : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ we have $y = f(x)$ such that $\text{Constraint}(\lambda, y, T)$ is satisfied. Proof-of-Work is represented by the $\mathcal{F}_{\text{random-oracle}}^{\text{ideal}}$ ideal functionality as described in Figure 2.5 in the random oracle model. Each player has mining power of q queries per round. A player with higher computational power can be viewed as controlling multiple players with fixed computational power (q queries per round). We use this random oracle functionality to construct a Proof-of-Work blockchain protocol which consists of (1) Block proposer selection rule and (2) Chain selection rule. The algorithm is described in Algorithm 5.

We abstracted the mining procedure from other aspects of the PoW protocol which might depend on the protocol. For example, two popular chain selection rule are longest chain [5, 9] and Greedy Heaviest Observed Subtree [112, 113]. Similarly, construction of the block is dependent on the blockchain protocol and is encapsulated in Algorithm 5 as *ConstructBlock*(...) procedure.

Algorithm 5 Proof-of-Work (PoW) Protocol [111, 9]

- 1: **Initialization:** Player p_i with public key pk_i and corresponding secret key sk_i and initial state of the chain $C_i^0 = \{Gen\}$ for protocol defined genesis block Gen and protocol specified target T
 - 2: **Block Proposer Selection Rule** $BPSR(r, C_i^{r-1})$: For round r where the current view of p_i is chain $C_i^{r-1} = \{Gen, B_1, B_2, \dots, B_{r-1}\}$
 - 3: $mined \leftarrow False$
 - 4: **while** $mined$ is $False$ **do**
 - 5: **if** recv. $\langle B_r, pk_j, C_j^{r-1} \rangle$ **then**
 - 6: $C_i^{r-1} \leftarrow SelectChain(C_i^{r-1}, C_j^{r-1})$
 - 7: **if** $Valid(C_i^{r-1}, B_r, pk_j)$ **then**
 - 8: $C_i^r \leftarrow C_i^{r-1} \cup \{B_r\}$
 - 9: $mined \leftarrow True$
 - 10: **end if**
 - 11: **end if**
 - 12: $x \leftarrow \{0, 1\}^*$
 - 13: $B_r \leftarrow ConstructBlock(B_r, pk_i, sk_i, x, C_i^{r-1}, r)$
 - 14: **if** $\mathcal{F}_{random-oracle}(B_r) < T$ **then**
 - 15: $C_i^r \leftarrow C_i^{r-1} \cup \{B_r\}$
 - 16: $mined \leftarrow True$
 - 17: broadcast $\langle B_k, pk_i, C_i^{r-1} \rangle$
 - 18: **end if**
 - 19: **end while**
 - 20: $BPSR(r + 1, C_i^r)$
-

2.5.2.2 Proof-of-Stake (PoS) Blockchains

In PoW Blockchain, a player was selected as block proposer with probability proportional to the fraction of total system mining power contributed by the player. In Proof-of-Stake, this probability is proportional to the fraction of total stake in the system held by the player. Stake of each individual is captured through the blockchain (ledger) which stores all transactions (transfer of stake). Proof-of-Stake has a block-proposer and a set of players forming the committee, each selected with probability proportional to their stake in the system. A block is proposed by the proposer and agreed upon by the validation committee through some agreement protocol such as pBFT (Algorithm 3). Players put up their stake as collateral while proposing block/validating proposed block. If the player deviates, the collateralized stake is slashed which serves as a penalty for deviation.

However, there is a challenge in using bootstrapping protocol — *how is the stake initially distributed*. This problem of *bootstrapping* is a challenge because incorrect bootstrapping protocol might lead to initial centralization of the system which is difficult to get rid of because PoS in expectation maintains the same level of centralization as during protocol initiation. One of the contributions of this thesis is characterizing PoS bootstrapping protocols and defining the set of desired properties along with analysing existing protocols.

2.5.2.3 Proof-of-Burn (PoB) Blockchains

PoW enforces penalty through cost of mining and PoS through slashing collateralized stake. Similarly, in Proof-of-Burn (PoB) penalty is enforced upfront as the participating player has to burn their cryptocurrency to be eligible to propose next block.

Definition 2.7: Burn Protocol [114]

Burn Protocol Π_{burn} has two subprotocols $\Pi_{burn}^{genBurnAddr}$ and $\Pi_{burn}^{burnVerify}$. Given a tag $t \in \{0, 1\}^*$ invoking $\Pi_{burn}^{genBurnAddr}(1^k, t)$ gives a burn address $addr$. In addition, invoking $\Pi_{burn}^{burnVerify}(1^k, t, addr)$ outputs *true* only if $addr$ correctly encodes t .

Proof-of-Burn uses Burn Protocol Definition 2.7 to burn tokens while proposing a block. If the block is valid, appropriate block reward is given to the block proposer. If block is invalid, the burnt tokens are lost.

2.5.2.4 BFT based blockchains

Similar to PoW and PoS protocols, there permissioned blockchains where consensus is obtained by running a byzantine fault tolerant algorithm such as pBFT [1]. Permissioned blockchains have access control procedure which allows players to be part of the system unlike public blockchains such as Bitcoin [9] and Ethereum [5]. Block proposer selection is usually through random selection, round robin, reputation based selection of miners etc.

2.5.3 Blockchain Properties

In this section, we discuss the structure of the blockchain, and each individual block. Following this we discuss security properties of the blockchain. Security in blockchains is described differently from security in permissioned distributed consensus as is described in Section 2.5.4.

2.5.3.1 Blockchain Structure

Blockchain consists of a chain of blocks connected through a linked-list type of data structure called *hash-chain*. This uses some One Way Function Definition 2.3. Each player $p_i \in P$ has a *view* of the system at round r represented by the chain C_i^r . Consider two chains C_A and C_B , then if C_A is a prefix of C_B then it is denoted by $C_B \preceq C_A$. If C_B

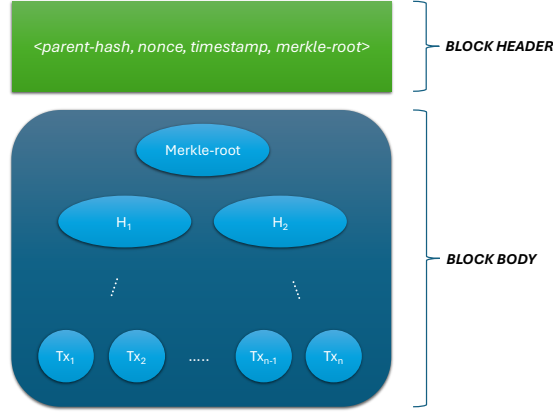


Figure 2.6: Block Structure

is a k -prefix of C_A i.e. a prefix on removing k blocks; this is represented by $C_B \preceq C_A^{|k}$. The block comprises of the body, and block header. The body comprises of a set of transactions stored in the form of Merkle-tree (Section 2.3.4). The block structure is depicted in Figure 2.6.

2.5.4 Required Properties

The security and liveness of blockchain protocols relies on the three traditional properties of *Chain Growth*, *Chain Quality* and *Common Prefix*. In addition to this, a relatively less addressed property which is also required for security (avoiding single points of failure) is *decentralization*. One part of the thesis focuses on addressing challenges in decentralization in presence of rational adversary, and mechanisms to form decentralized blockchains. The abovementioned four properties are described below:

2.5.4.1 Common Prefix

Two honest players P_1, P_2 which are a part of the system have their own versions of the blockchain, C_1 and C_2 . Common prefix property is satisfied when the two chains are consistent except for the k most recent blocks. Formally,

Definition 2.8: Common Prefix Property Q_{cp}^k [115]

The common prefix property Q_{cp}^k is satisfied for $k \in \mathbb{N}$ when for any two honest players P_1, P_2 with their versions C_1, C_2 of the blockchain are such that either $C_1^{[k]} \preceq C_2^{[k]}$ or $C_2^{[k]} \preceq C_1^{[k]}$,

Common prefix property is required so that the system can have finality on system state. Common prefix ensures that the set of blocks confirmed are consistent throughout all the honest players.

2.5.4.2 Chain Quality

Chain quality property ensures that atleast a minimum proportion μ of the blocks in any consecutive sequence are mined by the honest players. This property also implies a weaker property that any proposed/mined block is by an honest player with probability atleast μ . Formally,

Definition 2.9: Chain Quality Property $Q_{cq}^{\mu,l}$ [115]

A blockchain satisfies chain quality property for μ, l when for any honest player P with chain C , any l consecutive blocks have atleast μ fraction mined by honest players.

The chain quality property measures the proportion of blocks in the chain that are created by honest players. This metric ensures that malicious actors cannot dominate the blockchain, even if they control a portion of the network's resources.

2.5.4.3 Chain Growth

Chain growth refers to the minimum rate at which the blockchain expands over time. A robust blockchain system must demonstrate two key aspects of chain growth:

- **Fairness:** The rate of block production should proportionally reflect the resources contributed by players (implied by chain quality property).
- **Liveness:** Honest nodes must be able to append new blocks to the chain within a reasonable timeframe.

Chain growth property ensures that atleast a minimum number of blocks are proposed in a given duration. Mining takes place in discrete time intervals called *rounds*. In each round, atleast one block is proposed. Formally,

Definition 2.10: Chain Growth Property $Q_{cg}^{\tau,s}$ [115]

Chain growth property is satisfied for s, τ for a honest party P with chain C when after s rounds, the chain is atleast $\tau \cdot s$ blocks longer.

2.5.4.4 Decentralization

Decentralization is a fundamental property that ensures no single entity or small group of entities can control the system. Decentralization has been studied in literature [116]. The thesis studies decentralization in both Proof-of-Work and Proof-of-Stake. We say centralization does not increase (worsen) if in two consecutive rounds, the maximum control (stake or compute) by an individual participant of the system doesnot increase. Mathematically, if $\mathbf{f} = (f_1, f_2, f_3, \dots, f_n)$ is the fraction controlled in round r and fraction of compute controlled at round $r + 1$ is $\mathbf{g} = (g_1, g_2, \dots, g_n)$. Centralization doesnot worsen if for all rounds r ,

$$\max_{i \in [n]} f_i \geq \max_{i \in [n]} g_i \quad (2.1)$$

Key aspects of decentralization include:

- **Resource Distribution:** Power to create blocks (e.g., mining or staking power) should be evenly distributed to prevent centralization.
- **Network Participation:** A diverse and geographically distributed network of nodes ensures redundancy and resilience against censorship or localized failures.
- **Governance:** Decision-making processes should be inclusive and transparent, with mechanisms to prevent collusion or concentration of power.

Decentralization strengthens the trustless nature of blockchain systems, making them resistant to censorship, fraud, and single points of failure.

2.6 Game Theory Preliminaries

Game theory provides a rich framework for modeling strategic interactions among rational decision-makers. In this section, we review Stackelberg Games — an important class of games that are fundamental in protocol design.

2.6.1 Stackelberg Games

Stackelberg games describe a hierarchical framework in which one player, termed the *leader*, commits to a strategy first, and the other player, the *follower*, observes this commitment before choosing their response. This sequential decision-making process creates a first-mover advantage for the leader and often leads to outcomes that differ significantly from those in simultaneous-move games.

Mathematically, the Stackelberg framework is formulated as a bi-level optimization problem. The leader maximizes its payoff by anticipating the optimal reaction of the follower, who solves a subordinate optimization problem given the leader’s decision [117]. This formulation poses significant computational challenges, as the leader’s decision space directly influences the follower’s feasible set. Recent research has focused on developing

efficient algorithms—ranging from mixed-integer programming approaches to iterative and heuristic methods—to compute the Stackelberg equilibrium [118, 119].

Stackelberg games have found wide applications in various fields such as economics (notably in oligopoly markets), supply chain management, and energy systems. The leader’s strategic commitment can influence market dynamics, pricing strategies, and even network routing protocols, making this model an essential tool for analyzing hierarchical interactions. In our context, Stackelberg Games are used for proving security of protocols.

2.6.2 Incentive Compatibility

Incentive compatibility is a central concept in mechanism design and game theory. It ensures that each participant’s best interest is to act according to their true preferences or types. In the following subsections, we formally define key equilibrium concepts along with the corresponding mathematical formulations.

2.6.2.1 Dominant Strategy Equilibrium

Definition 2.11: Dominant Strategy Equilibrium [120]

A strategy profile $s^* = (s_1^*, s_2^*, \dots, s_n^*)$ is said to be a *dominant strategy equilibrium* if for every player i and for all alternative strategies $s_i \in S_i$, the following condition holds:

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*)$$

where $u_i : S_1 \times \dots \times S_n \rightarrow \mathbb{R}$ denotes the utility function of player i , and s_{-i}^* represents the strategy choices of all players other than i .

In a dominant strategy equilibrium, each player’s selected strategy yields the highest payoff irrespective of the strategies chosen by the other players. This makes the concept particularly powerful in scenarios where commitment and credibility are essential.

2.6.2.2 Nash Equilibrium

Definition 2.12: Nash Equilibrium [121]

Consider a game where each player i has a private type θ_i drawn from a type space Θ_i with a common prior $F(\theta_1, \dots, \theta_n)$. Let $s_i : \Theta_i \rightarrow S_i$ be the strategy function of player i . The strategy profile $s^* = (s_1^*, s_2^*, \dots, s_n^*)$ is a *Nash equilibrium* strategy if, for every player i and for every $s_i \in S_i$,

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*)$$

holds true. This implies that no player can unilaterally deviate from s_i^* and achieve a higher payoff, assuming the other players maintain their strategies s_{-i}^* .

The Nash equilibrium concept relaxes the conditions of dominant strategies by accounting for mutual best responses among players. It represents a state of strategic stability in many game-theoretic models.

2.6.2.3 Bayesian Nash Equilibrium

Definition 2.13: Bayesian Nash Equilibrium [122]

Consider a game with incomplete information where each player i has a private type θ_i drawn from a type space Θ_i with a common prior $F(\theta_1, \dots, \theta_n)$. Let $s_i : \Theta_i \rightarrow S_i$ be the strategy function of player i . The strategy profile $s^* = (s_1^*, s_2^*, \dots, s_n^*)$ is a *Bayesian Nash equilibrium* if, for every player i and for all $\theta_i \in \Theta_i$,

$$\mathbb{E}_{\theta_{-i} \sim F(\cdot | \theta_i)} \left[u_i \left(s_i^*(\theta_i), s_{-i}^*(\theta_{-i}); \theta_i, \theta_{-i} \right) \right] \geq \mathbb{E}_{\theta_{-i} \sim F(\cdot | \theta_i)} \left[u_i \left(s_i(\theta_i), s_{-i}^*(\theta_{-i}); \theta_i, \theta_{-i} \right) \right],$$

where the expectation is taken over the types of the other players θ_{-i} conditional on θ_i .

This equilibrium concept extends the Nash framework to settings with incomplete information by incorporating the beliefs of the players about the types of others. It is widely used in economic models and auction theory.

2.7 Other related works

Consensus in permissioned and permissionless blockchains has been extensively studied. Delegate Byzantine Fault Tolerance (dBFT) improves on classical PBFT by using a primary-backup agreement and speculative execution to achieve graceful degradation under up to $f < \frac{n-1}{3}$ adversaries [123]. Subsequent security analyses—most notably Wang et al. [124] formal model of the NEO dBFT protocol—highlight subtle liveness and safety attacks under partial synchrony. To scale BFT consensus to large, open networks, Algorand applies cryptographic sortition and a two-phase BA $\star$ protocol to tolerate $f < \frac{n}{3}$ Byzantine nodes with one-minute finality [54], while Prism deconstructs Nakamoto consensus into parallel voting and proposal chains to approach network bandwidth limits [125]. OHIE further parallelizes Bitcoin-style chains into k independent instances, maintaining Nakamoto-derived persistence and liveness in each and merging them into a total order [126].

Off-chain and hybrid designs address throughput and latency bottlenecks. Virtual payment channels allow multi-hop, off-chain value transfer without on-chain intermediaries, using state-sharing and punish mechanisms to guarantee balance security [127]. Lockable signatures generalize this to script-independent channels—leveraging one-time-pad or BLS aggregation—to enable cross-platform payment networks with on-chain privacy guarantees [128]. Finally, Proof of Learning abstracts verifiable training proofs for distributed ML as succinct state-machine commitments, ensuring correctness and adversarial accountability with sublinear verification cost [129].

Recent advances in *rational consensus* extend classical fault-tolerant agreement by incorporating self-interested agents alongside crash- and Byzantine-faulty nodes. Ranchal-Pedrosa and Gramoli formalize (k, t) -crash-robustness under partial synchrony, showing

tight impossibility bounds when $k + 2t \geq n$ and reductions between crash- and Byzantine-robust protocols [130]. Amoussou-Guenou *et al.* model rational and Byzantine players in committee-based blockchains, proving that safety holds if the acceptance quorum $\kappa > t$ but liveness requires $\kappa \leq k + 1$, and characterizing the exact equilibrium region $t < \kappa \leq k + 1$ [131]. Earlier, Groce *et al.* showed that perfect and statistical security against purely rational adversaries is possible for $t < n$, even though classical BA becomes impossible for $t \geq n/3$ [132].

Complementing these consensus-centric studies, the *Rational Protocol Design* paradigm embeds game-theoretic utilities into the UC framework. Garay *et al.* introduce attack-payoff security for MPC, deriving cost-vs-utility bounds under static and adaptive corruptions [133]. Badertscher *et al.* apply this to Bitcoin, modeling block rewards and fees as Stackelberg-game utilities to recover incentive compatibility under front-running adversaries [134]. Karakostas *et al.* further analyze PoW/PoS protocols via ε -Nash dynamics and infraction predicates, decoupling compliance from equilibrium [135]. Building on rational-Byzantine interplay, Ranchal-Pedrosa & Gramoli’s TRAP protocol leverages a baiting mechanism to achieve consensus if and only if a (k, t, m) -baiting strategy exists [136]. Finally, Eyal & Sirer expose selfish-mining vulnerabilities for any mining power β , and Garay *et al.* model variable-difficulty dynamics via martingale techniques, both highlighting strategic deviations in open-blockchain settings [137, 138].

Chapter 3

Consensus in the Rational Setting

“What does not benefit the hive does not benefit the bee either.”

— Marcus Aurelius, *Meditations* (Book VI)

3.1 Introduction

Agreement and Distributed Consensus have been central topics in distributed computing ever since the formulation of the Byzantine Generals Problem [139, 50]. These concepts are foundational in building distributed file systems, fault-tolerant architectures, and more recently, blockchain-based systems. The core idea is to enable a group of n nodes, where up to f may behave incorrectly or maliciously, to agree on a shared value v . When this agreement process is repeated over time, a further requirement is that all participants must maintain a consistent order of the agreed-upon values across rounds—this extension is known as *Atomic Broadcast* (ABC).

Classic results have shown that consensus is achievable in synchronous networks even with up to $t < n/3$ Byzantine faults [50, 1]. The pBFT protocol by Castro et al. [1] expanded this result to partially synchronous environments. On the other hand, the FLP impossibility theorem [140] demonstrated that reaching agreement in fully asynchronous networks using *deterministic* methods is impossible if even a single node is faulty. This

limitation led to the development of randomized algorithms [108, 109, 141], which can overcome the impossibility in asynchronous settings under the same fault bounds ($t < n/3$).

Different consensus algorithms are tailored for various failure models. For instance, Paxos [142, 143] and Raft [144] are designed to handle c crash faults, where nodes may simply stop functioning—this is referred to as Crash Fault Tolerance (CFT(c)). In contrast, protocols like pBFT [1] and Honeybadger [145] address the Byzantine setting, tolerating up to $t < n/3$ nodes that may act arbitrarily—this forms the Byzantine Fault Tolerant model (BFT(t)).

Ranchal-Pedrosa and Gramoli [146] introduced a broader threat model that incorporates not only t Byzantine nodes but also k rational ones capable of forming coalitions. This rational threat model, known as *RFT*(k, t), redefines the agreement problem as *Rational Consensus* (RC). They proposed a protocol called TRAP based on a *baiting mechanism*, showing that consensus can be reached at a Nash Equilibrium (NE) under the conditions $t < n/3$ and $k + t < n/2$. Protocols where the honest behavior aligns with a NE are termed *Nash Incentive Compatible* (NIC).

However, our analysis reveals that TRAP may permit another NE strategy—one that leads to disagreement when extended to Atomic Broadcast (ABC). Rational agents might gravitate towards this undesirable equilibrium, jeopardizing the security of the system. To ensure robustness against such outcomes, game-theoretic security considers equilibria to be reliable only when they represent Pareto-optimal or Focal equilibria [147]¹. An even stronger guarantee is achieved when the equilibrium is a Dominant Strategy Equilibrium (DSE), rather than just a Nash Equilibrium (NE).

3.2 Problem Description

Currently, there are no known protocols that successfully realize Atomic Broadcast (ABC) within the rational threat model. This chapter addresses this limitation by pre-

¹For an in-depth discussion, refer to Section 3.4.3 or [147]

senting both impossibility results and introducing a new protocol, **pRFT**, which accomplishes ABC in the rational model under specific assumptions about the utility functions of rational participants.

3.2.1 Solution Approach

We extend the classical $RFT(k, t)$ model [146] in two key ways: (i) by incorporating discounted utilities in a repeated interaction setting, and (ii) by introducing heterogeneity among rational agents based on their behavioral incentives. These behavioral types are characterized by a parameter θ :

- $\theta = 1$: agents are incentivized to create disagreement,
- $\theta = 2$: agents aim to carry out censorship attacks,
- $\theta = 3$: agents seek to disrupt liveness through denial-of-service tactics.

We prove that for rational players of type $\theta = 2$ and $\theta = 3$, it is impossible to achieve Rational Consensus (RC) under the $RFT(k, t)$ model when $\frac{n}{3} \leq t + k < \frac{n}{2}$. Consequently, our focus narrows to type $\theta = 1$ players. For this case, we present the protocol **pRFT**, which guarantees consensus under the same threat threshold, assuming all rational players conform to type $\theta = 1$ ².

Previous works addressed rational security by designing incentives such that rational players are motivated to expose colluding deviators—a process termed “baiting”. In these constructions, the system’s security relies on achieving an equilibrium where at least m players choose to bait, deterring deviation. However, such approaches remain vulnerable to alternate equilibria that compromise protocol integrity.

In contrast, our approach eliminates the reliance on rational baiting by embedding accountability directly into the protocol’s structure. Every participant is required to stake a collateral deposit. If a participant deviates and is detected, they forfeit their collateral.

²See Section 3.4.1.1 for an elaboration on rational player types

Network	Threat Model		
	$CFT(c)$	$BFT(t)$	$RFT(t, k)$
Synchronous	$2c < n$ [142]	$2t < n$ [139]	$t < \frac{n}{2}, k < \frac{n}{2}$ [139]
Partially-synchronous	$2c < n$ [142]	$3t < n$ [1]	$t < \frac{n}{4}, t + k < \frac{n}{2}$
Asynchronous	$c < \frac{n}{3}$ [141]	$t < \frac{n}{3}$ [141]	$t < \frac{n}{3}$ [141]

The results highlighted in blue are contributions of this chapter.

Table 3.1: Bounds for consensus in different threat models.

Since rational players are averse to losing their stake, the safest strategy is to adhere to the protocol—regardless of the actions of others. This ensures that compliance becomes a *dominant strategy*, thereby achieving *Dominant Strategy Incentive Compatibility* (DSIC).

Based on our analysis, we position our protocol (highlighted in blue) in Table 3.1, comparing it against existing consensus bounds across varying network and threat models. Moreover, we demonstrate that **pRFT** attains optimal message complexity and communication overhead, comparable to state-of-the-art accountable protocols [148, 149], while operating under a strictly more general threat model.

3.2.2 Chapter’s Contributions

In this chapter, we build upon the Rational Threat Model introduced in [146], and introduce an extended classification of rational agents using a type parameter θ , which captures their behavioral motivations. Specifically, we define:

- $\theta = 3$: rational agents are incentivized to disrupt liveness and may carry out censorship or force disagreement.
- $\theta = 2$: rational agents may attempt censorship or disagreement but do not interfere with liveness.
- $\theta = 1$: rational agents aim solely to induce disagreement.

Given k rational players and t Byzantine players (where $t < t_0$) in a network of n total participants, we demonstrate the following fundamental limitations in Section 3.5:

- Consensus becomes unattainable when all rational players are of type $\theta = 3$, even if $k + t < n/2$ and $t_0 < n/3$, in both partially synchronous and asynchronous communication settings (Theorem 3.1).
- Consensus cannot be achieved for type $\theta = 2$ rational agents under the same network conditions: $k + t < n/2$ and $t < n/3$ (Theorem 3.2).
- Existing baiting-based protocols (such as TRAP [146]) admit alternate Nash equilibria that lead to failure in agreement (Theorem 3.3). This necessitates a protocol that ensures a single, stronger equilibrium point—ideally one that satisfies Dominant Strategy Equilibrium (DSE) rather than just Nash Equilibrium (NE)³.

To address these challenges, we introduce a new protocol, **pRFT**, in Section 3.6, which achieves consensus within the $RFT(k, t)$ threat model. Our contributions are summarized below:

- **pRFT** successfully achieves consensus when rational agents are of type $\theta = 1$, provided that $k + t < n/2$ and $t < n/4$.
- The protocol guarantees both correctness (via Dominant Strategy Equilibrium) and liveness in synchronous and partially synchronous environments.
- We establish that **pRFT** matches the best-known message complexity among accountable consensus protocols⁴.

³Refer to Section 3.4.3 for a deeper discussion on equilibrium concepts.

⁴See Section 3.6.3.1 for a formal definition and discussion of accountability in consensus mechanisms.

3.3 Additional Preliminaries

In this section, we introduce key definitions and review existing work pertinent to this chapter.

3.3.1 Blockchain and Atomic Broadcast Protocol

Blockchain systems implement *Atomic Broadcast* (ABC)—formally defined in Section 3.3.2—by executing a sequence of consensus decisions that preserve a total order on all agreed values.

In open (permissionless) networks, probabilistic ABC is typically provided by PoW [4] or PoS [54] schemes. In permissioned settings, a fixed committee $\mathcal{P} = \{\mathcal{P}_1, \dots, \mathcal{P}_n\}$ employs BFT-style protocols to agree on a single block per round. Each block B contains a batch of transactions $\overline{tx}$ representing state updates and references its immediate predecessor.

The outcome is a block sequence C , although each participant $\mathcal{P}_i$ retains its local view C_i . Protocols like PBFT [1] and Honey Badger [145] wait until full agreement before finalizing a block. Others, including Algorand [54] and our proposed pRFT, introduce *tentative blocks* that may be rolled back if later evidence of adversarial activity emerges. A block becomes permanently accepted only after a subsequent block is finalized during a period of synchrony. The *common-prefix property* holds if, for any depth z , truncating the last z blocks from each C_i yields a sequence that is a prefix of every honest chain [6].

3.3.1.1 Player Types

The system comprises n participants, partitioned into:

- **Honest Players:** Also known as altruistic, they follow the protocol exactly.
- **Byzantine Players:** They may adopt any disruptive strategy (e.g., violate safety or liveness) and are unaffected by incentive mechanisms.
- **Rational Players:** Each maximizes a specified utility function (see Section 3.4.1.2), deviating only if a higher payoff is attainable.

3.3.2 Flavours of Consensus

The consensus problem originated with the Byzantine Generals' formulation [50] and has since been studied as Byzantine Broadcast (BB), Byzantine Agreement (BA) [139, 50], Atomic Broadcast (ABC) [150, 151], and Rational Consensus (RC) [60, 146]. We now outline the most relevant definitions; further details appear in Chapter 2.

3.3.2.1 Byzantine Broadcast & Agreement

The Byzantine Generals' Problem, formalized by Lamport et al. [139], models a system with t Byzantine and $n - t$ honest nodes. We adopt the BA definition from [151, Definition 2].

3.3.2.2 Atomic Broadcast

Atomic Broadcast extends BA by requiring repeated agreement rounds, maintaining a ledger of values in a globally consistent order. We follow the formalization in [151, Definition 5].

3.3.2.3 Rational Consensus

As interest in incentive-aware protocol analysis grows, *Rational Consensus* (RC) generalizes ABC to mixed Byzantine-rational adversaries. We base our RC definition and the notion of robustness on [146], augmenting it for repeated execution by enforcing a c -strict ordering condition (the rational analogue of ABC).

Definition 3.1: (t, k) -robustness

Consider a protocol Π is run by $\mathcal{P} = \{\mathcal{P}_1, \mathcal{P}_2 \dots, \mathcal{P}_n\}$ players where t players are byzantine and k players are rational (follow the strategy with the highest incentive) while remaining $n - t - k$ players are honest. The protocol Π is (t, k) -robust if it satisfies:

- (t, k) -validity If all altruistic players receive value v then they all agree on value v
- (t, k) -agreement All altruistic players output the same value in each round.
- c -strict ordering If the ledger of agreed blocks is C_1 and C_2 for two altruistic players with $|C_1| \leq |C_2|$, then $C_1^{\lfloor c} \subseteq C_2^{\lfloor c}$ holds^a.
- (t, k) -eventual liveness if an honest player outputs B then all honest players output B eventually.

^a $C^{\lfloor c}$ is the ledger after removing the last c blocks

We define a stronger notion of RC when protocols also satisfy censorship resistance (we define (t, k) -censorship resistance) and call such RC protocols as strongly (t, k) -robust.

Definition 3.2: (t, k) -censorship resistance

A protocol Π satisfies (t, k) -censorship resistance if when all honest players have transaction tx as input, then eventually all honest players output a block with transaction tx .

Definition 3.3: strong (t, k) -robustness

A protocol Π is strongly (t, k) -robust if Π is (t, k) -robust and (t, k) -censorship resistant.

3.3.3 Cryptographic and Network Preliminaries

Digital Signatures

We rely on a standard Public Key Infrastructure (PKI) to provide unforgeable digital signatures, assuming adversaries are probabilistic polynomial-time machines with oracle access and that forgery succeeds only with negligible probability. Such authenticated broadcast primitives trace back to Dolev and Strong’s work on Authenticated Byzantine Agreement [51].

3.3.3.1 Trusted Setup

Prior to running the protocol, a trusted initialization phase—akin to that in [108]—is executed (e.g., via a common third party) to distribute public keys among all participants. Every signed message is then verifiable against this registry.

3.3.3.2 Network Models

We assume reliable, point-to-point links: messages are never lost or tampered with, though delivery may be delayed. We distinguish three timing models:

1. *Synchronous*: Every message arrives within a known bound Δ .
2. *Asynchronous*: No upper bound on delivery time, but all messages eventually arrive.
3. *Partially Synchronous* [152]: Initially network behaves asynchronously; after a Global Stabilization Time (GST), it becomes synchronous with delay at most Δ .

3.3.4 Baiting-Based Consensus Protocols

Protocols in the baiting paradigm (e.g., TRAP [146]) consider a coalition of up to k rational and t Byzantine nodes deviating jointly. Rational participants are encouraged to submit a *Proof-of-Fraud*—a collection of $t_0 + 1$ conflicting signatures—thereby “baiting” the

colluders (see Section 3.6.3.1 and [149, 148] for details). If at least m rational nodes bait, one is chosen at random to receive a reward $\mathcal{R}$. Each party posts a collateral deposit $\mathcal{L}$ that is forfeited if their signature appears in a Proof-of-Fraud. Meanwhile, a successful disagreement yields total gains $\mathcal{G}$, split equally so each rational node in K obtains $\mathcal{G}/k$. For a full exposition of TRAP’s model and guarantees, consult [146].

3.4 Our Model

We formalize Rational Consensus (RC) as a multi-player game involving Byzantine, rational, and altruistic (honest) nodes. Here we specify: (i) the game structure, (ii) utility functions, and (iii) network assumptions.

3.4.1 The Game

Let $\mathcal{P} = \{\mathcal{P}_1, \dots, \mathcal{P}_n\}$ be the set of participants maintaining a ledger of **Blocks**. Each block contains a batch of valid transactions $\overline{tx} = (tx_i)_{i=1}^z$ referring to prior confirmed blocks. Consensus on block B_r occurs in discrete *rounds* r , with a designated leader $\mathcal{P}_\ell$ where $\ell = 1 + (r \bmod n)$.

3.4.1.1 Player Categories

- **Byzantine** ($T, |T| = t$): May behave arbitrarily to disrupt safety or liveness, unaffected by incentives.
- **Rational** ($K, |K| = k$): Choose actions maximizing a predefined utility (see Section 3.4.1.2). They follow protocol Π unless a deviation grants a non-negligible advantage. Players have types $\theta \in \{0, 1, 2, 3\}$.
- **Honest** ($H, |H| = h = n - k - t$): Adhere strictly to Π provided their expected reward exceeds non-participation.

We permit joint collusion between rational and Byzantine nodes, up to size $k + t$.

Player Type (θ)	System State (σ)				Preferred States
	σ_{NP}	σ_{CP}	σ_{Fork}	σ_0	
$\theta = 3$	α	α	α	0	No Progress, Censorship, Fork
$\theta = 2$	$-\alpha$	α	α	0	Censorship, Fork
$\theta = 1$	$-\alpha$	$-\alpha$	α	0	Fork
$\theta = 0$	$-\alpha$	$-\alpha$	$-\alpha$	0	Honest Execution

Table 3.2: Payoff function $f(\sigma, \theta)$

3.4.1.1.1 System States Depending on player strategies and network behavior, the system may reside in one of the following states:

- **No Progress** (σ_{NP}): In every round r , no new block is appended.
- **Conditional Progress** (σ_{CP}): In round r , confirmed blocks include only transactions outside a censored set Z .
- **Disagreement (Fork)** (σ_{Fork}): At some height h , two honest nodes have distinct confirmed blocks $B_i \neq B_j$.
- **Honest Execution** (σ_0): Protocol proceeds without safety or liveness violations.

3.4.1.1.2 Player Types We further model rational player type⁵ through $\theta \in \{0, 1, 2, 3\}$. The players' types depend on their incentives for different states of the distributed system. We characterize the function $f(\sigma, \theta)$ for payoff when player type is θ and the system is in state σ . This function is represented in Table 3.2 (for some positive constant α). Types of rational players are described below:

- $\theta = 3$: Such players are incentivized if the system state is σ_{NP}, σ_{CP} or σ_{Fork} .

⁵The notion of player type θ corresponds only to rational players because Byzantine type ($\theta = 3$) and Altruistic/Honest type ($\theta = 0$) is already fixed by definition.

- $\theta = 2$: Such players are incentivized if system state is σ_{CP} or σ_{Fork} .
- $\theta = 1$: Such players are incentivized if system state is σ_{Fork} .
- $\theta = 0$: This type of rational player is disincentivized if the system is in any state except σ_0 . However, they might be incentivized against sending messages or performing verification of messages, which has been previously analysed by Amoussou-Guenou et al. [57].

If there are multiple types of rational players, we analyze for security for the worst types amongst them. If K_i is set of rational players with type $\theta = i$, we say, $K = \cup_{i=0}^3 K_i$ is of type $\theta = \max\{i | K_i \neq \emptyset\}$.

3.4.1.2 Utility Structure

Byzantine players follow the strategy that causes maximal disruption to the protocol Π irrespective of the associated utility. If the protocol is *individually rational*, honest players follow Π honestly. Therefore, we need only model the utility for the rational players $\mathcal{P}_i \in K$.

3.4.1.2.1 Strategy Space In addition to defining the possible states of the system and the possible types of a rational player, we define the set of strategies available with the rational players in each round.

- **Abstain** π_{abs} : $\mathcal{P}_i$ does not send messages in the particular phase or round.
- **Double-Sign** π_{ds} : $\mathcal{P}_i$ signs on two conflicting messages in the same phase and round.
- **Honest** π_0 : $\mathcal{P}_i$ follows the specified protocol Π honestly.

3.4.1.2.2 Penalty There also exists a penalty that is incurred by player $\mathcal{P}_i$ if there exists proof that with overwhelming probability, the player has deviated from the protocol. The penalty is a fixed constant L for each player which is the collateral deposited by these players before participating in consensus. If proof of malicious behaviour is found,

this deposit is stashed/burnt [153] and the penalty mechanism should be such that for a player that has followed the protocol honestly the penalty should be 0 except with negligible probability.⁶ The penalty is determined through a function $D(\pi, \sigma)$ based on the mechanism which takes value 1 if a penalty is incurred and 0 otherwise.

Based on the strategy π and type θ , we define the utility of rational players in a round r by taking an expectation over the set of possible states, i.e. $\sigma \in S$ as:

$$u_i(\pi, \theta, r) = \mathbb{E}_{\sigma \sim S}[f(\sigma, \theta)] - L \cdot D(\pi, \sigma) \quad (3.1)$$

The function $f : S \times \{0, 1, 2, 3\} \rightarrow \{-\alpha, 0, \alpha\}$ (for some positive constant α) is given in Table 3.2. If we consider the expected utility in a particular round for player $\mathcal{P}_i \in K$ as $u_i(\pi, \theta, r)$, then the expected utility across rounds can be defined as:

$$U_i(\pi, \theta) = \sum_{r=0}^{\infty} \delta^r u_i(\pi, \theta, r) \quad (3.2)$$

3.4.2 Threat Model

We model threat via $\mathcal{M}$ where $|T| = t \leq t_0$ and $|K| = k$. Here t_0 is the upper-bound on byzantine players to ensure security against byzantine-only attacks.

We make a simple observation under the threat model $\mathcal{M}$, for any $t_0 \geq 1$, the necessary condition for any protocol Π to reach agreement with the threshold $\tau \in [\lfloor \frac{n+t_0}{2} \rfloor + 1, n - t_0]$. This threshold is such that $n - t_0$ players should agree on a value for agreement.

Claim 3.1. *A protocol Π achieves consensus with agreement of at least τ players agreeing on the same value under threat model $\mathcal{M} := \langle (\mathcal{P}, T, K), \theta, t_0 \rangle$ only if $\tau \in [\lfloor \frac{n+t_0}{2} \rfloor + 1, n - t_0]$*

Proof. We prove the contrapositive of this claim. Consider the two cases where $\tau > n - t_0$ and $\tau \leq \lfloor \frac{n+t_0}{2} \rfloor$. If $\tau > n - t_0$ then a message/vote from at least one byzantine player

⁶This condition ensures [Individual Rationality](#) of Honest players

is required to reach a consensus. Under this situation, each byzantine player can play π_{abs} which would compromise the (t, k) -eventual liveness property of the protocol Π . If $\tau \leq \lfloor \frac{n+t_0}{2} \rfloor$ then consider the existence of network partition such that two subsets of players A and B are unable to communicate with each other except through set of adversaries T . Here, $A \cup B = \mathcal{P} \setminus T$, $A \cap B = \emptyset$ and $|A| = |B| = \frac{n-t_0}{2}$. If the leader in some round is $\mathcal{P}_l \in T$ then leader proposes v_a to A and v_b to B . Since $|A| + |T| \geq \lfloor \frac{n-t_0}{2} \rfloor + t_0 \geq \tau$ and similarly $|B| + |T| \geq \lfloor \frac{n-t_0}{2} \rfloor + t_0 \geq \tau$, both partitions reach consensus on conflicting values which invalidates the (t, k) -agreement property. Therefore, Π is not (t, k) -robust in either case. $\square$

3.4.3 Equilibrium & Incentive Compatibility

3.4.3.0.1 Nash Equilibrium (NE) While analysing the rational security of protocols, a protocol is considered secure if following the protocol honestly is a *Nash Equilibrium* strategy. Therefore, the aim is to design a protocol that is Nash Incentive Compatible. This means that following the protocol honestly is the *nash-equilibrium* strategy for all rational players. We define Nash Incentive Compatible protocol as follows:

Definition 3.4: Nash Incentive Compatible

A protocol Π with set of K rational players is *Nash Incentive Compatible* (NIC) for a given utility structure U if $\forall i \in K$ in the set of rational players K following strategy s_i following the honest strategy π_0 is Nash Equilibrium. i.e. $\forall i \in K, \forall \pi$

$$U_i(s_i = \pi, \{s_j = \pi_0\}_{j \neq i}) \leq U_i(s_i = \pi_0, \{s_j = \pi_0\}_{j \neq i})$$

3.4.3.0.2 Focal Point Given a game, it could have multiple equilibria. Amongst, multiple equilibria, a particular equilibrium may attract more attention than other equilibria. Such equilibrium is often referred as *focal point* [147]. Consider the following example

game between three players $P1$ having strategy space $\{A, B\}$, $P2$ having $\{a, b\}$ and $P3$ having $\{\alpha, \beta\}$. The utility is as given in Table 3.3 given in the order (U_{P1}, U_{P2}, U_{P3}) .

	a		b	
	α	β	α	β
A	(1, 1, 1)	(1, 1, 0)	(1, 0, 1)	(-2, 2, 2)
B	(0, 1, 1)	(1, -2, 1)	(2, 2, -2)	(0, 0, 0)

Table 3.3: Example of two equilibria in a 3-player game (Here, utility is in the order (A, a, α))

The game has two Nash equilibria, — (B, b, β) and (A, a, α) . The latter is attractive as it offers higher utility to all the players. Such focal points are important in analyzing a security game.

3.4.3.0.3 Challenges with multiple Nash Equilibria in a Security Game In case there are multiple NEs of a security game, if one of them implies security, does not imply security in general. The attackers would take the game towards an insecure equilibrium or rational players may play strategies resulting in equilibrium with higher utilities to them. Thus, we must explore all equilibria and ensure security at the worst equilibrium.

In our analysis, we are going to argue that in the generalized model in this paper, there are multiple equilibria and baiting-based equilibrium [146] that ensure the security of the protocol is one of them (similar to (B, β, b) in Table 3.3). We show in Theorem 3.3 that there is another equilibrium (similar to (A, α, a)) which may be more attractive to rational players. At the later equilibrium, the protocol is not secure. Thus, in the generalized model, TRAP [146] need not be secure.

3.4.3.0.4 Dominant Strategy Equilibrium (DSE). A better equilibrium from the weaker *stable Nash equilibrium* is a DSE. DSE equilibrium points are not contested by other

equilibrium points. Thus, we can safely assume rational agents follow a DSE strategy, and the protocol is Dominant Strategy Incentive Compatible (defined below).

Definition 3.5: Dominant Strategy Incentive Compatible

A protocol Π with a set of K rational players is *Dominant Strategy Incentive Compatible* (DSIC) for a given utility structure U if $\forall i \in K$ following honest strategy $s_i = \pi_0$ is Nash Equilibrium. i.e. $\forall i \in K, \forall \pi, \forall s_j \forall j \in K/\{i\}$

$$U_i(s_i = \pi, \{s_j\}_{j \neq i}) \leq U_i(s_i = \pi_0, \{s_j\}_{j \neq i})$$

We propose pRFT in Section 3.6 which is DSIC; therefore providing a better security guarantee.

3.5 Impossibilities

From the discussion of the previous section, rational players can be of different types $\theta \in \{0, 1, 2, 3\}$. We also know through Claim 3.1 that any protocol requires $\tau \in [\lfloor \frac{n+t_0}{2} \rfloor + 1, n - t_0]$ for security against *byzantine* attacks (Note that, this is necessary but not sufficient). We show through Theorem 3.1 that under a stricter (more adversarial) agent type for rational players, achieving RC for any $k + t > \frac{n}{3}$ is impossible.

Theorem 3.1 (Rational Consensus under $\theta = 3$). *Under the threat model $\langle (\mathcal{P}, T, K), \theta = 3, t_0 \rangle$ no rational consensus protocol is (t, k) -robust when $\lceil \frac{n}{3} \rceil \leq k + t \leq \lceil \frac{n}{2} \rceil - 1$.*

Proof. Consider a protocol Π belonging to the set of protocols $\mathcal{C}_3$ that achieve consensus under threat model $\mathcal{M} = \langle (\mathcal{P}, T, K), \theta = 3, t_0 \rangle$. Let $n = |\mathcal{P}|$ therefore, $|T| + |K| < \frac{n}{2}$. In such a case, consider the threshold of messages required for agreement by the protocol to be τ . From Claim 3.1 we have $\tau > \lfloor \frac{n+t_0}{2} \rfloor \geq \lceil \frac{n}{2} \rceil$ (since $t_0 \geq 1$). This means that for consensus, the message/signature of at least one player in $K \cup T$ is required. In this case, if each player $\mathcal{P}_l \in K \cup T$ follows strategy π_{abs} – not sending any messages, then consensus

is not reached. Since abstaining from sending messages in a round is indistinguishable from message delays due to partially-synchronous networks, π_{abs} cannot be distinguished from π_0 under partially-synchronous network. Therefore, $D(\pi_{abs}, \sigma) = 0$. The utility for following π_{abs} for each rational player is

$$\begin{aligned} U_i(\pi_{abs}, \theta = 3) &= \sum_{r=0}^{\infty} \delta^r u_i(\pi_{abs}, 3, r) \\ &= E_{\sigma \sim S}[f(\sigma, 3)] - 0 \\ &= \alpha > 0 = U_i(\pi_0, \theta = 3) \end{aligned}$$

Therefore, each player in set K is incentivized to play π_{abs} over following protocol which compromises (t, k) -eventual liveness property and therefore any such arbitrary Π is not a (t, k) -robust Rational Consensus protocol. $\square$

We also show a pessimistic result for rational players of $\theta = 2$. We show through Theorem 3.2 that for every protocol if the rational players are of type $\theta = 2$ then there always exists a strategy which compromises (t, k) -censorship resistance while ensuring (t, k) -eventual liveness. Note that this impossibility holds despite of existence of threshold encryption schemes [145],

Theorem 3.2 (Rational Consensus under $\theta = 2$). *Under threat model $\langle (\mathcal{P}, T, K), \theta = 2, t_0 \rangle$ no rational consensus protocol is **strongly** (t, k) -robust when $\lceil \frac{n}{3} \rceil \leq t+k \leq \lceil \frac{n}{2} \rceil - 1$.*

Proof. The proof follows by showing the existence of a strategy π_{pc} where adversarial collusion set follows (1) π_{abs} when the leader is honest, (2) π_0 (and omit censored transactions) when the leader is adversarial. Due to the indistinguishability between crash faults and π_{abs} , no accountable protocol is possible.

The proof follows by showing a strategy followed by $\mathcal{P}_i \in K \cup T$ that is incentivized for rational players and following this strategy, for any protocol Π it is impossible to achieve strongly (t, k) -robust RC. In Step 1, we describe the strategy, in Step 2, we show that

rational players are incentivized to follow this strategy and in Step 3 we show that for any protocol Π it is impossible to achieve RC under this strategy.

Step 1 (Strategy π_{pc}): From Theorem 3.1 we have that for any protocol Π with $\tau \geq \lfloor \frac{n+t_0}{2} \rfloor + 1$ in any round r , the collusion $K \cup T$ can cause disagreement by following π_{abs} . Let us consider a transaction tx_h which is input to all honest players by round r_0 . The strategy which $\mathcal{P}_i \in K \cup T$ follows for round $r \geq r_0$ is:

- If leader in round r is $\mathcal{P}_l \notin K \cup T$ then follow π_{abs} .
- If leader in round r is $\mathcal{P}_l \in K \cup T$ then propose Block with transaction set $\overline{tx}$ such that $tx_h \notin \overline{tx}$.

We abbreviate this strategy as π_{pc} (**p**artial-**c**ensorship).

Step 2 (Incentive Compatibility): We now show that following π_{pc} is incentivized for rational players $\mathcal{P}_i \in K$. We first make a simple observation that in from round r_0 to $r_0 + n - 1$, in expectation there will be $k + t$ blocks mined (when the leader is $\mathcal{P}_l \in K \cup T$). Therefore, the protocol achieves (t, k) -eventual liveness. In addition, since there are no conflicting values proposed in any round, no disagreement is reached. The rational players are of type $\theta = 2$ which means their utility from round r_0 onwards is given by

$$U_i(\pi, \theta = 2) = \sum_{r=r_0}^{\infty} \delta^r (\mathbb{E}[\alpha f(\sigma, 2)] - L \cdot D(\pi))$$

Since system will not be in state σ_{NP} , the payoff from $\mathbb{E}[\alpha_2 f_2] > 0$ if probability $Pr(\sigma = \sigma_{CP}) > 0$ (according to our strategy $Pr(\sigma = \sigma_{CP}) = 1$). In addition, since there are no duplicate messages signed, and players do not crash forever, following π_{pc} is indistinguishable from following π_0 which means $D(\pi_{pc}) = D(\pi_0) = 0$. This means $\forall \mathcal{P}_i \in K \cup T$

$$U_i(\pi_{pc}, \theta = 2) > U_i(\pi_0, \theta = 2)$$

The set $K \cup T$ is therefore incentivized to deviate from the honest protocol π_0 to follow π_{pc} for any protocol Π .

Step 3 (No strongly (t, k) -robust Rational Consensus: We now argue that if $K \cup T$ follows π_{pc} for any consensus protocol then it is impossible to achieve **strongly (t, k) -robust RC** for any $\frac{n}{3} \leq k+t < \frac{n}{2}$. Consider any round $r \geq r_0$. In this round, if leader $\mathcal{P}_l \in K \cup T$ the leader selectively includes transactions in transaction set $\overline{tx}$ such that $tx_h \notin \overline{tx}$. If $\mathcal{P}_l \notin K \cup T$ the coalition causes view change without agreement on a block. Thus, any block confirmed (and thus included) doesnot contain transactoin tx_h although the transaction is input to all honest players at round r_0 . This violates the (t, k) -**censorship resistance** and therefore for any arbitrary protocol⁷ Π achieving **strongly (t, k) -robust RC** is impossible. $\square$

Having proven the impossibility of having a protocol that achieves RC for rational player types $\theta = 3$ and 2 , we now relax the player type further to $\theta = 1$. Under this utility model for rational players, Gramoli et al. [146] proposed a Baiting-based protocol (which they call TRAP) and show Baiting is *necessary* and *sufficient* to achieve RC under partially-synchronous network settings for $t+k < \frac{n}{2}$. Following our brief discussion of the model and result of [146] in Section 3.3.4, we show that the Baiting-based protocol does not ensure RC in repeated rounds of consensus due to the existence of another Nash equilibrium point which leads to disagreement. Further, this point being focal equilibria (see discussion in Section 3.4.3), it will be preferred over the secure equilibria point proposed by [146]. We demonstrate this result in Theorem 3.3 below. For notational consistency of this result with [146] we use $R := f(\sigma_{fork}, 1)$

Theorem 3.3 (Baiting based Rational Consensus under $\theta = 1$). *Consider any baiting-based RC protocol Π the threat model $\mathcal{M} = \langle (\mathcal{P}, K, T), \theta = 1, t_0 \rangle$. The set of rational players following π_{fork} is a Nash-equilibrium for $|K| > 2 + t_0 - t$. Thus, Π is not (t, k) -robust RC for $t_0 = \lceil \frac{n}{3} \rceil - 1$.*

Proof. To prove this, we show that the payoff for a rational player $\mathcal{P}_i$ on following π_{bait} is lesser than the payoff if they follow π_{fork} (which is the strategy followed by the collusion).

⁷we did not assume any property about the protocol

The net gain in payoff for the collusion $K \cup T$ is G if the system ends up in state σ_{Fork} which is distributed among the rational players. If the player $\mathcal{P}_i \in K$ follows π_{bait} then she is not a part of the collusion and will get 0 payoff if the system ends up in σ_{Fork} .

The payoff for $\mathcal{P}_l$ on following π_{Fork} along with the rest of the coalition $K \cup T$ is therefore

$$U_i(\pi_{Fork}, 3) = \frac{G}{k}$$

Consider for $|K| > 2 + t_0 - t$ i.e. $k + t > 2 + t_0$. If $\mathcal{P}_l$ deviates from the coalition $K \cup T$ to follow π_{bait} . The system can end up in two possible states. σ_{Fork} is if the rest of the collusion $T \cup K/\{\mathcal{P}_l\}$ is able to create a fork/disagreement despite $\mathcal{P}_l$ following π_{bait} . In this case, the utility for $\mathcal{P}_l$ is 0 (since they were not a part of the collusion). Second is if the Proof-of-Fraud submitted by $\mathcal{P}_l$ is accepted and the protocol functions normally σ_0 . The payoff in this case is

$$U_i(\pi_{bait}, 3) = R \cdot Pr(\sigma = \sigma_0) + 0 \cdot Pr(\sigma = \sigma_{Fork})$$

However, if $k + t > 2 + t_0$ then we can achieve disagreement even if $1 + t_0$ players follow π_{Fork} (since $t_0 + 1 = \lceil \frac{n}{3} \rceil$). This can be under the partition of $\mathcal{P}/(T \cup K/\{\mathcal{P}_i\})$ into two disjoint sets A, B such that $|A|$ and $|B|$ are such that $|A| + k + t \geq \tau$ (and similarly $|B|$). For forking, we have to ensure each partition has at $n - t_0$ messages. If m players from the collusion deviates to follow π_{bait} the condition for system to **not** end in $\sigma = \sigma_{Fork}$ is

$$|A| + (k - m) + t < \tau < n - t_0$$

$$|A| + k + t - n + t_0 < m$$

We have $n - |B| = |A| + k + t$ and $|B| = \frac{n-t-k}{2}$. Therefore,

$$m > t_0 + \frac{k + t - n}{2}$$

For $k > 3 > (3t_0 + 1) - 2t_0 - t + 2 = n - 2t_0 - t + 2$, we have from algebraic reordering the RHS of inequality > 1 . Thus, any unilateral deviation ($m = 1$) is not sufficient to avoid σ_{Fork} . Therefore, $Pr(\sigma = \sigma_{Fork}) = 1$ and $Pr(\sigma = \sigma_0) = 0$. The utility is therefore 0 which

gives us $U_i(\pi_{ForK}, 3) > U_i(\pi_{bait}, 3)$. Following π_{abs} will also lead to $\mathcal{P}_i$, not part of $K \cup T$ and therefore the payoff is 0. Following π_0 leads to payoff 0 for any system state. Thus, following π_{ForK} is Nash Equilibrium strategy for $\mathcal{P}_l \in K$. $\square$

In this section, we demonstrated the impossibility of achieving RC when rational players are of the type $\theta = 3$ or 2 and $k + t > \frac{n}{3}$. We further show that when rational players are of type $\theta = 1$, the previously existing “baiting-based” solution [146] has a Nash equilibrium strategy, resulting in Disagreement. From our discussion in Section 3.4.3, this is a more stable equilibrium point than the secure equilibrium point proposed by [146] (that relies on $m > \frac{t+k-n}{2} + t_0$ deviating together; ref. Lemma 4.3 and Lemma 4.4 in [146]). In the next section, we propose a protocol **pRFT** which achieves (t, k) -robust RC without relying on baiting by rational players.

3.6 pRFT: Rational Consensus Protocol

Baiting-based consensus [146] introduces interesting insights about using Proof-of-Fraud (PoF) to penalize deviating rational players. The protocol relies on incentivizing rational players to bait the collusion, and we show in Theorem 3.3 existence of another (better) Nash Equilibrium for any rational player to deviate to π_{ForK} than follow honest-baiting π_{bait} strategy. To solve this problem, we attempt to capture PoF through honest players prone to any incentive manipulation. Towards this, we propose **pRFT**, which is described below.

3.6.1 Protocol

The **pRFT** protocol runs in discrete rounds. The set of players involved are $\mathcal{P} = \{\mathcal{P}_1, \mathcal{P}_2, \dots, \mathcal{P}_n\}$ and in round r the leader is $\mathcal{P}_l$ for $l = 1 + (r \bmod n)$. We assume the network is partial-synchronous, and reliable-broadcast i.e. messages reach to the receiver untampered, although they might suffer network delays. Note that for brevity, we

abstract the cryptographic verification of the message to be done by the Recv procedure (lines 7, 11, 17, 24, 27 and 29 of the protocol in Figure 3.1). Therefore, any message coming through it will contain only valid signatures, and invalid messages are discarded. Each round progresses in 4 distinct phases, described as follows:

3.6.1.1 Propose Phase

The leader $\mathcal{P}_l$ has a set of transactions that they want to publish to the ledger (blockchain). $\mathcal{P}_l$ selects a set of these transactions $\overline{tx} = \{tx_1, tx_2 \dots, tx_c\}$ and form a Block B_r . She then proposes this block by broadcasting over the network via a *propose* message with their cryptographic signature s_l on the hash $h_l := \mathbf{H}(\text{Block}||r)$ of the Block⁸. They construct message $m := \langle \text{Propose}, B_l, h_l, r \rangle$ and signature s_l^{pro} to broadcast (m, s_l^{pro}) . All non-leader players $\mathcal{P}_i$ receive the propose message from the leader and move to the vote phase. The non-leader players $\mathcal{P}_i \forall i \neq l$ (1) check the validity of the *propose* message from the leader $\mathcal{P}_l$ (2) broadcast a *vote* message if the *propose* message is valid. In checking the validity of the *propose* message, the player verifies $H_l = \mathbf{H}(\text{Block}||r)$ and verifies signature s_l on the message $\langle \text{Propose}, B_l, h_l, r \rangle$. They then sign s_i^{vote} on the message $m_i^{vote} := \langle \text{Vote}, h_i, s_l^{pro}, r \rangle$ and broadcast (m_i^{vote}, s_i^{vote}) over the network.

3.6.1.2 Vote Phase

In this phase, they wait for $n - t_0$ *valid* vote messages from other players from the previous phase for some (same) proposed value h_* . If no such value is obtained, the player sets $h_* := \perp$ (default empty value). Each player commits to the value h_* by constructing a Commit message. This consists of the decided value h_* and set V_i of $\geq n - t_0$ votes on this value $V_i = \emptyset$ if $h_* = \perp$. The player $\mathcal{P}_i$ signs on the message $m_i^{com} := \langle \text{Commit}, h_*, s_l^{pro}, V_i, r \rangle$ and broadcasts (m_i^{com}, s_i^{com}) to all other players.

⁸ h_l also contains the round number therefore, signed messages from one round can not be used in another round using replay attack.

3.6.1.3 Commit Phase

Upon receiving $\geq n - t_0$ commit messages for a particular (same) value h_{tc} , the player reaches *tentative-consensus* on this block. Each player $\mathcal{P}_i$ shares their tentative consensus by sharing value h_{tc} which got $\geq n - t_0$ commit messages and a Proof-of-Commitment which is the set W_i (for player $\mathcal{P}_i$ of $n - t_0$ signatures on the commit messages on value h_*). They construct message $m_i^{rev} := \langle \text{Reveal}, h_{tc}, h_l, W_i, r \rangle$ and signature s_i^{rev} on this message to broadcast (m_i^{rev}, s_i^{rev}) .

3.6.1.4 Reveal Phase

Each player verifies across the set of Proof-of-Commitments W_j for each (m_j^{rev}, s_j^{rev}) received by $\mathcal{P}_i$ any attempts of *double-signature* in the proof vectors. Each player accumulates a set of double signatures as Proof-of-Fraud (PoF) by invoking the **ConstructProof** procedure (see Figure 3.4). These PoF can be used to *burn* the tokens/coins deposited by the deviating player by including a corresponding transaction in a future block. If there are $\geq n - t_0$ messages and a total of $\leq t_0$ players with double signatures (PoF) then the player reaches *Final Consensus* on the proposed block B_l . In this case, they construct a message $m_i^{fin} := \langle \text{Final}, h_l, s_l^{pro} \rangle$ and signature s_i^{fin} on it. They then broadcast this pair (m_i^{fin}, s_i^{fin}) to the client/network. Otherwise, if the set of double signatures $\geq t_0 + 1$, they broadcast this PoF with $\geq t_0 + 1$ double-signatures (represented by set D_i). They then construct message $m_i^{exp} := \langle \text{Expose}, D_i, r \rangle$ and signature s_i^{exp} and broadcast (m_i^{exp}, s_i^{exp}) . If the player obtains $> \frac{n}{2}$ Final messages, this means at least one honest player has finalized on the block. Then, this player also finalizes and broadcasts a final message.

3.6.2 View Change

In each round players wait for proposal messages in the proposed phase or $\geq n - t_0$ messages in the other phases. Either due to delays in the network or $> t_0$ players deviating from the protocol, if a timeout happens (when the duration of phase exceeds the local

waiting time of Δ) then *view change* is initiated. The view change protocol proceeds as follows:

1 A player triggers view-change if the following happens:

- a timeout in waiting time Δ
- conflicting signatures on 2 different values v_{l1}, v_{l2} in the same round by leader $\mathcal{P}_l$
- conflicting signatures by $\geq t_0 + 1$ players in some phase.

Under either of these scenarios, the player signs the message $m_i^{vc} := \langle \text{View-Change}, \text{Phase}, r, \rangle$ with signature s_i^{vc} and broadcast (m_i^{vc}, s_i^{vc}) to the network.

2 If a player $\mathcal{P}_i$ receives a view-change message from player $\mathcal{P}_j$ for some phase in some round, they store the message (1) Wait for $\geq n - t_0$ such messages (from the same phase) or timeout or, (2) If they have $\geq n - t_0$ messages from that phase, they send the corresponding messages to $\mathcal{P}_j$.

3 If a player receives $\geq n - t_0$ *view-change* messages including their own *view-change* (represent this set as V_i), they construct a *commit-view* message $m_i^{cv} := \langle \text{CommitView}, V_i, r \rangle$ along with the signature s_i^{cv} . The player will discontinue the current round and wait for a view change.

4 If a player receives a commit-view message, they verify if it consists of $n - t_0$ valid (signed) view-change messages. If so, they commit to view change and send a *commit-view* message.

5 When the player receives $> n - t_0$ commit-view messages, they commit to view change and change round from r to $r + 1$ and begin the corresponding propose phase.

The view change sub-protocol should ensure the following two properties:

- **Consistency:** If a player $\mathcal{P}_i \in H$ has committed to view-change, then any other player $\mathcal{P}_j \in H$ should also eventually commit to view-change (should not reach agreement in r).

- **Robustness:** The set T cannot launch a view-change if the leader is honest $\mathcal{P}_l \in H$.

We show in the following Claim 3.2 that the view-change protocol of pRFT satisfies both of these properties.

Claim 3.2. *The **view-change** sub-protocol of pRFT satisfies both **Consistency** and **Robustness**.*

Proof. We prove Consistency through contradiction. Consider $\mathcal{P}_1 \in A$, $\mathcal{P}_2 \in B$ such that $A, B \subset H, A \cap B = \emptyset$. Assume consistency does not hold, i.e. $\mathcal{P}_1$ commits to view-change and broadcasts *CommitView* message (ref. Table 3.3). Since communication channels are reliable and messages are not dropped, the only way to disrupt consistency is if $\mathcal{P}_2$ reaches agreement before this *CommitView* message from $\mathcal{P}_1$ reaches $\mathcal{P}_2$. For this, we require $|B| + k + t \geq n - t_0$. However, for a valid *CommitView* we require $|A| + k + t \geq n - t_0$. Adding them up, $|A| + |B| + 2(k + t) > 2n - 2t_0$. This gives us $n + k + t > |A| + |B| + 2(k + t) \geq 2n - 2t_0$ and on rearrangement $k + t + 2t_0 > n$ which is a contradiction for the considered threat model $\mathcal{M} = \langle (\mathcal{P}, T, K), \theta = 1, \frac{n}{4} \rangle$. Therefore, consistency is satisfied.

For robustness, consider in a round with honest leader $\mathcal{P}_l \in H$. In this case, if $\mathcal{P}_i \in T$ broadcast *ViewChange* message, and abstain from participation, still the protocol is able to gather $\geq n - t_0$ message. This is because (1) $t \leq t_0$ and (2) rational players are of type $\theta = 1$ due to which they are disincentivized from causing liveness attack. Hence, not enough ($\geq n - t_0$) view change messages are gathered and T cannot cause view-change by themselves. The protocol therefore satisfies Robustness property. $\square$

3.6.3 Discussion

The protocol leverages Proof-of-Fraud (PoF) to penalize rational players on the following π_{ds} . In addition, similar to [54] pRFT uses **tentative** and **final** consensus. We discuss (i) how PoF is realized in pRFT, (ii) the advantage of tentative and final consensus, and (iii) message complexity of pRFT.

3.6.3.1 Accountability and Proof-of-Fraud (PoF)

pRFT implements penalty mechanism via Proof-of-Fraud. Each player that is a part of the consensus committee $\mathcal{P}$ deposits some amount L as collateral. This collateral is locked unless some specified q number of blocks are mined. If there is some malicious behaviour by player $\mathcal{P}_i$, and a PoF exists against them, this PoF can be used as an input to the transaction to burn the collateral L of the player $\mathcal{P}_i$. Due to space constraints, we formally present the PoF construction in Figure 3.4. The property where deviation of more than t_0 (for some t_0) players is captured along with the identities of deviating players is called accountability and some existing consensus protocols provide accountability [149, 146, 148] of deviating players. Motivating from [149, Definition 1] we define Accountability as follows:

Definition 3.6: Accountability

If two honest parties output different values, then eventually all honest parties reach a state s_j and receive/construct some Proof-of-Fraud (PoF) $\pi \in \{0, 1\}^*$ such that $\exists$ verification algorithm $V(\cdot)$ the value $V(\pi)$ outputs set of $\geq t_0 + 1$ deviating (guilty) players.

3.6.3.2 Tentative and Final Consensus

After the correct execution of phases 1-3 (propose, vote and commit) of the pRFT, each player reaches a *tentative consensus* on the Block B_r . B_r reaches *final consensus* after the correct execution of step 4 if no $\mathcal{P}_i \in K$ deviates from the protocol.

3.6.3.2.1 Effectiveness of Tentative Consensus It is interesting to observe that tentative consensus will be finalized unless a rational player deviates from the protocol. For rational players of type $\theta = 1$ they are incentivized only in system state σ_{Fork} for which they would have to follow π_{ds} . However, as we show in the following Section 3.7,

this deviation can be caught in phase 4 of the protocol. Due to this, any $\mathcal{P}_i \in K$ is disincentivized from deviating from the protocol, ensuring that Tentative Consensus will convert to Final Consensus.

3.6.3.3 Message Complexity

In normal executions, pRFT uses all-to-all broadcasts in 4 phases, leading to the message complexity n^3 . Additionally, in Vote, Commit and Reveal phases, they share a set of signatures on messages from previous rounds. Therefore, the message complexity becomes $\kappa \cdot n^4$ for security parameter κ used in PKI. The message complexity and size are on par with the best protocols that provide accountability, such as [149] (while pRFT tolerates a stricter adversary model compared to these solutions). We present this comparison in Table 3.5 (from [149, Table 1]).

pRFT($\bar{p}_{i=1}^n, t_0$)

1: Propose Phase: 2: if $i = r \bmod n$ then 3: $\text{Block}_r := \text{ConstructBlock}(\overline{tx}, r, b_{par}, p_i)$ 4: $h_l := \text{Hash}(\text{Block}_r)$ 5: Broadcast($\langle \text{Propose}, B_r, h_l, r \rangle_i, s_i^{pro}$) 6: else 7: On Recv. ($\langle \text{Propose}, B_r, h_l, r \rangle, s_l^{pro}$): 8: Broadcast($\langle \text{Vote}, h_l, s_l^{pro}, r \rangle_i, s_i^{vote}$) 9: end if 10: Vote Phase: 11: On Recv. ($\langle \text{Vote}, h_j, s_l^{pro}, r \rangle, s_j^{vote}$): 12: $\text{votes}[h_j] := \text{votes}[h_j] \cup \{s_j^{vote}\}$ 13: if for some h_*, $\text{vote}[h_*].size \geq n - t_0$ then 14: Broadcast ($\langle \text{Commit}, h_*, s_l^{pro}, \text{vote}[h_*], r \rangle_i, s_i^{com}$) 15: end if 16: Commit Phase: 17: On Recv. ($\langle \text{Commit}, h_j, s_l^{pro}, \overline{\text{vote}}_j, r \rangle, s_j^{com}$): 18: $\text{commit}[h_j] := \text{commit}[h_j] \cup \{s_j^{com}\}$ 19: if for some h_*, $\text{commit}[h_*].size \geq n - t_0$ then 20: Broadcast($\langle \text{Reveal}, h_*, s_l^{pro}, \text{commit}[h_*], r \rangle_i, s_i^{rev}$) 21: end if	22: Reveal Phase: 23: $D_i := \emptyset, M_i := \emptyset, F_i := \emptyset$ 24: On Recv. ($\langle \text{Reveal}, h_j, s_l^{pro}, \overline{\text{com}}_j, r \rangle, s_j^{rev}$): 25: $M_i \leftarrow M_i \cup \{\overline{\text{com}}_j\}$ 26: $D_i := \text{ConstructPoF}(M_i)$ 27: On Recv. ($\langle \text{Final}, B_j, s_l^{pro} \rangle_j, s_j$) 28: $F_i \leftarrow F_i \cup \{s_j^{fin}\}$ 29: On Recv. ($\langle \text{Expose}, D_j, r \rangle, s_j$) 30: Stash($D_j$), $r := r + 1$ 31: if $D_i > t_0$ then 32: Broadcast($\langle \text{Expose}, D_i, r \rangle_i, s_i^{expose}$) 33: else if $M_i \geq n - t_0$ then 34: Broadcast($\langle \text{Final}, B_l, s_l^{pro} \rangle_i, s_i^{fin}$) 35: else if $F_i > \frac{n}{2}$ then 36: Broadcast($\langle \text{Final}, B_l, s_l^{pro} \rangle_i, s_i^{fin}$) 37: end if
---	---

Figure 3.1: pRFT Protocol

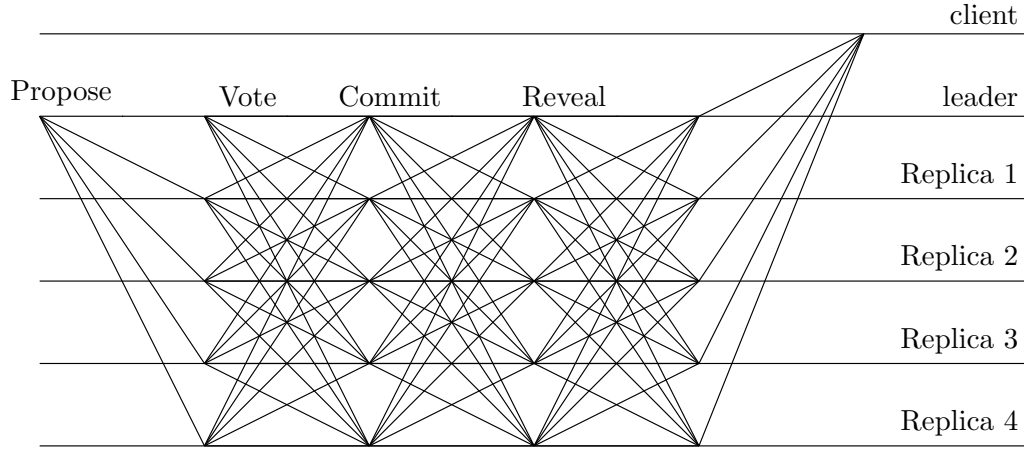


Figure 3.2: Normal execution of pRFT protocol

Message Type	Message
<i>propose</i>	$((Propose, B_l, h_l, r), s_l^{pro})$
<i>vote</i>	$((Vote, h_i, s_l^{pro}, r), s_i^{vote})$
<i>commit</i>	$((Commit, h_*, s_l^{pro}, V_i, r), s_i^{com})$
<i>reveal</i>	$((Reveal, h_{tc}, h_l, W_i, r), s_i^{rev})$
<i>expose</i>	$((Expose, D_i, r), s_i^{exp})$
<i>final</i>	$((Final, h_l, s_l^{pro}), s_i^{final})$
<i>view-change</i>	$((ViewChange, Phase, r), s_i^{vc})$
<i>commit-view</i>	$((CommitView, V_i, r), s_i^{cv})$

Figure 3.3: Messages sent in pRFT

ConstructProof (M, t_0)	
1:	$D := \emptyset$
2:	for $i \in [n], j \in [n]/\{i\}$ do
3:	for $k \in [n]$ do
4:	if $M(i, k) \neq M(j, k)$ then
5:	$D \leftarrow D \cup \{(M(i, k), M(j, k))\}$ and goto 8
6:	end if
7:	end for
8:	if $ D \geq t_0 + 1$ then
9:	return D
10:	end if
11:	end for
12:	return D

Figure 3.4: Construction of Proof-of-Fraud (PoF)

Protocol	Message Complexity	Message Size	Accountability
pBFT [1]	$O(n^3)$	$O(\kappa \cdot n^4)$	×
Hotstuff [53]	$O(n^2)$	$O(\kappa \cdot n^3)$	×
Polygraph [149]	$O(n^3)$	$O(\kappa \cdot n^4)$	✓
pRFT	$O(n^3)$	$O(\kappa \cdot n^4)$	✓

†While polygraph achieves same guarantees, their threat model is much weaker than pRFT's

Figure 3.5: Message Complexity for different consensus protocols compared with pRFT

3.7 Theoretical Analysis of pRFT

We analyse the security and liveness of the protocol under partial synchronous network and threat model $\mathcal{M} = \langle (\mathcal{P}, T, K), \theta = 1, \lceil \frac{n}{4} \rceil - 1 \rangle$. Therefore in the worst case, $|T| = t_0$ and $n = 4t_0 + 1$. Further, $k + t < \frac{n}{2}$. First we show that irrespective of strategy followed by $\mathcal{P}_i \in T$ if remaining $\mathcal{P} \setminus T$ do not deviate from the protocol, agreement is reached on one value (in periods of synchrony) when leader is non-deviating. In a period of asynchrony, view-change (due to timeout) happens.

Claim 3.3. *In any round r of pRFT such that leader $\mathcal{P}_l \notin T$, irrespective of the strategy of the set T , if remaining $\mathcal{P} \setminus T$ play $\pi \neq \pi_{fork}$ then exactly one of the following holds:*

- *If network is synchronous^a and $\mathcal{P} \setminus T$ play honestly, agreement is reached on a single block B_l*
- *If network is asynchronous^b or some $\mathcal{P}_i \in K$ follows $\pi \notin \{\pi_{fork}, \pi_0\}$, timeout triggers view-change.*

^anote that this period of synchrony can happen either if network is synchronous or GST event has already happened in partially-synchronous network

^bhappens in partially-synchronous networks before GST event

Proof. The proof follows by showing that any arbitrary network partition of $\mathcal{P}/\mathcal{T}$ does not lead to ≥ 2 disjoint subsets A, B such that $A \cup T$ and $B \cup T$ are $\geq n - t_0$. Therefore, either timeout happens due to insufficient ($< n - t_0$) messages in all partitions or agreement is reached in exactly one partition. The complete proof is omitted due to space constraints. □

To prove that pRFT realizes (t, k) -robust RC, we first show through Lemma 3.1 that any $\mathcal{P}_i \in K$ is disincentivized from deviating from the protocol i.e. following any $\pi \neq \pi_0$. We finally conclude in Theorem 4.5 that pRFT is a (t, k) -robust RC protocol under the threat model $\mathcal{M}$.

Lemma 3.1. *For any $\mathcal{P}_i \in K$ under threat model $\mathcal{M} = \langle (\mathcal{P}, K, T), \theta = 1, \lceil \frac{n}{4} \rceil - 1 \rangle$ and protocol $\mathbf{pRFT}$, following the protocol honestly (i.e. strategy π_0) is dominant strategy incentive compatible (DSIC) for $|K| + |T| < \frac{n}{2}$ and $t < t_0$. That is, $U_i(\pi_0, 1) \geq U_i(\pi, 1) \quad \forall \pi, \forall \mathcal{P}_i \in K$.*

Proof. Consider any arbitrary rational player $\mathcal{P}_i \in K$. We show that by playing π_{fork} , $\mathcal{P}_i$ either: (1) get caught in the PoF, (2) cause view-change or (3) cause agreement on a single value. Consider π_{fork} is played by $\mathcal{P}_i$ and two honest players $\mathcal{P}_a, \mathcal{P}_b \in H$ receive conflicting signatures on value h_a, h_b (such that $h_a \neq h_b$). First, consider the case when network is synchronous (messages are reaching on time). In this case, signature on h_a reaches to $\mathcal{P}_b$ and similarly signature on h_b reaches $\mathcal{P}_a$. Either number of double signatures are $\leq t_0$ in which case, agreement is satisfied due to Claim 3.3. If number of double signature is $> t_0$ then PoD is constructed by either (or both) $\mathcal{P}_a$ or (and) $\mathcal{P}_b$ and $\mathcal{P}_i$ suffers penalty (with some non-zero probability). The payoff in this case is $u_i(\pi_{fork}, 1, r) = -L \cdot D(\pi_{fork}, \sigma) < 0$. Consider if the network is partially-synchronous and the network partition of honest nodes is such that A, B are two partitions and $\mathcal{P}_a \in A$ and $\mathcal{P}_b \in B$. If either partition is small enough that $k+t+|A| < n-t_0$ or $k+t+|B| < n-t_0$, then agreement is not reached in that partition. In this case, either agreement is not reached for both $\mathcal{P}_a$ and $\mathcal{P}_b$ or agreement is reached for exactly one of $\mathcal{P}_a$ or $\mathcal{P}_b$ (therefore on one same value). Let the probability of no agreement be q_d and agreement on exactly one value be q_a . The utility is therefore, $u_i(\pi_{fork}, 1, r) = q_d f(\sigma_{NP}, 1) = -q_d \alpha \leq 0$. Notice that it cannot be that agreement is reached in both partitions, as in that case, $|A| + k + t > n - t_0$ and $|B| + k + t > n - t_0$. As $|A| + |B| < |H|$, Adding them up, we have $2n - 2t_0 \leq n - (k+t) + 2(k+t) \Rightarrow n < k+t+2t_0$.

However, according to our threat model $\mathcal{M}$, $k+t < \frac{n}{2}$ and $t_0 < \frac{n}{4}$. Therefore, $k+t+2t_0 < n$. Therefore, such a case is not possible in our threat model. Therefore, if the network in (or during) that round is synchronous, $u_i(\pi_{fork}, 1, r) < 0$ and under partially-synchronous

network $u_i(\pi_{fork}, 1, r) \leq 0$. Therefore, the expected utility for the round is $\forall i \in [n], r \in \mathbb{R}$

$$\mathbb{E}_{\sigma \sim S}[u_i(\pi_{fork}, 1, r)] \leq 0 \implies U_i(\pi_{fork}, 1) \leq 0 = U_i(\pi_0, 1)$$

Following any other strategy will lead to at most t_0 double signatures (from byzantine players) and from Claim 3.3, the protocol will either reach an agreement or view-change (through timeout or duplicate values proposed by the adversarial leader). Additionally, since rational players are of type $\theta = 1$, they will not try to cause censorship of transactions. Therefore, following $\pi \notin \{\pi_0, \pi_{fork}\}$ for $\mathcal{P}_i \in K$ gives

$$U_i(\pi, 1) \leq 0 = U_i(\pi_0, 1)$$

Hence, following π_0 gives more payoff than any other strategy. Thus following pRFT is DSIC for any $\mathcal{P}_i \in K$. $\square$

From Lemma 3.1, we conclude that any rational player of type $\theta = 1$ will follow pRFT honestly under the threat model $\mathcal{M}$ as described above. Our discussion from Section 3.6.3.2 implies that if all players in K behave rationally, then all tentative consensus will be converted to final consensus except during timeout due to network asynchrony. We now conclude in Theorem 4.5 that pRFT realizes (t, k) -robust RC.

Theorem 3.4. *pRFT is a (t, k) -robust rational consensus protocol under threat model $\mathcal{M} = \langle (\mathcal{P}, T, K), \theta = 1, \lceil \frac{n}{4} \rceil - 1 \rangle$ for $|K| + |T| < \frac{n}{2}$ under synchronous and partial-synchronous networks.*

Proof. From Lemma 3.1 we know all K follow π_0 . All H follow π_0 by definition of Honest players. Therefore, only deviating players are from the set T . From Claim 3.3 we know that in such a scenario, one of two things happen – view-change (if the network is in a period of asynchrony), or agreement (if all messages from a particular phase has been delivered. In synchronous settings, agreement is trivially satisfied due to Claim 3.3. In partially-synchronous setting, all messages from a round are eventually delivered before

the next GST. Therefore, some block B_r (and therefore all blocks before B_r) are finalized during the period of synchrony. Since $n - t_0$ are following π_0 (and not trying to cause censorship attack), pRFT is a **strongly (t, k) -robust RC** protocol. $\square$

To summarize, we have shown through Lemma 3.1 that following honest strategy is DSIC for all players $\mathcal{P}_i \in K$ in partially-synchronous network. Together with Claim 3.3 this means under synchronous and partially synchronous networks, pRFT is **strongly (t, k) -robust** (Theorem 4.5).

3.8 Conclusion

In this chapter, we investigate Rational Consensus (RC) in partially synchronous networks. We begin by extending the $RFT(k, t)$ model of [146] to support repeated consensus rounds with discounting of each round's payoff. We furthermore refine the taxonomy of rational participants into four incentive classes, indexed by $\theta \in \{0, 1, 2, 3\}$. In Section 3.5, we prove that when $\frac{n}{3} < k + t < \frac{n}{2}$ and rational nodes are motivated to attack liveness or censor transactions, no protocol can achieve **strongly (t, k) -robust** consensus. We also demonstrate that the TRAP protocol of [146] admits an undesirable Nash equilibrium—preferred by rational players over the intended outcome (see Section 3.4.3).

We then present pRFT, which attains RC under the conditions $k + t < \frac{n}{2}$ and $t_0 < \frac{n}{4}$. We show that pRFT meets pRFT security in partially synchronous settings and is **strongly (t, k) -robust**. Moreover, pRFT achieves optimal message complexity $O(n^3)$ and per-node message size $O(\kappa n^4)$. By eliminating dependence on baiting strategies, our work broadens the theory of incentive-compatible Atomic Broadcast in rational adversarial models.

Chapter 4

Study of Centralization in PoW Blockchains

“The individual only has power in so far as he ceases to be an individual. You know the Party slogan ‘Freedom is Slavery.’ Has it ever occurred to you that it is reversible? Slavery is freedom. Alone-free-the human being is always defeated. ”

— 1984, George Orwell

Institutionalize power has often led to the eradication of the idea of individuality. This accumulation of power poses a threat to the system being aligned differently from the participants it represents. Institutions often buy control by offering “freedom” from responsibility (as the famous quote by George Orwell above states). Similar challenge of institutionalized aggregation of power is seen in Proof-of-Work blockchains due to the formation of mining pools. This control is taken from solo miner for the exchange of “convenience” of being part of the blockchain system.

We discuss in this chapter the problem of formation of mining pools and solo miner’s incentive to join such institutions. We attribute this problem to the design of the incentive structure of the system. We propose an alternate incentive structure which eliminates the incentive to join mining pools.

4.1 Introduction

In Proof-of-Work (PoW) systems, network participants—commonly known as miners—allocate computational resources to solve a cryptographic challenge. Successfully finding a solution allows a miner to append a new block to the chain and claim the associated block reward. Each miner’s effectiveness is measured by its hash rate, i.e., the number of hash attempts it can perform per unit time. As PoW chains have grown in popularity and mining hardware has evolved from CPUs to GPUs and now ASICs, an ongoing “arms race” has driven the aggregate network hash rate to record heights. Although the expected payoff per miner remains constant, the variance of those rewards has risen significantly.

Mining Pools. To smooth out the high variance of solo mining, individuals combine their hash power into [mining pools](#). While each member’s share of every block’s reward diminishes, the pool’s collective chance of finding blocks increases, yielding more frequent but smaller payouts. Various reward-sharing schemes (RSS)—including Pay-Per-Share (PPS), Pay-Per-Last-N-Shares (PPLNS), Proportional, and Geometric—have been proposed. Under specific assumptions, PPS and PPLNS minimize reward variance among participants [154]. As a result of their lower-risk profile, mining pools have surged in popularity, with a handful growing so large that they now pose a potential threat to the decentralization guarantees of PoW protocols.

Protocol Designs to Mitigate Centralization. Several proposals have aimed to discourage pool formation and preserve decentralization in PoW networks [155, 156, 157, 158, 159, 160, 161, 162]. Some reduce block-proposal variance directly [157, 158], while others modify the block-reward mechanism itself to achieve a similar effect [162, 155]. Despite these efforts, miners’ inherent preference for lower-variance returns continues to drive them into pools, leaving the system vulnerable to centralization [163].

Background on Decentralized PoW Cryptocurrencies. With the rise of peer-to-peer, decentralized digital currencies like Bitcoin and Ethereum, demand for these tokens

has skyrocketed. New coins enter circulation either through purchase or by being mined directly by network participants. A distributed ledger records all transactions, and consensus on its state is maintained via a PoW protocol [4]. In this model, anyone can attempt to mine the next block by solving a cryptographic puzzle—the first to succeed earns freshly minted coins according to the network’s block reward rules.

Growth and Centralization of Mining Pools. Over the past five years, Bitcoin’s total mining power has increased tenfold [164], as illustrated in Figure 4.2. Because solo miners face long, uncertain waits for block rewards, they frequently join pools to receive steadier payments. However, this trend has led to outsized concentration: the top three Bitcoin pools—Foundry USA ($\approx 29\%$), Ant Pool ($\approx 16\%$), and F2 Pool ($\approx 14\%$)—now command about 59% of the network’s hash rate (Figure 4.1). Similarly, in ZCash, Nanopool ($\approx 25\%$), Mining Pool Hub ($\approx 18\%$), and Suprnova ($\approx 8\%$) collectively control over half the mining power [165]. Such levels of centralization undermine the security assumption that no single entity (or coalition) wields a majority of computational resources.

Risks of Centralization PoW security hinges on an honest-majority assumption: that over 50% of hash power is controlled by well-intentioned participants. When a few pools aggregate more than half the network’s power, they gain the ability to censor transactions, launch denial-of-service attacks, or execute double-spending (51%) attacks [166]. This threat is not hypothetical—GHash.IO briefly exceeded 55% of Bitcoin’s hash rate in June 2014, triggering community outcry and a pledge to cap its share below 40% thereafter [167]. Addressing mining-pool centralization is therefore critical to maintaining PoW blockchains’ trustworthiness.

Miners, motivated by maximizing expected reward while minimizing risk, naturally gravitate toward pooling. Game-theoretic analyses have examined pool-joining behavior and optimal RSS [154, 168], as well as strategies for maximizing miner utility [169, 170]. Yet a comprehensive evaluation of how these incentives drive PoW centralization remains an open area for investigation.

Our Goal

This chapter develops a game-theoretic framework for a miner’s decision to join a PoW blockchain, identifying the factors that push the network toward either centralization or sustained decentralization. Moreover, we introduce a mechanism that ensures solo mining is never less profitable than pool participation, thereby preserving a decentralized structure.

Our Approach Let each block-reward mechanism (BRM) be denoted by Γ . We then define a two-player game $\mathcal{G}(\Gamma)$ between the prospective miner p_1 and the PoW blockchain system modeled as player p_2 . The utility of p_1 combines three components:

1. *Expected reward*, which increases with the miner’s hash power.
2. *Risk*, captured by the ρ -th moment of the reward distribution—a generalization of variance as a risk measure [171, 154].
3. *Switching costs*, reflecting the overhead (e.g., additional hash queries, latency) incurred when moving between pools (see Section 4.4.2).

We define two key system properties: (i) *Fairness*, following the unbiased reward-sharing criteria of [154], and (ii) ρ -*Decentralization*, which limits the aggregate power of any coalition. Finally, we categorize BRMs as

- *memoryless*, where each block’s reward is independent of prior state (e.g., Bitcoin [4]); and
- *retentive*, where payouts depend on historical ledger information (e.g., Fruitchain [162]).

Our Contributions

We begin by examining memoryless BRMs, incorporating a switching-cost parameter c with cost function D . When $c = 0$ (no switching penalty), a miner p_1 can freely distribute hash power among pools, enabling a fully decentralized outcome. However, we prove that

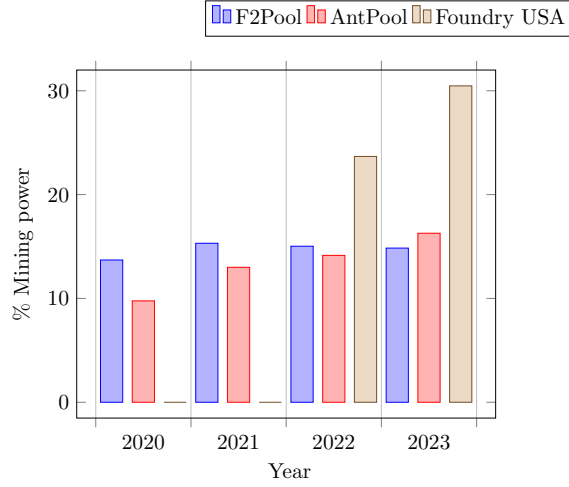


Figure 4.1: Centralization in Bitcoin over 4 years [172]

for any memoryless BRM, if c exceeds a critical threshold, achieving ρ -decentralization becomes impossible. This motivates our shift to retentive BRMs (e.g., Fruitchains [162]), where rewards for each block account for past ledger entries. While Fruitchains reduce a miner’s risk and improve solo-mining utility, they still leave pool-joining more attractive. Drawing on these insights, we introduce a new retentive mechanism, **decentBRM**, in which each block’s reward is shared among all miners who have previously mined a block. We show that under **decentBRM**, solo mining constitutes an equilibrium strategy (Lemma 4.2), thus ensuring ρ -decentralization.

In brief, our main contributions are:

- We formulate a two-player game $\mathcal{G}(\Gamma)$ (Section 4.4.1), where p_1 represents a joining miner and p_2 models the remainder of the system (including pools and solo miners).
- We introduce the concept of ρ -*Decentralization* in Section 4.4.3.
- For memoryless BRMs, we establish:
 1. If $c = 0$, the protocol is ρ -decentralized (Lemma 4.1).

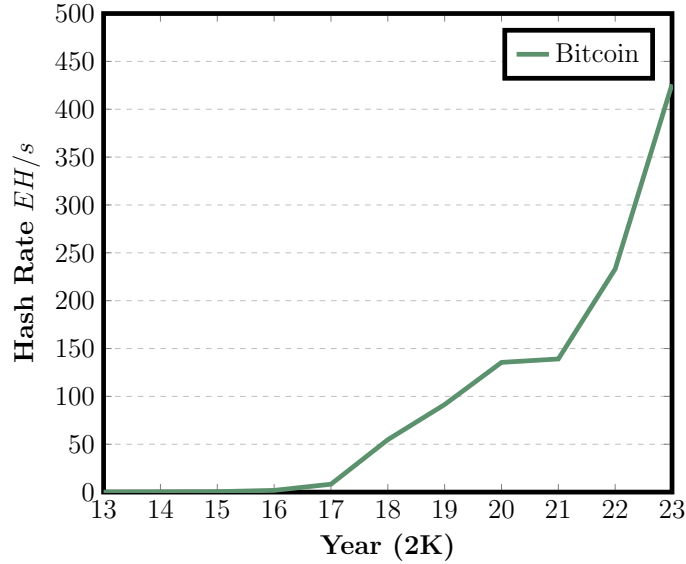


Figure 4.2: Bitcoin Hashrate Distribution

2. There exists a threshold $\underline{c}$ such that for $c \geq \underline{c}$, centralization is inevitable (Theorem 4.1).
- We prove that no memoryless BRM can make solo mining strictly preferable to pool participation (Theorem 4.2).
 - We design a novel retentive BRM, **decentBRM**, which uniquely incentivizes solo mining over pool-joining, thereby achieving ρ -decentralization and mitigating pool-driven centralization.

4.2 Related Work

Game-theoretic methods have been widely applied to blockchain security, covering topics such as consensus protocols [173], transaction-fee mechanisms [174, 175, 176], and miner strategies under low-variance schemes [154, 168].

Centralization in PoW and Mining Pools Centralization in PoW networks is a recognized security risk [177, 178, 179]. High reward variance drives miners to pool their resources [180], and pool reward-sharing schemes—PPS, Proportional, Geometric, PPLNS—offer varying trade-offs between expected payoff and risk [154]. Game-theoretic analyses of RSS (e.g., [154, 168]) identify variance-optimal schemes but do not address how BRMs themselves influence network decentralization. Our work fills this gap by evaluating BRMs’ role in discouraging pool formation. The impact of BRMs on blockchain security has been studied in PoW [179] and PoS [181] contexts; we extend this to decentralization in PoW networks [166, 178].

Game-Theoretic Miner Models Prior miner-behavior models include mean-field games [182], evolutionary games [183], and dynamic pool-selection models [184], but they typically consider only expected rewards and omit risk or switching costs. Other works explore pool choices under specific RSS conditions [185, 154, 168, 170, 181], and empirical portfolio strategies for risk-averse miners [169]. In contrast, our framework integrates expected reward, higher-order risk measures, and transition costs across general BRMs.

Protocols for Decentralized Mining Efforts to reduce mining variance include Bobtail [161], Fruitchains [162], Hybrid PoW [158], SmartPool [160], Proof-of-Mining [156], Sign-to-Mine [159], and HaPPY-Mine [155]. While these protocols limit pool-size incentives, they do not eliminate the benefits of pooling entirely [163]. Our `decentBRM` mechanism achieves a stronger guarantee by making solo mining the equilibrium choice.

4.3 Preliminaries

In this section, we explain (i) blockchain preliminaries, (ii) block reward mechanisms, and (iii) the functioning of mining pools.

4.3.1 Blockchain Preliminaries

4.3.1.1 Proof-of-Work (PoW)

A *blockchain* is a ledger maintained and updated by a set of *miners* which we also refer to as *players*. The blockchain comprises an ordered chain of blocks, where a block at height k is B_k and is preceded by a “parent-block” B_{k-1} for all $k > 0$ ¹. A block B_k consists of block-header and transaction data. The header contains information (1) hash (cryptographic digest) of the parent block B_{k-1} (Null genesis block), (2) height of the block k , (3) Merkle-root (cryptographic fingerprint) of the set of transactions included in the block, and (4) a random number n , called *Nonce*. Note that the block structure may vary among different PoW blockchains, but these four elements are invariably present in the block header and are sufficient for our understanding. The protocol decides on some *Target* T and considers a block *mined* if its header’s hash is lesser than the target T . Thus, each miner searches for a nonce till the block is mined by investing in computing power to compute hashes. The **hash rate** of a miner is how many queries it can make in a time interval. Any **round** k is defined as the duration from after block B_{k-1} is mined till when block B_k is mined. The entire protocol is parameterized by the security parameter λ which is also the size of the Hash function.

4.3.1.2 Block Reward Mechanism

At round k , a PoW blockchain protocol rewards a miner for its work if it successfully mines the block B_k . Typically, the rewards are of two types: (i) *block reward* and (ii) *transaction fees*. We abstract out the *block-reward mechanism* (BRM) of the PoW blockchain for a block B_k mined in round k based on the history $\mathcal{H}_k$ at round k as $\Gamma(B_k, \mathcal{H}_k)$. At round k , for most of the PoW blockchains, BRMs offer block rewards to the miner independent of the chain’s evolution till round k . We refer to such BRM as *memoryless BRM*. We formally define the memorylessness condition below.

¹ $k = 0$ is genesis block

Definition 4.1: Memoryless BRM

A BRM $\Gamma(\cdot, \cdot)$ has *memoryless* property if for any round k and any history $\mathcal{H}_k^1, \mathcal{H}_k^2$ we have

$$\Gamma(B_k, \mathcal{H}_k^1) = \Gamma(B_k, \mathcal{H}_k^2) \quad (4.1)$$

A trivial BRM which is memoryless is $\Gamma(B_k, \mathcal{H}_k) = 0$ for all $B_k, \mathcal{H}_k$. Another example of memoryless BRM is bitcoin's BRM ($\Gamma_{memoryless}$). $\Gamma_{memoryless}$ rewards only the miner who has mined the current block B_k and is therefore invariant of the previous blocks mined (history). Reward in a memoryless BRM might depend on the block's height. (In Bitcoin, the block reward halves after every 210K blocks are mined.) More sophisticated BRMs can leverage the miners' historical activity and pay more to the miners working harder. Examples of such retentive (or non-memoryless) BRMs are Fruitchains [162] and decentBRM (protocol proposed in Section 4.6).

In memoryless BRMs, miners face much uncertainty in mining the block due to the increased competition. Instead of solo mining, often multiple miners come together for mining and share the rewards amongst themselves; though expected rewards remain the same, the risk (variance in the rewards) decreases. For example, suppose a solo miner starts mining; there is a 10% chance of mining a block with a reward of 10 coins. If two friends with the same computing power decide to mine jointly and split the rewards equally, the expected reward is still one coin each. However, now, the chance of '0' reward is 81% as opposed to 90% in solo mining. Thus, PoW mining leads to *mining pools*.

4.3.2 Mining Pools

In PoW blockchains, although the expected reward for each miner is sufficiently adequate, the frequency at which each miner gets rewarded (through mining a block) is very low. Multiple miners work together to mine a block. All these miners together are called a *mining pool*. As the pool has a higher mining power than individual miners, a higher chance

of mining block in any round. The pool is managed by a pool manager and is constituted of multiple miners. We assume the PoW blockchain under analysis has $\mathcal{P} = \{1, 2, \dots, p\}$ mining pools. Following is the working of mining pool i .

- Pool manager constructs the skeleton of the block the pool will attempt to mine. If the block is mined, the reward is transferred to the pool (with public key pk_i).
- Each miner tries different nonces to solve the difficulty puzzle. They send the results of some (or all) queries they made, which serve as input to the [reward-sharing scheme](#) (RSS) opted by the pool for when a block is mined.

We now describe the message transfer within the mining pool.

4.3.2.1 Message Distribution

Each mining pool comprises different miners. Miners send messages to the mining pool i , which usually comprises of the miner identity (public key) s of the miner and uniformly samples binary string $n \in \{0, 1\}^\lambda$ (nonce for the block header) such that the hash of the block is $< T_i$ for some target decided by the pool. This target is easier than the PoW blockchain target (i.e. $T_i > T$). Each message belongs to the message set $\mathcal{M}$. We represent the set of messages obtained in a round k by pool i as $\mathcal{M}_k^i$. We represent the message as the tuple $m := (s, n)$.

RSS have been well studied in the literature [154, 186]. Our work abstracts out the expected reward for analysis and is RSS agnostic².

4.3.2.2 Reward Sharing Scheme [154]

Reward Sharing Scheme (RSS) is a function that takes in the set of messages received by a mining pool, along with the identity of a miner s which is part of the pool, and outputs what fraction of the reward should be given to the miner. The set of messages

²under certain conditions due to Claim [4.1](#)

received by a mining pool i in round k is $\mathcal{M}_i^k$. Then, for any miner s that is part of the pool, the RSS for pool i is a function $\psi_i(s, \mathcal{M}_i^k) \in [0, 1]$. The reward given to the player s is therefore, $\Gamma_k^s := \Gamma(B_k, \mathcal{H})\psi_i(s, \mathcal{M}_i^k)$. If we are analyzing reward for a single player, then $\psi : \{s, s'\} \times \mathcal{M} \rightarrow [0, 1]$ where s indicates the fraction of reward given to the miner, and s' represents the fraction given to rest of the miners. Details on Message distribution and reward sharing scheme can be found in [154].

As the introduction explains, we want to study centralization in mining pools. Few mining pools control most of the mining power, posing security concerns for the PoW blockchains. We assume all the miners are strategic players interested in maximizing their rewards and minimize the risk. Thus, we model problem of a player who want to join mining as a game in the next section.

4.4 The Game

We consider the blockchain system with p mining pools as $\mathbf{M} = (\Gamma, \psi)$ which consists of Γ – block reward mechanism and ψ_i – reward sharing scheme opted by the mining pool i . As there is a large number of miners already in the system, we model the dilemma of a new player about how to distribute its computing power as a two-player game $\mathcal{G}(\Gamma)$ for BRM Γ . Player p_1 is the miner joining the system and p_2 abstracts the remaining system (comprising of mining pools and solo miners) into a single player.

In this section, (i) we explain different classes of player 1 – discussing risk tolerance of p_1 , (ii) the strategy space of the players, (iii) the utility structure of p_1 that is joining the PoW system, and (iv) the desired properties for PoW system – fairness and decentralization.

4.4.1 Players

The Game consists of players $\mathcal{P} = \{p_1, p_2\}$. Player p_1 is characterized by $\theta_1 \in \mathbb{R}_{>0} \times \mathbb{N}$ represented as $\theta_1 = (M_1, \rho)$ where M_1 is the total hash rate of player p_1 and ρ is its risk-tolerance. The player p_2 has type $\theta_2 \in \mathbb{R}_{>0} \times \mathcal{A}$ where $\mathcal{A}$ is the set of possible reward

sharing schemes followed by different mining pools. We make the following assumption wherever necessary: owing to the enormous mining power of the PoW blockchain system compared to any single miner [172].

Assumption 4.1 (Approximate Mining Power). *For player p_1 characterized by $\theta_1 = (M_1, \rho)$ and p_2 by $\theta_2 = (M_2, A)$, the mining power of a single player is minimal compared to the rest of the system i.e. $M_1 \ll M_2$.*

4.4.1.1 Risk Tolerance

Let R_k be the random variable representing player p_1 's reward in round k . The primary reason for p_1 to join one or more mining pools is minimizing the risk R_k . Solo mining has a very high variance (the second-order moment of R_k) in the reward obtained. According to [171], ‘‘Pioneered by Nobel laureate Harry Markowitz in the 1950s, the mean-variance (MV) formulation is a fundamental theory for risk management in finance.’’. MV is the difference between the mean reward and its standard deviation – the risk. Instead of just modelling risk through second moment of the reward, we parameterize it by a tolerance of player p_1 using $\rho \in \mathbb{N}$. A ρ -risk tolerant player will want to minimize $(\mathbb{E}[R_k^\rho])^{1/\rho}$. A very high ρ is characteristic of risk-averse whereas $\rho = 1$ is a risk-neutral. We do not consider risk-loving players as they would prefer solo mining.

4.4.1.2 Strategy Space and Environment

Each player follows a strategy sampled from the set $\mathcal{S} = \Delta(p+1)$ which is a $(p+1)$ -simplex; $\bar{g}$ for player p_1 and $\bar{f}$ for player p_2 . $\bar{g} = (g_0, g_1, \dots, g_p)$ such that $\sum_{i=0}^p g_i = 1$. g_i indicates the fraction of mining power used to mine with mining pool i , and g_0 indicates the fraction used for solo mining. Similarly, $\bar{f}$ is the strategy for player p_2 . Therefore, the hash rate for pool i is given by $M_2 \cdot f_i$ at the time p_1 is joining. Thus, the total of $M_2 f_0 + M_1 g_0$ computing power – solo miners use hash rate, and for each mining pool i ,

the hash rate is $M_2 f_i + M_1 g_i$. As $M_2 \gg M_1$, we assume that the joining of a single player p_1 does not affect $\bar{f}$.

4.4.1.2.1 Discrete Strategy Space The strategy space described above captures that a player can reallocate some of its mining power across different mining pools. However, practically, there will be some limit on the minimum computing power it can assign to a particular pool. Hence, we discretize the strategy space based on this minimum possible change. We define a strategy space with minimum permissible change being α as α -*Discrete Strategy Space*, formally defined in Definition 4.2 below.

Definition 4.2: α -discrete Strategy Space

A strategy space S_α is α -discrete iff $\forall g^1, g^2 \in S_\alpha, \forall i \in \{0, 1, 2, \dots, k\} \exists l \in \mathbb{Z}_{\geq 0}$

$$|g_i^1 - g_i^2| = l \cdot \alpha$$

4.4.2 Utility of a Player

Having defined players and the strategy space, we now define the utility for player p_1 . Towards this, consider the random variable R_k representing the reward obtained by the player in round k . If $\Gamma(B_k, \mathcal{H}_k)$ is the (PoW protocol specific) BRM in a round k and $\psi_i : \{s_1, s_{-1}\} \times \mathcal{M} \rightarrow [0, 1]$ is the Reward Sharing Scheme (RSS) opted by pool i . Then, the reward in round k is given by random variable R_k (randomness is due to distribution $\mathcal{M}(\bar{g}, \bar{f})$) as:

$$R_k = \begin{cases} \Gamma(B_k, \mathcal{H}_k) & \text{with prob. } \frac{g_0 M_1}{M_1 + M_2} \\ \Gamma(B_k, \mathcal{H}_k) \psi_i(s_1, \mathcal{M}_{i,r}(\bar{g}, \bar{f})) & \text{with prob. } \frac{g_i M_1 + f_i M_2}{M_1 + M_2} \\ 0 & \text{with prob. } \frac{f_0 M_2}{M_1 + M_2} \end{cases} \quad (4.2)$$

4.4.2.1 Message Distribution

Miners who are part of some mining pool send messages to the pool during mining attempts. If the message was sent by player p_1 , then public key $s = s_1$ and $s = s_{-1}$ otherwise. These messages are random variables $m \in \{s_1, s_{-1}\} \times \{0, 1\}^\lambda$ induced by the hash rate distribution $\bar{g}, \bar{f}$ and is therefore represented as $\mathcal{M}(\bar{g}, \bar{f})$. The distribution is such that for any randomly sampled message $m \in \mathcal{M}_k^i(\bar{g}, \bar{f})$ in any arbitrary round k , we have:

$$m = \begin{cases} (s_1, n) & \text{with prob. } \frac{g_i M_1}{M_1 g_i + M_2 f_i} \\ (s_{-1}, n) & \text{with prob. } \frac{f_i M_2}{M_1 g_i + M_2 f_i} \end{cases}$$

Here, n is the solution to the puzzle (nonce), and λ is the size of the hash function output, also a security parameter of the protocol.

4.4.2.1.1 Pool Hopping / Switching Cost Player p_1 typically may hop between different pools (called pool-hopping [187]), to minimize risks. However, changing the pool incurs some costs, e.g., a drop in the effective hash rate. We model such pool hopping costs as *switching cost function* $D : \Delta(K + 1) \rightarrow \mathbb{R}_{\geq 0}$. D has the following properties:

P1. $D(\bar{g})$ monotonically increases with $|\{g_i | g_i \neq 0; i \in \{0, 1, 2, \dots, k\}\}|$.

P2. For $\bar{g}^1, \bar{g}^2 \in S_\alpha$, if conditions (i) $g_q^1 = g_q^2 \forall q \in [1, k] \setminus \{i, j\}$, (ii) $g_i^1 + g_j^1 = g_i^2 + g_j^2$, and (iii) $g_i^1 \cdot g_j^1 > g_i^2 \cdot g_j^2$ holds, then $D(\bar{g}^1) \leq D(\bar{g}^2)$

The property **P1** states if a player hops across more pools, the switching cost increases as it needs to switch more frequently. The property **P2** corresponds to higher switching cost (due to more frequent switching between mining pools) if mining power is distributed more equally between two (or more) pools. In addition to this, if the strategy space is α -discrete, then the switching cost changes by at least some minimum amount D_{min} between two strategies. We call this Marginal Switching Cost and is formally defined in Definition 4.3.

Definition 4.3: Marginal Switching Cost

For switching cost function $D(\cdot)$ and strategies $g^1, g^2 \in S_\alpha$ (when $D(\bar{g}^1) \neq D(\bar{g}^2)$) where S_α is an α -discrete Strategy space, the *Marginal cost* of switching $D_{min} > 0$ is

$$D_{min} := \min_{g^1, g^2 \in S; g^1 \neq g^2} |D(g^1) - D(g^2)|$$

In summary, player p_1 's utility constitutes reward (positively) and risk and switching costs (negatively). We weigh them with a, b, c , respectively. Thus, in a round k its utility is:

$$u_1(\bar{g}, \bar{f}, r; (\theta_1, \theta_2)) = a \cdot \mathbb{E}[R_k] - b \cdot (\mathbb{E}[R_k^\rho])^{1/\rho} - c \cdot D(\bar{g}) \quad (4.3)$$

Considering a discount factor $\delta \in [0, 1]$ the utility of p_1 across rounds (starting from round r_0) is given by

$$U_1(\bar{g}, \bar{f}; (\theta_1, \theta_2)) = \sum_{r=r_0}^{\infty} \delta^{r-r_0} u_1(\bar{g}, \bar{f}, r; (M_1, \rho, M_2)) \quad (4.4)$$

We show the equilibrium strategy for player p_1 as well as fairness and decentralization defined in the next subsection are agnostic to player p_2 's utility U_2 (see Remark 4.2). Thus, we do not need to define it and hence, we skip it.

4.4.3 Game Progression and Properties

The goal of this game is to model the event of player p_1 joining the system and following utility maximizing strategy $\bar{g}^* \in S_\alpha$.

4.4.3.1 Game

Our game progresses in two steps. First, player p_2 plays, disclosing its opted strategy $\bar{f}$ following which, player p_1 plays strategy $\bar{g}$ such that $U_1(\bar{g}, \bar{f}; (\theta_1, \theta_2))$ is maximized. Note

that p_1 observes $\bar{f}$ before playing $\bar{g}$. Thus, $\bar{g}$ can be a depends of $\bar{f}$. For notational ease, we slightly exploit notations and represent $\bar{g}_{\bar{f}}$ as just $\bar{g}$ unless necessary. Therefore optimal strategy $\bar{g}^*$ for a given strategy $\bar{f}$ is solution to the optimization problem for p_1 (after observing $\bar{f}$) given by:

$$\begin{aligned} \max_{\bar{g}} \quad & U_1(\bar{g}, \bar{f}; ((M_1, \rho), (M_2, A))) \\ \text{s.t.} \quad & g_i \geq 0 \forall i \in \{0, 1, \dots, p\} \\ & \sum_{i \in \{0, 1, \dots, p\}} g_i = 1 \end{aligned}$$

Definition 4.4: Equilibrium Strategy

For a BRM Γ , and game $\mathcal{G}(\Gamma)$, the strategy $\bar{g}^*$ is Equilibrium strategy for p_1 if $\forall \theta_1 \in \mathbb{R}_{>0} \times \mathbb{N}, \forall \theta_2 \in \mathbb{R} \times \mathcal{A}, \forall \bar{g}' \in S_\alpha$ and $\forall \bar{f}$ we have

$$U_1(\bar{g}^*, \bar{f}; (\theta_1, \theta_2)) \geq U_1(\bar{g}', \bar{f}; (\theta_1, \theta_2))$$

We call strategy $\bar{g}^*$ as Dominant Strategy for the given $\bar{f}$.

Based on the outcome of the game, we consider the following properties which are required in our analysis.

4.4.3.2 Fairness

The reward sharing scheme for a mining pool i is fair for player p_1 if its reward sharing scheme serves as a hash rate estimator. This notion of fairness is motivated by the definition of unbiased RSS in [154]. For completeness, we define fairness wrt. our setting in Definition 4.5.

Definition 4.5: Fairness [154]

If mining pool i (controlling f_i fraction of total mining power M_2) follows Reward Sharing Scheme $\psi_i : \{s_1, s_{-1}\} \times \mathcal{M}^* \rightarrow [0, 1]$, then given mining pool is fair for player p_1 following strategy $\bar{g}$ if

$$\mathbb{E}[\psi_i(1, \mathcal{M}_k^i)] = \frac{g_i M_1}{g_i M_1 + f_i M_2} \quad (4.5)$$

We show through the Claim 4.1 below that a mining pool should have a fair RSS to incentivize other miners to join the pool.

Claim 4.1. *If a mining pool i follows a RSS ψ_i that is not fair, then for any BRM Γ and corresponding game $\mathcal{G}(\Gamma)$, any strategy $\bar{g}^2$ with $g_0^2 = x$ and $g_i^2 = y$ is dominated by $\bar{g}^1$ with $g_0^1 = x + y$ and $g_i^1 = 0$ for p_1 .*

Proof. Consider an arbitrary mining pool i following RSS ψ_i . Given BRM is Γ and $\mathcal{G}(\Gamma)$ is the corresponding game. We consider two cases (i) $\mathbb{E}[\psi_i(s_1, \mathcal{M}_k^i)] < \frac{g_i M_1}{g_i M_1 + f_i M_2}$ and (ii) $\mathbb{E}[\psi_i(s_1, \mathcal{M}_k^i)] > \frac{g_i M_1}{g_i M_1 + f_i M_2}$.

Case 1 $\left(\mathbb{E}[\psi_i(s_1, \mathcal{M}_k^i)] < \frac{g_i M_1}{g_i M_1 + f_i M_2}\right)$: In this case, consider the expected reward for player p_1 from investing $g_i = x$ in pool i .

$$\begin{aligned} \mathbb{E}[R_k] &= \frac{\Gamma(B_k, \mathcal{H}_k) g_0 M_1}{M_1 + M_2} \\ &\quad + \sum_{j=1}^p \Gamma(B_k, \mathcal{H}_k) \psi_j(s_1, \mathcal{M}_k^j) \frac{f_j M_2 + g_j M_1}{M_1 + M_2} \\ &= \frac{R_{block} g_0 M_1}{M_1 + M_2} + \Gamma(B_k, \mathcal{H}_k) \psi_i(s_1, \mathcal{M}_k^i) \\ &\quad + \sum_{j=1, j \neq i}^p \Gamma(B_k, \mathcal{H}_k) \psi_j(s_1, \mathcal{M}_k^j) \frac{f_j M_2 + g_j M_1}{M_1 + M_2} \\ &< \frac{R_{block} (g_0 + g_i) M_1}{M_1 + M_2} \\ &\quad + \sum_{j=1, j \neq i}^p \Gamma(B_k, \mathcal{H}_k) \psi_j(s_1, \mathcal{M}_k^j) \frac{f_j M_2 + g_j M_1}{M_1 + M_2} \end{aligned}$$

In the utility equation for player p_1 in the game $\mathcal{G}(\Gamma)$, coefficient a for expected reward $\mathbb{E}[R_k]$ is such that the coefficients for risk and switching cost b and c are negligible wrt. a . Further, since the increase in $\mathbb{E}[R_k]$ is non-negligible by switching from strategy $\bar{g}^2 = (g_0, g_1, \dots, g_i, \dots, g_p)$ to $\bar{g}^1 = (g_0 + g_i, g_1, \dots, 0, \dots, g_p)$, therefore player p_1 always prefers $\bar{g}^1$ over $\bar{g}^2$. Hence pool i does not get any computing resource from any p_1 of arbitrary type $\theta_1 \in \mathbb{R}_{>0} \times \mathbb{N}$.

Case 2 $\left(\mathbb{E}[\psi_i(s_1, \mathcal{M}_k^i)] > \frac{g_i M_1}{g_i M_1 + f_i M_2} \right)$: This case is infeasible for the mining pool. This is because, if for any incoming miner p_1 we have $\psi_i > \frac{g_i M_1}{g_i M_1 + f_i M_2}$, then summing over all the miners that are part of the system, we get $\psi_i > 1$. This is because for any miner part of the mining pool i , we established from Case 1 that $\psi_i \geq \frac{g_i M_1}{g_i M_1 + f_i M_2}$. However, $\psi_i > 1$ means the mining pool is distributing more rewards than they obtain, which bankrupts the mining pool and therefore is not possible.

Having established both cases, we come to the conclusion that for any mining pool i , the RSS should be Fair according to the Definition 4.5 for the pool to survive and miners join the pool. $\square$

4.4.3.3 Decentralization

The mechanism (Γ, ψ) is comprised of block reward scheme Γ and reward sharing scheme ψ . We define decentralization of a mechanism as ρ -Decentralized when miners joining the system have risk tolerance ρ . We call a Mechanism ρ -*Decentralized* if the relative mining power of the highest mining pool does not change upon a strategic player joining the system.

Definition 4.6: ρ -Decentralized

Consider a mechanism (Γ, ψ) is ρ -Decentralized for game $\mathcal{G}(\Gamma)$ played between player p_1 and PoW blockchain system p_2 where:

- p_1 characterized by $\theta_1 = (M_1, \rho)$ follows equilibrium strategy $\bar{g}^*$ on observing $\bar{f}$.
- p_2 characterized by $\theta_2 = (M_2, A)$ follows any arbitrary strategy $\bar{f}$

That is, $\bar{g}^*$ satisfies

$$\max_{i \in \{1, 2, \dots, k\}} f_i \geq \max_{j \in \{1, 2, \dots, k\}} \frac{g_j^* M_1 + f_j M_2}{M_1 + M_2} \quad (4.6)$$

Remark 4.1. *If a mechanism is ρ -Decentralized then for every player joining the system, the value of centralization decreases (or remains the same). Therefore, the mechanism ensures that the system decreases (or at least preserves) centralization in the PoW system.*

The remark follows from the definition of Decentralization (Definition 4.6). Consider miners $m_1, m_2, m_3, \dots$ joining the system leads to $G_j = \max_{i \in \{1, 2, \dots, k\}} f_{i,j}$ where $f_{i,j}$ is the mining power of the pool i after miner m_j joins the system. Therefore, if a system is ρ -Decentralized, then after miners $m_1, m_2, m_3, \dots$ with risk tolerance ρ join the system, by Definition 4.6, we have $G_1 \geq G_2 \geq G_3 \geq \dots$. Therefore, centralization always decreases or remains the same.

Remark 4.2. *Fairness and Decentralization guarantees provided by mechanism (Γ, ψ) are agnostic to the utility function $U_2(\cdot)$ of p_2 .*

Remark 4.2 follows from the observation that fairness property does not require any constraints on utility and decentralization property should be satisfied for all strategies followed by p_2 .

In summary, the game used for analyzing centralization due to mining pools is defined as $\mathcal{G} := \langle \{p_1, p_2\}, \{\theta_1, \theta_2\}, S_\alpha, \Gamma, \mathcal{M}, (U_1, U_2) \rangle$. In the following section, we use this definition of $\mathcal{G}$ to analyse conditions that lead to centralization or decentralization of a PoW blockchain system with some BRM Γ .

4.5 Theoretical Analysis

This section provides analysis of (i) memoryless BRMs like Bitcoin [4] ($\Gamma_{memoryless}$), (ii) retentive (non-memoryless) BRMs such as Fruitchains [162] ($\Gamma_{fruitchains}$). In this analysis, we find conditions under which these mechanisms lead to centralization and decentralization of the system. We use bitcoin reward function for memoryless BRMs because they abstract out the

4.5.1 Memoryless BRMs ($\Gamma_{memoryless}$)

Bitcoin is a well-known PoW-based blockchain protocol, which uses memoryless BRM (Definition 4.1). Consider indicator function $\mathbb{I}$ for block B_k and player with public-key s as $\mathbb{I}(s, B_k) = 1$ if the block is mined by player s and 0 otherwise. $\Gamma_{memoryless}$ in round k for player (or mining pool) with public key s is

$$\Gamma_{memoryless}^{s_1}(B_k, \mathcal{H}_k) = \mathbb{I}(s_1, B_k) R_{block}$$

For notational ease, we write the BRM $\Gamma_{memoryless}^{s_1}(B_k, \mathcal{H}_k)$ as $\Gamma_{memoryless, k}^{s_1}$. If the block is mined by the player p_1 or one of the p mining pools, p_1 gets rewarded proportionately as given by Equation 4.2. The game using Bitcoin-like memoryless BRMs $\Gamma_{memoryless}$ is $\mathcal{G}(\Gamma_{memoryless}) := \langle \{p_1, p_2\}, \{\theta_1, \theta_2\}, S_\alpha, \Gamma_{memoryless}, \mathcal{M}, (U_1, U_2) \rangle$. Consider player p_1 has public-key s_1 and mining pool i has public key pk_i . From Equation 4.2, the reward for p_1

in $\mathcal{G}(\Gamma_{memoryless})$ (after Assumption 4.1) is

$$R_k^{memoryless} = \begin{cases} \Gamma_{memoryless,k}^{s_1} & \text{with prob. } \frac{g_0 M_1}{M_2} \\ \Gamma_{memoryless,k}^{pk_i} \psi_i(s_1, \mathcal{M}_{i,r}(\bar{g}, \bar{f})) & \text{with prob. } f_i \\ 0 & \text{with prob. } f_0 \end{cases}$$

Due to Claim 4.1, mining pools follow a Fair Reward Sharing Scheme. Therefore, for pool i

$$\psi_i(s_1, \mathcal{M}_{i,k}(\bar{g}, \bar{f})) = \frac{g_i M_1}{g_i M_1 + f_i M_2} \approx \frac{g_i M_1}{f_i M_2}$$

Therefore, the utility for p_1 from Equation 4.3 is given as

$$\begin{aligned} U_1(\bar{g}, \bar{f}; ((M_1, \rho), (M_2, A))) &\approx a \sum_{i=1}^p \frac{g_i M_1}{M_2} R_{block} \\ &\quad - b \cdot \left(R_{block}^\rho \cdot \sum_{i=1}^k \frac{g_i^\rho}{f_i^{\rho-1}} \right)^{1/\rho} \\ &\quad - c \cdot D(\bar{g}) \end{aligned}$$

We first show through Lemma 4.1 that a PoW system with incoming player p_1 characterized by $\theta_1 = (M_1, \rho = 2)$ (i.e. variance is considered as the risk) is ρ -Decentralized for memoryless BRMs.

Lemma 4.1. *A PoW blockchain with memoryless BRM $\Gamma_{memoryless}$ is ρ -Decentralized under $\mathcal{G}(\Gamma_{memoryless})$ for any $\rho > 1$ if $c = 0$.*

Proof. We consider a game $\mathcal{G}(\Gamma_{memoryless})$ and analyse the DSE for player p_1 . The utility for p_1 in any round r given by Equation 4.3. Since we study 2-decentralization, p_1 is of type $\theta_1 = (M_1, \rho)$ for any $M_1 \in \mathbb{R}_{>0}$. The utility for any round r is,

$$\begin{aligned} u_i(\bar{g}, \bar{f}; (\theta_1, \theta_2)) &\approx a \sum_{i=1}^p \frac{g_i M_1}{M_2} R_{block} + \frac{g_0 M_1}{M_2} R_{block} \\ &\quad + (R_{block})^\rho \sum_{i=1}^p \frac{g_i^\rho}{f_i^{\rho-1}} + D(\bar{g}) \end{aligned}$$

We have due to Assumption 4.1, the probability of a miner p_1 solo-mining $\frac{g_0 M_1}{M_2}$ is also very small compared to her mining as part of a pool. We first show that $\bar{g} = \bar{f}$ is the minima for risk $-\mathbb{E}[R_{block}^2]$. Consider the risk when $\bar{g} = \bar{f}$.

$$(\mathbb{E}[R_{block}^\rho])^{1/\rho} = R_{block} \left(\sum_{i=1}^p \frac{g_i^\rho}{f_i} \right)^{1/\rho} = R_{block} \left(\sum_{i=1}^p f_i \right)$$

Now, consider for any $\bar{g}' \neq \bar{f}$. We divide $g_i \in \bar{g}'$ into three parts $X = \{i | g_i > f_i, g_i \in \bar{g}'\}$, $Y = \{i | g_i < f_i, g_i \in \bar{g}'\}$, and $Z = \{i | g_i = f_i, g_i \in \bar{g}'\}$. The risk for $\bar{g}'$ is given as

$$\begin{aligned} (\mathbb{E}[R_{block}^\rho])^{1/\rho} &= R_{block} \left(\sum_{l \in X} \frac{g_l^\rho}{f_l^{\rho-1}} + \sum_{m \in Y} \frac{g_m^\rho}{f_m^{\rho-1}} + \sum_{n \in Z} \frac{g_n^\rho}{f_n^{\rho-1}} \right)^{1/\rho} \\ &= R_{block} \left(\sum_{l \in X} \frac{(f_l + \tau_l)^\rho}{f_l^{\rho-1}} + \sum_{m \in Y} \frac{(f_m - \tau_m)^\rho}{f_m^{\rho-1}} + \sum_{n \in Z} \frac{f_n^\rho}{f_n^{\rho-1}} \right)^{1/\rho} \\ &\geq R_{block} \left(\sum_{i=1}^p f_i + \rho \left(\sum_{l \in X} \tau_l - \sum_{m \in Y} \tau_m \right) \right)^{1/\rho} \\ &= R_{block}^\rho \left(\sum_{i=1}^p f_i \right)^{1/\rho} \end{aligned}$$

We use the lower bound $x^v - v \cdot x^{v-1}y \leq (x - y)^v$ and $\sum_{l \in X} \tau_l = \sum_{m \in Y} \tau_m$ for obtaining the above inequality. We have obtained that if risk is $r_1 = (\mathbb{E}[R_{block}^\rho])^{1/\rho}$ when $\bar{g} = \bar{f}$ and $r_2 = (\mathbb{E}[R_{block}^\rho])^{1/\rho}$ when $\bar{g} \neq \bar{f}$, then $r_1 \leq r_2$. Hence, strategy $\bar{g} = \bar{f}$ is risk-optimal for any ρ .

Since, (i) expected reward is the same for all strategies $\bar{g}$, (ii) risk is minimized for strategy $\bar{g} = \bar{f}$ and (iii) $c = 0$, the strategy $\bar{g} = \bar{f}$ gives the optimal utility for p_1 . $\square$

We also discuss a scenario where joining the largest mining pool is the best response for a player p_1 joining the mining pool. If any new player joins the largest pool, then the PoW blockchain system will become centralized as more new miners join the system. We show in Theorem 4.1 below that joining the largest mining pools is the equilibrium strategy for any player p_1 joining the mining pool under certain conditions. The proof follows because this strategy's utility is higher than any other strategy given p_2 follows some $\bar{f}$.

Theorem 4.1. A PoW blockchain system with memoryless BRM $\Gamma_{memoryless}$ and game $\mathcal{G}(\Gamma_{memoryless})$ for player p_1 characterized by $\theta_1 \in \mathbb{R}_{>0} \times \mathbb{N}$ has equilibrium strategy $\bar{g}^*$ where $g_i^* = 1$ for $i = \arg \max_{i \in \{1,2,\dots,k\}} f_i$ if $c \geq \underline{c} = \frac{b \cdot R_{block} \cdot M_1 \cdot p}{M_2 \cdot D_{min}}$.

Proof. To prove this theorem, we consider the utility from two strategies (1) $\bar{g}^1$ where $g_i^1 = 1$ for $i = \arg \max_{j \in [1,p]} f_j$ and 0 otherwise, (2) any other $\bar{g}^2 \in S_\alpha$ in α -Marginal Strategy space. Wlog. we take $f_1 \geq f_2 \geq \dots \geq f_p$. Consider the utility of player p_1 in Game $\mathcal{G}(\Gamma_{memoryless})$ following strategy $\bar{g}^1$ after Assumption 4.1 as

$$U_1 = u_1(\bar{g}^1, \bar{f}; (\theta_1, \theta_2)) \approx \frac{a R_{block} M_1}{M_2} - \left(\frac{b \cdot R_{block} M_1}{M_2 f_1^{(\rho-1)/\rho}} \right) - c \cdot D(\bar{g}^1)$$

Similarly, consider the utility for some strategy $\bar{g}^2 = (g_1, g_2, \dots, g_p)$ after Assumption 4.1 is,

$$U_2 = u_1(\bar{g}^2, \bar{f}; (\theta_1, \theta_2)) \approx \frac{a R_{block} M_1}{M_2} - c \cdot D(\bar{g}^2) - \frac{b \cdot R_{block} M_1}{M_2} \left(\sum_{j=1}^p \frac{g_j^\rho}{f_j^{\rho-1}} \right)^{1/\rho}$$

We now take the value $U_1 - U_2$ which gives us

$$\begin{aligned} U_1 - U_2 &= c \cdot (D(\bar{g}^2) - D(\bar{g}^1)) \\ &\quad - \frac{b \cdot R_{block} M_1}{M_2} \left(\frac{1}{f_1^{(\rho-1)/\rho}} - \left(\sum_{j=1}^p \frac{g_j^\rho}{f_j^{\rho-1}} \right)^{1/\rho} \right) \\ &\geq c \cdot (D(\bar{g}^2) - D(\bar{g}^1)) \\ &\quad - \frac{b \cdot R_{block} M_1}{M_2} \left(p^{(\rho-1)/\rho} - 1 \right) \\ &\geq c \cdot D_{min} - \frac{b \cdot R_{block} M_1 p^{(\rho-1)/\rho}}{M_2} \\ &\geq c \cdot D_{min} - \frac{b \cdot R_{block} M_1 p}{M_2} \geq 0 \end{aligned}$$

The first inequality comes from the lower bound on $\left(\sum_i \frac{g_i^\rho}{f_i^{\rho-1}} \right) = 1$ from proof of Theorem 4.1 and $\frac{1}{f_1} \leq p$ by simple algebra (note $f_1 \geq f_2 \geq \dots \geq f_p$). Second inequality

comes since $\bar{g}^1, \bar{g}^2 \in S_\alpha$ are two distinct points on α -Marginal Strategy Space, therefore $|D(\bar{g}^2) - D(\bar{g}^1)| \geq D_{min}$. However, by property **P1** of the function D , we have $D(\bar{g}^2) \geq D(\bar{g}^1)$. Therefore, $D(\bar{g}^2) - D(\bar{g}^1) \geq D_{min}$. The last inequality comes from $c \cdot M_2 \cdot D_{min} \geq b \cdot R_{block} \cdot M_1 p$.

Therefore, $U_1 \geq U_2$. This means strategy $\bar{g}^1$ is Equilibrium from Definition 4.4 and therefore the system becomes centralized as all the new miners are incentivized to join the largest mining pool. $\square$

For complete decentralization of the system, our goal is to ensure that mining pools are not formed. This can happen if, for any player p_1 joining the system, solo mining is incentivized over joining any subset of mining pools. For memoryless BRMs, we show the impossibility of constructing such BRMs that incentivize solo mining over pool mining for any value of $\rho > 1$ through Theorem 4.2.

Theorem 4.2. *It is impossible to construct a memoryless BRM Γ such that equilibrium strategy for p_1 characterized by $\theta_1 = (M_1, \rho)$ (for $\rho > 1$) in game $\mathcal{G}(\Gamma)$ is solo mining i.e. $\bar{g}^*$ such that $g_0^* = 1$.*

Proof. To prove this theorem, we consider payoff in any arbitrary round k where the block mined is B_k and history is $\mathcal{H}_k$. The memoryless BRM is $\Gamma_{memoryless}$ and game is $\mathcal{G}(\Gamma_{memoryless})$. Wlog. consider $f_1 \geq f_2 \geq \dots f_p$. To show that any memoryless BRM will lead to the formation of mining pools, we compare utility for player p_1 of any type $\theta_1 \in \mathbb{R}_{>0} \times \mathbb{N}$ for two strategies:

- $\bar{g}^{lm}$ – when p_1 dedicates mining power largest mining pool. Here $g_i^{lm} = 1$ $i = \arg \max_{j \in [1, k]} f_j$. Utility for this strategy is $U_{lm} = u_1(\bar{g}^{lm}, \bar{f}; (\theta_1, \theta_2))$.
- $\bar{g}^{sm}$ – when miner dedicates their entire mining power to solo mining. In this case, $g_0^{sm} = 1$. Utility for this strategy is $U_{sm} = u_1(\bar{g}^{sm}, \bar{f}; (\theta_1, \theta_2))$.

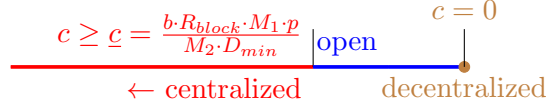


Figure 4.3: Bounds on switching cost parameter for $\Gamma_{memoryless}$

After Assumptions 4.1 utility U_{lm} and U_{sm} are given as

$$U_{lm} = \frac{a \cdot R_{block} \cdot M_1}{M_2} - \frac{b \cdot R_{block} \cdot M_1}{M_2} (f_1)^{1/\rho} - c \cdot D(\bar{g}^{lm})$$

$$U_{sm} = \frac{a \cdot R_{block} \cdot M_1}{M_2} - \frac{b \cdot R_{block} \cdot M_1^{1/\rho}}{M_2^{1/\rho}} - c \cdot D(\bar{g}^{sm})$$

We make the simple observation that (i) $D(\bar{g}^{sm}) \neq D(\bar{g}^{lm})$ and (ii) from Assumption $\frac{M_1}{M_2} \ll 1 \implies \left(\frac{M_1}{M_2}\right)^{1/\rho} > \frac{p \cdot M_1}{M_2} > \frac{f_1 \cdot M_1}{M_2}$. Therefore, $U_{lm} > U_{sm}$ for $\rho > 1$. With this, we conclude that $\bar{g}^{sm}$ (solo-mining strategy) is always dominated by $\bar{g} = \bar{f}$ which means mining pools will exist in Memoryless BRMs. $\square$

Need for Retentive BRMs. Lemma 4.1 and Theorem 4.1 help us find bound on c for which system is centralized, or ρ -Decentralized. We exhibit these bounds in Figure 4.3. We also make an interesting observation that $\underline{c}$ decreases as M_2 increases. Therefore, as the mining power of PoW blockchain system increases (which is already happening [164]), the system's new miners are more and more prone towards joining the largest mining pool. This fact, in addition to Theorem 4.2 makes us realize the need for a retentive BRM where the equilibrium strategy for miners is solo mining rather than joining or forming a mining pool.

4.5.2 Retentive BRMs ($\Gamma_{fruitchain}$)

In this section, we (i) provide construction of retentive BRM used in Fruitchain Blockchain [162], (ii) show that fruitchain reduces risk for solo miners (iii) show that fruitchains also fail to disincentivize the formation of mining pools.

4.5.2.1 Fruitchains [162]

It is a PoW-based blockchain that uses the concept of partial blocks to reduce risk in mining. Partial blocks have lesser target difficulty and are therefore easier to mine. A partial block refers to the most recent full block (blocks with higher target difficulty) B_k as their parent. When a partial block (referencing parent B_k) is mined, its status is *unconfirmed* (not a part of the ledger) and is represented as β_i^k (for the i^{th} unconfirmed partial block referencing B_k). These partial blocks are confirmed when a full block references them. That is, for some $r > k$, block B_r references β_i^k , then the partial block is considered confirmed (part of the blockchain) and represented as γ_i^k . A partial block gets rewarded when it's confirmed. The currently mined block can only confirm partial blocks that were mined within the z most recent full blocks mined. Partial blocks get rewarded $R_{partial}$ and full blocks get rewarded R_{full} . The parameters $R_{partial}, R_{full}, T_{full}$ (target difficulty of the full block), $T_{partial}$ (target difficulty of the partial block) and z are set by the system.

The current mined (full-block) is B_k then it's history is represented by

$$\mathcal{H} := (\{B_{k-1}, B_{k-2}, \dots, B_1\}, \{\bar{\gamma}^{k-2}, \bar{\gamma}^{k-3}, \dots, \bar{\gamma}^2\}, \{\bar{\beta}^{k-1}, \bar{\beta}^{k-2}, \dots, \bar{\beta}^{k-z}\})$$

4.5.2.2 Game Formulation

Consider the set of transactions confirmed in block B_k be contained in set $\mathcal{Z}_k$. Also, consider the indicator function $\mathbb{I}$ such that $\mathbb{I}(s_1, \gamma) = 1$ if some block γ is mined by player p_1 (having pub-key s_1) and 0 otherwise. Therefore, $\Gamma_{fruitychain}$ for block B_k and history $\mathcal{H}$ is represented for player p_1 as

$$\Gamma_{fruitychain}^{s_1}(B_k, \mathcal{H}_k) = R_{full}\mathbb{I}(s_1, B_k) + R_{partial} \sum_{\gamma \in \mathcal{Z}_k} \mathbb{I}(s_1, \gamma)$$

For ease of notation, we represent $\Gamma_{fruitychain,k}^{s_1}(B_k, \mathcal{H}_k) = \Gamma_{fruitychain,k}^{s_1}$. Similarly, $\mathbb{I}(pk_i, \gamma)$ is the indicator function representing if a block γ was mined by pool i with pub-key pk_i . The game using memoryless BRM similar to fruitchains is represented as $\mathcal{G}(\Gamma_{fruitychain,k})$. From

Equation 4.2, the reward for player p_1 in $\mathcal{G}(\Gamma_{fruitchain,k})$ in round k under Assumption 4.1 is represented by

$$R_k^{fruitchain} = \begin{cases} \Gamma_{fruitchain,k}^{s_1} & \text{with prob. } \frac{g_0 M_1}{M_2} \\ \Gamma_{fruitchain,k}^{pk_i} \psi_i(s_1, \mathcal{M}(\bar{g}, \bar{f})) & \text{with prob. } f_i \\ 0 & \text{otherwise} \end{cases}$$

4.5.2.3 Analysis

The advantage of using retentive BRMs is they reduce risk for any new-coming player in case of solo mining. Consider two BRMs $\Gamma_{memoryless}, \Gamma_{fruitchain}$ as previously described and $\mathcal{G}(\Gamma_{memoryless})$ and $\mathcal{G}(\Gamma_{fruitchain})$ be the games defined over the respective BRMs. Through Theorem 4.3 we show that $\Gamma_{fruitchain}$ provides lower risk to player p_1 than $\Gamma_{memoryless}$.

Theorem 4.3. *Retentive BRM $\Gamma_{fruitchain}$ provides lower risk than $\Gamma_{memoryless}$ towards solo mining strategy to player p_1 characterized by $\theta_1 = (M_1, \rho)$ for any $\rho > 1$.*

Proof. To prove this theorem, we consider the ratio of the risk (ρ^{th} moment) to the expected payoff (raised to ρ^{th} power) for any player p_1 that is mining in the PoW blockchain in two cases – $\Gamma_{memoryless}$ and $\Gamma_{fruitchain}$. The Target difficulty for full blocks in Fruitchains as well as Bitcoin is T_{full} and for partial blocks in Fruitchains is $T_{partial}$. Since it is easier to mine a partial block, we have $T_{full} < T_{partial}$. The number of partial blocks mined in expectation, for every full block in fruitchains is, therefore, $\frac{T_{partial}}{T_{full}}$ ³. In addition, reward given on mining a full block in fruitchains should be equal to the reward from memoryless BRMs. Therefore,

$$R_{block} = R_{full} + \frac{T_{partial}}{T_{full}} \cdot R_{partial} \quad (4.7)$$

We now calculate $(\mathbb{E}[R_k^\rho])^{1/\rho}$ for both Memoryless BRMs which we call $Risk_{memoryless}$ and fruitchains which we call $Risk_{fruitchain}$.

³The result follows from simple probability theory, but details can be found in [162]

Case 1 ($\Gamma_{memoryless}$): The game being played is $\mathcal{G}(\Gamma_{memoryless})$ and player p_1 has type $\theta_1 = (M_1, \rho)$ and player p_2 has type $\theta_2 = (M_2, A)$. This means, by the time system makes M_2 queries for mining, the player p_1 makes M_1 queries. The probability of a query made by p_1 in mining a block is $\gamma_1 = \frac{T_{full}}{2^\lambda}$. The reward obtained is R_{block} . Therefore,

$$Risk_{memoryless} = (E[R_k^\rho])^{1/\rho} = \gamma_1^{1/\rho} \cdot R_{block}$$

Case 2 ($\Gamma_{fruitchain}$): The game being played is $\mathcal{G}(\Gamma_{fruitchain})$ and the p_1 has type $\theta_1 = (M_1, \rho)$ and player p_2 has type $\theta_2 = (M_2, A)$. Probability of mining in one query a full block is $\gamma_1 = \frac{T_{full}}{2^\lambda}$ and for partial block is $\gamma_2 = \frac{T_{partial}}{2^\lambda}$. The reward obtained is R_{full} and $R_{partial}$ respectively. Therefore,

$$Risk_{fruitchain} = (E[R_k^\rho])^{1/\rho} = \left(\gamma_1 R_{full}^\rho + \gamma_2 R_{partial}^\rho \right)^{1/\rho}$$

Now we calculate the ratio of risks below

$$\begin{aligned} \frac{Risk_{fruitchain}}{Risk_{memoryless}} &= \left(\frac{\gamma_1 R_{full}^\rho + \gamma_2 R_{partial}^\rho}{R_{block}^\rho \gamma_1} \right)^{1/\rho} \\ &\leq \left(1 - \frac{R_{partial}(\gamma_1 T_{partial}^\rho - \gamma_2 T_{full}^\rho)}{R_{block}^\rho T_{full}^\rho \gamma_1} \right) \\ &< 1 \end{aligned}$$

The first inequality is obtained from $R_{full} = R_{block} - \frac{T_{partial}}{T_{full}} R_{partial}$ and second inequality is obtained because for any $\rho > 1$ we have $\gamma_1 T_{partial}^\rho - \gamma_2 T_{full}^\rho > 0$. Therefore, we show that $Risk_{fruitchains} < Risk_{memoryless}$ for solo miners. Since strategy $\bar{g}$ is the same in both cases, the switching cost is the same for both Memoryless BRMs and Fruitchains. In addition, the expected reward remains the same due to Equation 4.7. Hence, the utility for solo miners is higher in fruitchains than memoryless BRMs for the same expected block reward. $\square$

Although Fruitchains reduce the risk of solo mining, it also reduces the risk for mining pools. Therefore, centralization is an issue in Fruitchains, as demonstrated by Theorem 4.4. Note that centralization in fruitchains has been studied under a different model by [163] and the findings about fruitchains using our model align with their results.

Theorem 4.4. *Consider a player p_1 characterized by $\theta_1 = (M_1, \rho)$ joining the PoW blockchain system with BRM $\Gamma_{fruitchain}$. The player is always incentivized to join mining pools instead of solo mining.*

Proof. Consider a player p_1 characterized by $\theta_1 = (M_1, \rho)$ and p_2 characterized by (M_2, A) . The game being played is $\mathcal{G}(\Gamma_{fruitchain})$. Player p_1 plays after observing $\bar{f}$ (strategy chosen by p_2). Wlog. consider $f_1 \geq f_2 \geq \dots f_p$. Consider a round k where block B_k is mined. We need to show that joining mining pools dominates the strategy to perform solo mining. We represent the strategies:

- $\bar{g}^{sm} = (1, 0, 0, \dots, 0)$ as strategy for *solo-mining* with $g_0^{sm} = 1$. The utility corresponding to solo-mining is U_{sm} .
- $\bar{g}^{lm} = (0, 1, 0, 0, \dots, 0)$ as the strategy where the miner joins the largest mining pool. The corresponding utility is U_{lm} .

Our goal is to show that $U_{lm} > U_{sm}$ for all instances of $\mathcal{G}(\Gamma_{fruitchain})$ with different characterizations of p_1 and p_2 . We first make the observation that $D(\bar{g}^{lm}) = D(\bar{g}^{sm})$ because in either case, there is no two pools to switch between. Now, U_{sm} is given by

$$\begin{aligned} U_{sm} &= a \cdot \mathbb{E}[R_k] - b \cdot (\mathbb{E}[R_k^\rho])^{1/\rho} - c \cdot D(\bar{g}^{sm}) \\ &= \frac{a \cdot \Gamma_{fruitchain,k} M_1}{M_2} - b \cdot \Gamma_{fruitchain,k} \cdot \left(\frac{M_1}{M_2} \right)^{1/\rho} - c \cdot D(\bar{g}^{sm}) \end{aligned}$$

Similarly, we get U_{lm} as

$$\begin{aligned} U_{lm} &= a \cdot \mathbb{E}[R_k] - b \cdot (\mathbb{E}[R_k^\rho])^{1/\rho} - c \cdot D(\bar{g}^{lm}) \\ &= \frac{a \cdot \Gamma_{fruitchain,k} M_1}{M_2} - b \cdot \Gamma_{fruitchain,k} \cdot \frac{M_1}{M_2} \frac{1}{f_1^{(\rho-1)/\rho}} - c \cdot D(\bar{g}^{lm}) \\ &\leq \frac{a \cdot \Gamma_{fruitchain,k} M_1}{M_2} - b \cdot \Gamma_{fruitchain,k} \cdot \frac{M_1}{M_2} - c \cdot D(\bar{g}^{lm}) \end{aligned}$$

Therefore, taking $U_{lm} - U_{sm}$ and since $D(\bar{g}^{sm}) = D(\bar{g}^{lm})$, we get

$$U_{lm} - U_{sm} = b \Gamma_{fruitchain,k} \left(\left(\frac{M_2}{M_1} \right)^{1/\rho} - \frac{M_1}{M_2} \right) > 0$$

Inequality follows since for $M_1 < M_2$ (from Assumption 4.1) we have $\frac{M_1}{M_2} < \left(\frac{M_1}{M_2}\right)^{1/\rho}$ for any $\rho > 1$. Thus, joining the largest pool *strictly dominates* solo mining for any instance of $\mathcal{G}(\Gamma_{frutchains})$. $\square$

While frutchains fail to achieve decentralization in the sense that solo mining is an equilibrium strategy for any new player joining the system, $\Gamma_{frutchain}$ gives us an important insight. It reduces the gap between the optimal strategy $\vec{g}^*$ and solo mining strategy $(1, 0, 0, \dots, 0)$. In the following section, we present a novel retentive BRM **decentBRM**. Under this BRM, solo mining becomes DSE after some number of rounds for any player p_1 .

4.6 decentBRM : Optimal BRM

In this section, we (i) propose **decentBRM**, a risk-optimal BRM, and (ii) show that under the game $\mathcal{G}(\Gamma_{\text{decentBRM}})$, solo mining is the equilibrium strategy, which means using $\Gamma_{\text{decentBRM}}$ leads to a decentralized PoW blockchain.

4.6.1 decentBRM

The proposed BRM distributes rewards obtained from a block equally among miners who are part of the PoW blockchain. The reward from block B_k is distributed among miners of blocks $B_1, B_2, \dots, B_k$ equally. This allows a player to obtain a reward proportional to its mining power in each round. Since mining is a random process, for any player joining at round r_0 , we consider the expected number of blocks mined by the player to be proportional to its mining power after $r_0 + T$ with very high probability.

The implementation of this protocol can be through a special transaction that inputs two (1) block B_k , (2) B_q (for $q < k$) that was mined by public-key pk_q and (3) signature from pk_q to transfer $R_{block} \frac{B_k}{k}$ to the output address. This implementation is feasible and can be carried out through a *fork* in most of the existing PoW-based blockchains.

Next, we model this BRM under $\mathcal{G}$ and show that after T rounds from joining, p_1 performing solo mining is an equilibrium strategy.

4.6.2 Game Formulation

Consider a player p_1 joining the PoW blockchain with the BRM $\Gamma_{\text{decentBRM}}$. In round k where the block mined is B_k and the history $\mathcal{H}_k = \{B_{k-1}, B_{k-2}, \dots, B_1\}$. The indicator function for block B is $\mathbb{I}(s, B) = 1$ if B is mined by a player with pub-key s and 0 otherwise. The reward R_k is given by

$$\Gamma_{\text{decentBRM}}^{s_1}(B_k, \mathcal{H}_k) = \sum_{i=1}^k \frac{R_{\text{block}}}{k} \mathbb{I}(s_1, B_i)$$

For ease of notation we represent $\Gamma_{\text{decentBRM}}^{s_1}(B_k, \mathcal{H}_k) = \Gamma_{\text{decentBRM},k}^{s_1}$. The game is represented as $\mathcal{G}(\Gamma_{\text{decentBRM}})$ where the reward obtained by player p_1 characterized by (M_1, ρ) in round k is given by R_k under Assumption 4.1. The public key for player p_1 is s_1 and for pool i is pk_i .

$$R_k^{\text{decentBRM}} = \begin{cases} \Gamma_{\text{decentBRM},k}^{s_1} & \text{with prob. } \frac{g_0 M_1}{M_2} \\ \Gamma_{\text{decentBRM},k}^{pk_i} \psi_i(s_1, \mathcal{M}(\bar{g}, \bar{f})) & \text{with prob. } f_i \\ 0 & \text{otherwise} \end{cases}$$

4.6.3 Analysis

We now show that **decentBRM** leads to a decentralized PoW blockchain system. Towards this, we show using Lemma 4.2 that for any player p_1 with type $\theta_1 = (M_1, \rho)$, solo mining $\bar{g}^{sm} = (1, 0, 0, \dots, 0)$ is equilibrium strategy. The lemma follows by showing that after T rounds, the expected utility (expectation over the randomness of the mining protocol) is higher for solo mining than any other mining strategy with a very high probability.

Lemma 4.2. *In the game $\mathcal{G}(\Gamma_{\text{decentBRM}})$, following solo mining strategy $\bar{g}^{sm}$ is a equilibrium strategy after T rounds from joining for player p_1 characterized by $\theta_1 = (M_1, \rho)$.*

Proof. Consider player p_1 joining the protocol in round r_0 . After T rounds, consider any round $k > r_0 + T$. The block proposed in this round is B_k and $\mathcal{H}_k$ be the history. We consider two strategies:

- $\bar{g}^{sm}$ – *solo mining* where $g_0^{sm} = 1$. The utility corresponding to this strategy is U_{sm} .
- $\bar{g}^{mp}$ – p_1 invests in *mining pools* according to $g^{mp} \in S_\alpha$. The utility corresponding to this strategy is U_{mp} .

The reward from the block B_k for p_1 playing the solo mining strategy is given by the random variable R_k^{sm} . The probability that a block was mined by player p_1 under Assumption 4.1 is $\frac{M_1}{M_2}$.

$$\mathbb{E}[R_k^{sm}] = \sum_{i=1}^k \frac{M_1}{M_2} \cdot \frac{R_{block}}{k} = \frac{R_{block} \cdot M_1}{M_2}$$

The risk is given as $(\mathbb{E}[(R_k^{sm})^\rho])^{1/\rho}$. We calculate this risk as

$$(\mathbb{E}[(R_k^{sm})^\rho])^{1/\rho} = \frac{R_{block} \cdot M_1}{M_2} \left(\frac{M_1}{M_2} \right)^{1/\rho}$$

Similarly, consider random variable for strategy $\bar{g}^{mp}$ is R_k^{mp} .

$$\mathbb{E}[R_k^{mp}] = \sum_{i=1}^k \sum_{j=1}^p f_j \frac{R_{block}}{k} \frac{g_j M_1}{f_j M_2} = \sum_{j=1}^p \frac{R_{block} g_j M_1}{M_2} = \frac{R_{block} \cdot M_1}{M_2}$$

The risk is given as $(\mathbb{E}[(R_k^{mp})^\rho])^{1/\rho}$. We calculate the risk as

$$\begin{aligned} (\mathbb{E}[(R_k^{mp})^\rho])^{1/\rho} &= \left(\sum_{i=1}^k \sum_{j=1}^p f_j \left(\frac{R_{block}}{k} \frac{g_j M_1}{f_j M_2} \right)^\rho \right)^{1/\rho} \\ &= \frac{R_{block} M_1}{M_2} \left(\sum_{j=1}^p f_j (g_j)^\rho \right)^{1/\rho} \end{aligned}$$

Difference in utility $U_{sm} - U_{mp}$ is given by

$$\begin{aligned} U_{sm} - U_{mp} = & a(\mathbb{E}[R_k^{sm}] - \mathbb{E}[R_k^{mp}]) \\ & - b((\mathbb{E}[(R_k^{sm})^\rho])^{1/\rho} - (\mathbb{E}[(R_k^{mp})^\rho])^{1/\rho}) \\ & - c \cdot (D(\bar{g}^{sm}) - D(\bar{g}^{mp})) \end{aligned}$$

We know $D(\bar{g}^{sm}) - D(\bar{g}^{mp}) \leq 0$ because solo mining has no switching cost. In addition, the expected payoff is the same in both cases. Therefore,

$$U_{sm} - U_{mp} > \frac{R_{block} \cdot M_1 \cdot b}{M_2} \left(\left(\sum_{j=1}^p f_j (g_j)^\rho \right)^{1/\rho} - \left(\frac{M_1}{M_2} \right)^{1/\rho} \right) \gtrapprox 0$$

The last inequality is obtained because from Assumption 4.1 $\frac{M_1}{M_2}$ is very small. Therefore, we obtain $U_{sm} \gtrapprox U_{mp}$ which means solo mining is more profitable (or in worse cases, negligibly less profitable) than mining pools when BRM is $\Gamma_{\text{decentBRM}}$. $\square$

Since following $\bar{g}^{sm}$ is DSE, then the protocol is strongly ρ -Decentralized for any ρ , in Theorem 4.5.

Theorem 4.5. *A PoW blockchain using $\Gamma_{\text{decentBRM}}$ block reward mechanism is ρ -Decentralized for any $\rho > 1$.*

Proof. From Lemma 4.2, we have $\Gamma_{\text{decentBRM}}$ is such that solo mining is a dominant strategy. After player p_1 joins the system:

$$\max_{i \in [1, p]} f_i > \max_{j \in \{1, 2, \dots, p\}} \frac{g_j M_1 + f_j M_2}{M_1 + M_2}$$

Therefore, the protocol is ρ -Decentralized. $\square$

4.7 Conclusion

Conclusion. To conclude, we understood through the rational decision of a new miner (player p_1) joining the PoW blockchain system (player p_2) about the decentralization guarantees of the BRM offered by the PoW blockchain under a more general utility model.

We show that while bitcoin-like memoryless BRMs do not ensure decentralized BRMs, retentive BRMs offer better performance (lower risk for solo miners). We then propose **decentBRM**, a retentive BRM with solo mining as an equilibrium strategy. Therefore, the resulting PoW system will also be ρ -Decentralized for any ρ .

Chapter 5

Conclusion and Future Work

In this thesis, we have investigated two fundamental challenges in modern distributed systems populated by incentive-driven participants: (i) secure agreement protocols against rational adversaries, and (ii) preserving true decentralization in Proof-of-Work blockchains. We have shown that by carefully incorporating accountability, penalties, and history-aware reward mechanisms, one can significantly strengthen both security and decentralization guarantees without sacrificing liveness or performance.

5.1 Summary of Contributions

- **Rational Consensus under Accountability (pRFT).** We introduced the *partially-synchronous Rational Fault-Tolerance* (pRFT) protocol, that achieves deterministic consensus under honest majority even when up to $t + k < n/2$ participants are adversarial and at least one of the adversarial parties is rational. Unlike prior works that require less than $n/3$ adversaries or rely on focal equilibria, pRFT uses *identification* and *accountability* directly into the protocol: any deviation produces an unforgeable proof-of-fraud, and the misbehaving parties automatically forfeit their collateral deposits. We proved that following protocol honestly is a *dominant strategy equilibrium* (DSE), raising the adversarial tolerance threshold to nearly $n/2$ while ensuring both safety and liveness.

- **Decentralization via Retentive Block-Reward Mechanisms (DecentBRM).**

We formalized the miner’s pool-joining decision as a two-player game and classified block-reward mechanisms (BRMs) into *memoryless* and *retentive* classes. We proved that no memoryless BRM—where each block’s subsidy is history-independent—can prevent the variance-reduction incentive for pool formation, and thus cannot guarantee true decentralization. We proposed a novel *retentive* mechanism, **decentBRM**, which shares each new block’s reward among all prior block producers. Under **decentBRM**, solo mining becomes strictly at least as profitable as pool-joining for any risk-averse miner, making decentralization an equilibrium outcome.

Effectiveness of Mechanism Design. Our work demonstrates that incentive design need not be an afterthought to protocol security or decentralization. By leveraging standard cryptographic tools (digital signatures, collateral deposits) and modest protocol extensions, one can both raise the bound on rational-adversary tolerance in consensus and eliminate the centralizing pressure of mining pools in PoW blockchains—without introducing heavy network or computational overhead.

Rational Protocol Design. Beyond the two specific cases studied, our approach highlights a general principle: *accountability plus history-aware incentives* can transform rational protocol design. We anticipate that similar constructions will benefit other domains, such as transaction-fee mechanisms, proof-of-stake protocols, or cross-chain interoperability, wherever rational participants currently undermine system goals.

5.2 Future Directions

Our work lays foundation for applying mechanism design to achieve critical cryptographic properties in presence of Rational Players. We demonstrate the property of *decentralization* and *security* can be achieved through improvements to incentive scheme. Such

changes in incentives, and use of mechanism design can circumvent other such challenges facing cryptographic protocols.

Immediate extension is to prove optimal bounds for secure agreement in the Rational Fault Tolerance (RFT) model. We also acknowledge that decentBRM is a proof by construction that retentiveBRMs can achieve true decentralisation. However, the current version of the protocol is very inefficient and can be improved in runtime and memory complexity. Since the adversaries are computationally bounded, another future direction is to study the Rational Cryptography under the lens of Computationally bounded Game Theory.

We analyzed centralization/decentralization guarantees for the given protocol are such that for all $\rho > 1$ and $c \geq \underline{c}$ the PoW blockchain with $\Gamma_{memoryless}$ is centralized, and $c = 0$ it is ρ -Decentralized. However, there is an open gap $0 < c \leq \underline{c}$ where we don't know the guarantees of the protocol. Closing this gap is left for future work. In addition, analysis of other BRMs such as [155, 160], which uses different mining functions or smart contract functionality of another (external to the analyzed PoW system) blockchain, is left for future work.

We hope these contributions spark further research at the intersection of cryptography, game theory, and distributed systems, ultimately enabling more robust, fair, and resilient decentralized networks.

Bibliography

- [1] Miguel Castro and Barbara Liskov. “Practical Byzantine Fault Tolerance”. In: *Proceedings of the Third Symposium on Operating Systems Design and Implementation*. OSDI '99. New Orleans, Louisiana, USA: USENIX Association, 1999, pp. 173–186. ISBN: 1880446391.
- [2] Ittai Abraham et al. “Synchronous Byzantine Agreement with Expected $O(1)$ Rounds, Expected Communication, and Optimal Resilience”. In: *Financial Cryptography and Data Security: 23rd International Conference, FC 2019, Frigate Bay, St. Kitts and Nevis, February 18–22, 2019, Revised Selected Papers*. St. Kitts, Saint Kitts and Nevis: Springer-Verlag, 2019, pp. 320–334. ISBN: 978-3-030-32100-0. DOI: [10.1007/978-3-030-32101-7_20](https://doi.org/10.1007/978-3-030-32101-7_20). URL: https://doi.org/10.1007/978-3-030-32101-7_20.
- [3] Everett Hildenbrandt et al. “KEVM: A Complete Formal Semantics of the Ethereum Virtual Machine”. In: *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*. 2018, pp. 204–217. DOI: [10.1109/CSF.2018.00022](https://doi.org/10.1109/CSF.2018.00022).
- [4] Satoshi Nakamoto. *Bitcoin : A Peer-to-Peer Electronic Cash System*. 2009.
- [5] Vitalik Buterin et al. “Ethereum white paper”. In: ().
- [6] Juan Garay, Aggelos Kiayias, and Nikos Leonardos. “The Bitcoin Backbone Protocol: Analysis and Applications”. In: *Advances in Cryptology - EUROCRYPT 2015*. Ed. by Elisabeth Oswald and Marc Fischlin. Berlin, Heidelberg: Springer Berlin Heidelberg, 2015, pp. 281–310. ISBN: 978-3-662-46803-6.

- [7] Arthur Gervais et al. “Is Bitcoin a Decentralized Currency?” In: *IEEE S&P* 12 (2014), pp. 54–60.
- [8] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung. “The Google File System”. In: *Proceedings of the 19th ACM Symposium on Operating Systems Principles*. Bolton Landing, NY, 2003, pp. 20–43.
- [9] Satoshi Nakamoto. “Bitcoin: A peer-to-peer electronic cash system”. In: (2008).
- [10] Shubhani Aggarwal and Neeraj Kumar. “Chapter Twenty - Attacks on blockchain Working model.” In: *The Blockchain Technology for Secure and Smart Applications across Industry Verticals*. Ed. by Shubhani Aggarwal, Neeraj Kumar, and Pethuru Raj. Vol. 121. Advances in Computers. Elsevier, 2021, pp. 399–410. DOI: <https://doi.org/10.1016/bs.adcom.2020.08.020>. URL: <https://www.sciencedirect.com/science/article/pii/S0065245820300759>.
- [11] Kartik Nayak et al. “Stubborn Mining: Generalizing Selfish Mining and Combining with an Eclipse Attack”. In: *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. 2016, pp. 305–320. DOI: [10.1109/EuroSP.2016.32](https://doi.org/10.1109/EuroSP.2016.32).
- [12] Tao Li et al. “Rational protocols and attacks in blockchain system”. In: *Security and Communication Networks* 2020.1 (2020), p. 8839047.
- [13] Alejandro Ranchal-Pedrosa and Vincent Gramoli. “TRAP: The bait of rational players to solve Byzantine consensus”. In: *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*. 2022, pp. 168–181.
- [14] S.M. Ikhtiar Alam. *What is SWIFT in International Banking?* Jan. 2022. DOI: [10.13140/RG.2.2.23667.60961](https://doi.org/10.13140/RG.2.2.23667.60961).
- [15] Anton Wahrstätter et al. “Blockchain Censorship”. In: *arXiv preprint arXiv:2305.18545* (2023). Accessed: 2025-04-19. URL: <https://arxiv.org/pdf/2305.18545.pdf>.

- [16] Ethan Heilman et al. “Eclipse Attacks on Bitcoin’s Peer-to-Peer Network”. In: *24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, 2015, pp. 129–144. URL: <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/heilman>.
- [17] Muoi Tran et al. “A Stealthier Partitioning Attack against Bitcoin Peer-to-Peer Network”. In: *41st IEEE Symposium on Security and Privacy (SP ’20)*. May 2020, pp. 894–909. URL: <https://ieeexplore.ieee.org/document/9152616>.
- [18] Srivatsan Sridhar et al. “Content Censorship in the InterPlanetary File System”. In: *arXiv preprint arXiv:2307.12212* (2023). Accessed: 2025-04-19. URL: <https://arxiv.org/pdf/2307.12212.pdf>.
- [19] Fredrik Winzer, Benjamin Herd, and Sebastian Faust. “Temporary Censorship Attacks in the Presence of Rational Miners”. In: *2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&P Workshops)*. 2019, pp. 357–366. URL: <https://doi.org/10.1109/EuroSPW.2019.00046>.
- [20] Ed Felten. “Fighting censorship attacks on smart contracts”. In: *Medium* (2018). Accessed: 2025-04-19. URL: <https://medium.com/offchainlabs/fighting-censorship-attacks-on-smart-contracts-c026a7c0ff02>.
- [21] Maria Apostolaki, Aviv Zohar, and Laurent Vanbever. “Hijacking Bitcoin: Routing Attacks on Cryptocurrencies”. In: *2017 IEEE Symposium on Security and Privacy (S&P)*. 2017, pp. 375–392. URL: <https://ieeexplore.ieee.org/document/7958588>.
- [22] Ruben Recabarren and Bogdan Carbunar. “Tithonus: A Bitcoin Based Censorship Resilient System”. In: *arXiv preprint arXiv:1810.00279* (2018). Accessed: 2025-04-19. URL: <https://arxiv.org/pdf/1810.00279.pdf>.
- [23] Marcus Brandenburger et al. “Rollback and Forking Detection for Trusted Execution Environments Using Lightweight Collective Memory”. In: *Proceedings of the*

- 2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). 2017, pp. 157–168. DOI: [10.1109/DSN.2017.45](https://doi.org/10.1109/DSN.2017.45).
- [24] Secretarium. *Preventing Forking, Cloning & Rollback Attacks*. Accessed: 2025-04-11. 2023. URL: <https://secretarium.com/articles/preventing-forking-cloning--rollback-attacks>.
 - [25] Karthikeyan Nagaraj. “Exploring a Lesser-Known Blockchain Vulnerability: Blockchain Forking Attacks”. In: *Medium (Infosec Matrix)* (May 2023). URL: <https://cyberwing.medium.com/exploring-a-lesser-known-blockchain-vulnerability-blockchain-forking-attacks-karthikeyan-nagaraj-7caf16999397>.
 - [26] ResearchGate. *Preparing phase in a forking attack*. ResearchGate Figure. 2020. URL: https://www.researchgate.net/figure/Preparing-phase-in-a-forking-attack_fig3_342017074.
 - [27] Ittay Eyal and Emin Gun Sirer. *Majority is not Enough: Bitcoin Mining is Vulnerable*. arXiv preprint arXiv:1311.0243. 2013. URL: <https://arxiv.org/abs/1311.0243>.
 - [28] Andrew Miller. *Feather-forks: enforcing a blacklist with sub-50% hash power*. Accessed: 2025-04-11. 2013. URL: <https://bitcointalk.org/index.php?topic=312668.0>.
 - [29] Neptune Mutual. *Ethereum Classic 51% Attacks*. 2019. URL: <https://neptunemutual.com/blog/ethereum-classic-51-attacks/>.
 - [30] Zack Voell. “Ethereum Classic Suffers Reorganization That Resembles 51% Attack Amid Miner Complications”. In: *CoinDesk* (Aug. 2020). URL: <https://www.coindesk.com/markets/2020/08/01/ethereum-classic-suffers-reorganization-that-resembles-51-attack-amid-miner-complications>.

- [31] Sebastian Sinclair. “Ethereum Classic Suffers Second 51% Attack in a Week”. In: *CoinDesk* (Aug. 2020). URL: <https://www.coindesk.com/tech/2020/08/06/ethereum-classic-suffers-second-51-attack-in-a-week>.
- [32] Annika Wilde et al. *Forking Attacks on SGX Applications are Real*. Slides, ACSAC 2023 LASER Workshop. 2023. URL: <https://www.acsac.org/2023/files/web/laser-wilde-207-forking-sgx.pdf>.
- [33] Samira Briongos et al. “No Forking Way: Detecting Cloning Attacks on Intel SGX Applications”. In: *Proceedings of the 39th Annual Computer Security Applications Conference (ACSAC)*. 2023, pp. 744–758. DOI: [10.1145/3627106.3627187](https://doi.org/10.1145/3627106.3627187).
- [34] Zeta Avarikioti et al. “Bribe & Fork: Cheap PCN Bribing Attacks via Forking Threat”. In: *6th Conference on Advances in Financial Technologies (AFT 2024)*. Vol. 316. Leibniz International Proceedings in Informatics (LIPIcs). 2024, 11:1–11:22. DOI: [10.4230/LIPIcs.AFT.2024.11](https://doi.org/10.4230/LIPIcs.AFT.2024.11).
- [35] Varul Srivastava and Sujit Gujar. “Towards rational consensus in honest majority”. In: *2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS)*. IEEE. 2024, pp. 1439–1441.
- [36] Starknet contributors. *What is decentralization in blockchain?* Accessed: 2025-04-11. 2025. URL: <https://www.starknet.io/glossary/what-is-decentralization-in-blockchain/>.
- [37] Blockapps. *The Pillars of Blockchain Security: Decentralization and Encryption*. Blockapps blog. 2023. URL: <https://blockapps.net/blog/the-pillars-of-blockchain-security-decentralization-and-encryption/>.
- [38] Yujin Kwon et al. “Impossibility of Full Decentralization in Permissionless Blockchains”. In: *arXiv preprint arXiv:1905.05158* (2019).

- [39] Alex Hern. “Bitcoin currency could have been destroyed by ‘51%’ attack”. In: *The Guardian* (June 2014). URL: <https://www.theguardian.com/technology/2014/jun/16/bitcoin-currency-destroyed-51-attack-ghash-io>.
- [40] The Hacker News. “Ethereum Classic Hit by Double-Spend Attack Worth \$1.1 Million”. In: *The Hacker News* (Jan. 2019). URL: <https://thehackernews.com/2019/01/ethereum-double-spend-attack.html>.
- [41] Meni Rosenfeld. *Analysis of Bitcoin Pooled Mining Reward Systems*. arXiv preprint arXiv:1112.4980. 2011. URL: <https://arxiv.org/abs/1112.4980>.
- [42] Chainalysis Team. *Crypto Mining Pools Overview: How They Work, Benefits, and Risks*. Chainalysis blog post. May 2024. URL: <https://www.chainalysis.com/blog/crypto-mining-pools/>.
- [43] Wikipedia contributors. *Mining pool — Wikipedia, The Free Encyclopedia*. Wikipedia. Accessed: 2025-04-11. 2025. URL: https://en.wikipedia.org/wiki/Mining_pool.
- [44] Forbes Digital Assets. *DEMAND Launches New Bitcoin Mining Pool That Helps Independent Miners*. Forbes. Mar. 2025. URL: <https://www.forbes.com/sites/digital-assets/2025/03/18/demand-launch-first-stratum-v2-bitcoin-mining-pool-following-vc-close/>.
- [45] Tim Roughgarden and Clara Shikhelman. “Ignore the Extra Zeroes: Variance-Optimal Mining Pools”. In: *Financial Cryptography and Data Security*. Ed. by Nikita Borisov and Claudia Diaz. 2021, pp. 233–249.
- [46] Varul Srivastava and Sujit Gujar. “Decent-BRM: Decentralization through Block Reward Mechanisms”. In: *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*. 2024, pp. 2483–2485.
- [47] LESLIE LAMPORT, ROBERT SHOSTAK, and MARSHALL PEASE. “The Byzantine Generals Problem”. In: *ACM Transactions on Programming Languages and Systems* 4.3 (1982), pp. 382–401.

- [48] Manuel Barbosa et al. “SoK: Computer-Aided Cryptography”. In: *IACR Cryptol. ePrint Arch.* (2019), p. 1393. URL: <https://eprint.iacr.org/2019/1393>.
- [49] Michael J. Fischer, Nancy A. Lynch, and Michael Merritt. “Easy impossibility proofs for distributed consensus problems”. In: *Proceedings of the Fourth Annual ACM Symposium on Principles of Distributed Computing*. PODC ’85. Minaki, Ontario, Canada: Association for Computing Machinery, 1985, pp. 59–70. ISBN: 0897911687. DOI: [10.1145/323596.323602](https://doi.org/10.1145/323596.323602). URL: <https://doi.org/10.1145/323596.323602>.
- [50] Leslie Lamport, Robert Shostak, and Marshall Pease. “The Byzantine Generals Problem”. In: *ACM Trans. Program. Lang. Syst.* 4.3 (July 1982), pp. 382–401. ISSN: 0164-0925. DOI: [10.1145/357172.357176](https://doi.org/10.1145/357172.357176). URL: <https://doi.org/10.1145/357172.357176>.
- [51] D. Dolev and H. R. Strong. “Authenticated Algorithms for Byzantine Agreement”. In: *SIAM J. Comput.* 12.4 (Nov. 1983), pp. 656–666. ISSN: 0097-5397. DOI: [10.1137/0212045](https://doi.org/10.1137/0212045). URL: <https://doi.org/10.1137/0212045>.
- [52] Miguel Castro and Barbara Liskov. “Practical Byzantine fault tolerance”. In: *Proceedings of the Third Symposium on Operating Systems Design and Implementation*. OSDI ’99. New Orleans, Louisiana, USA: USENIX Association, 1999, pp. 173–186. ISBN: 1880446391.
- [53] Maofan Yin et al. “HotStuff: BFT Consensus with Linearity and Responsiveness”. In: *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*. PODC ’19. Toronto ON, Canada: Association for Computing Machinery, 2019, pp. 347–356. ISBN: 9781450362177. DOI: [10.1145/3293611.3331591](https://doi.org/10.1145/3293611.3331591). URL: <https://doi.org/10.1145/3293611.3331591>.
- [54] Yossi Gilad et al. “Algorand: Scaling Byzantine Agreements for Cryptocurrencies”. In: *Proceedings of the 26th Symposium on Operating Systems Principles*. SOSP ’17. Shanghai, China: Association for Computing Machinery, 2017, pp. 51–68. ISBN:

9781450350853. DOI: [10.1145/3132747.3132757](https://doi.org/10.1145/3132747.3132757). URL: <https://doi.org/10.1145/3132747.3132757>.
- [55] Bernardo Machado David et al. “Ouroboros Praos: An Adaptively-Secure, Semi-synchronous Proof-of-Stake Blockchain”. In: *International Conference on the Theory and Application of Cryptographic Techniques*. 2018.
 - [56] Andrew Miller et al. “The Honey Badger of BFT Protocols”. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’16. Vienna, Austria: Association for Computing Machinery, 2016, pp. 31–42. ISBN: 9781450341394. DOI: [10.1145/2976749.2978399](https://doi.org/10.1145/2976749.2978399). URL: <https://doi.org/10.1145/2976749.2978399>.
 - [57] Yackolley Amoussou-Guenou et al. “Rational vs Byzantine Players in Consensus-Based Blockchains”. In: *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems*. AAMAS ’20. Auckland, New Zealand: International Foundation for Autonomous Agents and Multiagent Systems, 2020, pp. 43–51. ISBN: 9781450375184.
 - [58] Ittai Abraham et al. “Solida: A Blockchain Protocol Based on Reconfigurable Byzantine Consensus”. In: *OPODIS 2017*. Accessed: 2025-04-19. 2017, pp. 1–18. DOI: [10.1007/978-3-319-88383-0_1](https://doi.org/10.1007/978-3-319-88383-0_1).
 - [59] Ittay Eyal and Emin Gün Sirer. “Bitcoin-NG: A Scalable Blockchain Protocol”. In: *NSDI 2016*. Accessed: 2025-04-19. 2016, pp. 45–59. URL: <https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/eyal>.
 - [60] Amitanand S. Aiyer et al. “BAR Fault Tolerance for Cooperative Services”. In: *SIGOPS Oper. Syst. Rev.* 39.5 (Oct. 2005), pp. 45–58. ISSN: 0163-5980. DOI: [10.1145/1095809.1095816](https://doi.org/10.1145/1095809.1095816). URL: <https://doi.org/10.1145/1095809.1095816>.
 - [61] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. “Impossibility of distributed consensus with one faulty process”. In: *J. ACM* 32.2 (Apr. 1985), pp. 374–

382. ISSN: 0004-5411. DOI: [10.1145/3149.214121](https://doi.org/10.1145/3149.214121). URL: <https://doi.org/10.1145/3149.214121>.
- [62] Xiaoyu Ji, Junru Li, and Yifan Song. “Linear-Communication Asynchronous Complete Secret Sharing with Optimal Resilience”. In: *Cryptology ePrint Archive* (2024).
 - [63] Achour Mostéfaoui, Hamouma Moumen, and Michel Raynal. “Randomized k-set agreement in crash-prone and Byzantine asynchronous systems”. In: *Theoretical Computer Science* 709 (2018). Special Issue of ICDCN 2016 (Distributed Computing Track), pp. 80–97. ISSN: 0304-3975. DOI: <https://doi.org/10.1016/j.tcs.2017.03.018>. URL: <https://www.sciencedirect.com/science/article/pii/S0304397517302311>.
 - [64] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. “Consensus in the presence of partial synchrony”. In: *J. ACM* 35.2 (Apr. 1988), pp. 288–323. ISSN: 0004-5411. DOI: [10.1145/42282.42283](https://doi.org/10.1145/42282.42283). URL: <https://doi.org/10.1145/42282.42283>.
 - [65] A. Fox and E.A. Brewer. “Harvest, yield, and scalable tolerant systems”. In: *Proceedings of the Seventh Workshop on Hot Topics in Operating Systems*. 1999, pp. 174–178. DOI: [10.1109/HOTOS.1999.798396](https://doi.org/10.1109/HOTOS.1999.798396).
 - [66] Marcin Andrychowicz and Stefan Dziembowski. “PoW-Based Distributed Cryptography with No Trusted Setup”. In: *Advances in Cryptology – CRYPTO 2015*. Berlin, Heidelberg: Springer-Verlag, 2022, pp. 379–399. ISBN: 978-3-662-47999-5. DOI: [10.1007/978-3-662-48000-7_19](https://doi.org/10.1007/978-3-662-48000-7_19). URL: https://doi.org/10.1007/978-3-662-48000-7_19.
 - [67] Oded Goldreich. *Foundations of Cryptography, Volume 2*.
 - [68] Leslie Lamport. *Constructing Digital Signatures from a One-Way Function*. Tech. rep. SRI-CSL-98. Technical Report. SRI International, 1979.
 - [69] Wikipedia contributors. *Merkle Signature Scheme*. https://en.wikipedia.org/wiki/Merkle_signature_scheme. Accessed: 2025-04-18. 2007.

- [70] John Rompel. “One-Way Functions are Necessary and Sufficient for Secure Signatures”. In: *Proceedings of the 22nd ACM Symposium on Theory of Computing*. ACM, 1990, pp. 387–394.
- [71] Jonathan Katz and Chan Yun Ko. *On Constructing Universal One-Way Hash Functions from Arbitrary One-Way Functions*. Tech. rep. University of Maryland, 2005.
- [72] Jonathan Katz and Yehuda Lindell. *Lecture 38: Universal One-Way Hash Functions*. University of Maryland Crypto Lecture Notes. 2005.
- [73] Oded Goldreich. *Foundations of Cryptography: Volume 1, Basic Tools*. Cambridge University Press, 2001.
- [74] Wikipedia contributors. *One-way Function*. https://en.wikipedia.org/wiki/One-way_function. Accessed: 2025-04-18. 2025.
- [75] Red Hat. *Post-quantum Cryptography: Hash-based Signatures*. <https://www.redhat.com/en/blog/post-quantum-cryptography-hash-based-signatures>. Accessed: 2025-04-18. 2022.
- [76] *SHA-256 Definition*. CoinMarketCap. Accessed: 2025-04-19. URL: <https://coinmarketcap.com/academy/glossary/sha-256>.
- [77] *Hash Function — Fingerprints for Data*. LearnMeABitcoin. Accessed: 2025-04-19. URL: <https://learnmeabitcoin.com/technical/cryptography/hash-function/>.
- [78] *What is Keccak256?* Nervos Knowledge Base. Accessed: 2025-04-19. URL: https://www.nervos.org/knowledge-base/what_is_keccak256_%28explainCKBot%29.
- [79] Ankitacode11. *Ethereum Blockchain: Cryptographic Algorithms and Hash Functions*. Medium. Accessed: 2025-04-19. URL: <https://medium.com/@ankitacode11/ethereum-blockchain-cryptographic-algorithms-and-hash-functions-12de89a5730f>.

- [80] *What is RIPEMD160?* Nervos Knowledge Base. Accessed: 2025-04-19. URL: https://www.nervos.org/knowledge-base/what_is-RIPEMD160_%28explainCKBot%29.
- [81] *Why does Bitcoin use two hash functions to create an address?* Bitcoin Stack-Exchange. Accessed: 2025-04-19. URL: <https://bitcoin.stackexchange.com/questions/9202/why-does-bitcoin-use-two-hash-functions-sha-256-and-ripemd-160-to-create-an-ad>.
- [82] *BLAKE (hash function)*. Accessed: 2025-04-19. URL: https://en.wikipedia.org/wiki/BLAKE_%28hash_function%29.
- [83] *Cryptography on Polkadot*. Polkadot Wiki. Accessed: 2025-04-19. URL: <https://wiki.polkadot.network/docs/learn-cryptography>.
- [84] *SHA-3*. Accessed: 2025-04-19. URL: <https://en.wikipedia.org/wiki/SHA-3>.
- [85] *Elliptic Curve Digital Signature Algorithm*. Accessed: 2025-04-19. URL: https://en.wikipedia.org/wiki/Elliptic_Curve_Digital_Signature_Algorithm.
- [86] *SECP256k1 Curve*. Accessed: 2025-04-19. URL: <https://en.bitcoin.it/wiki/Secp256k1>.
- [87] *Edwards-curve Digital Signature Algorithm*. Accessed: 2025-04-19. URL: <https://en.wikipedia.org/wiki/EdDSA>.
- [88] IETF Crypto Forum Research Group. *RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA)*. Internet RFC. Accessed: 2025-04-19. URL: <https://datatracker.ietf.org/doc/html/rfc8032>.
- [89] *BIP-340: Schnorr Signatures for secp256k1*. Bitcoin Improvement Proposal. Accessed: 2025-04-19. URL: <https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki>.
- [90] *Schnorr Signatures*. Bitcoin Ops. Accessed: 2025-04-19. URL: <https://bitcoinops.org/en/topics/schnorr-signatures/>.

- [91] *BLS Digital Signature*. Accessed: 2025-04-19. URL: https://en.wikipedia.org/wiki/BLS_digital_signature.
- [92] *Upgrading Ethereum: BLS Signatures*. Eth2 Book. Accessed: 2025-04-19. URL: https://eth2book.info/latest/part2/building_blocks/signatures/.
- [93] National Institute of Standards and Technology. *Digital Signature Standard (DSS) (FIPS 186-4)*. Federal Information Processing Standards Publication. Accessed: 2025-04-19. July 2013. URL: <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.186-4.pdf>.
- [94] P. Berman, J.A. Garay, and K.J. Perry. “Towards optimal distributed consensus”. In: *30th Annual Symposium on Foundations of Computer Science*. 1989, pp. 410–415. DOI: [10.1109/SFCS.1989.63511](https://doi.org/10.1109/SFCS.1989.63511).
- [95] “Authenticated algorithms for Byzantine agreement”. In: *SIAM Journal on Computing* 12.4 (1983), pp. 656–666.
- [96] Ittai Abraham et al. “Byzantine agreement, broadcast and state machine replication with near-optimal good-case latency”. In: *arXiv preprint arXiv:2003.13155* (2020).
- [97] Ittai Abraham et al. “Good-case Latency of Byzantine Broadcast: a Complete Categorization”. In: *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*. PODC’21. Virtual Event, Italy: Association for Computing Machinery, 2021, pp. 331–341. ISBN: 9781450385480. DOI: [10.1145/3465084.3467899](https://doi.org/10.1145/3465084.3467899). URL: <https://doi.org/10.1145/3465084.3467899>.
- [98] Jonathan Katz and Chiu-Yuen Koo. “On expected constant-round protocols for Byzantine agreement”. In: *Annual International Cryptology Conference*. Springer. 2006, pp. 445–462.
- [99] Silvio Micali and Vinod Vaikuntanathan. “Optimal and Player-Replaceable Consensus with an Honest Majority”. In: *MIT CSAIL Technical Reports* (2017).

- [100] Pierre-Louis Aublin, Sonia Ben Mokhtar, and Vivien Quéma. “RBFT: Redundant Byzantine Fault Tolerance”. In: *2013 IEEE 33rd International Conference on Distributed Computing Systems*. 2013, pp. 297–306. DOI: [10.1109/ICDCS.2013.53](https://doi.org/10.1109/ICDCS.2013.53).
- [101] Ramakrishna Kotla et al. “Zyzyva: Speculative Byzantine Fault Tolerance”. In: *SIGOPS Oper. Syst. Rev.* 41.6 (Oct. 2007), pp. 45–58. ISSN: 0163-5980. DOI: [10.1145/1323293.1294267](https://doi.org/10.1145/1323293.1294267). URL: <https://doi.org/10.1145/1323293.1294267>.
- [102] Dahlia Malkhi, Kartik Nayak, and Ling Ren. “Flexible Byzantine Fault Tolerance”. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’19. London, United Kingdom: Association for Computing Machinery, 2019, pp. 1041–1053. ISBN: 9781450367479. DOI: [10.1145/3319535.3354225](https://doi.org/10.1145/3319535.3354225). URL: <https://doi.org/10.1145/3319535.3354225>.
- [103] James Aspnes. “Lower bounds for distributed coin-flipping and randomized consensus”. In: *J. ACM* 45.3 (May 1998), pp. 415–450. ISSN: 0004-5411. DOI: [10.1145/278298.278304](https://doi.org/10.1145/278298.278304). URL: <https://doi.org/10.1145/278298.278304>.
- [104] Michael Ben-Or. “Another advantage of free choice (Extended Abstract): Completely asynchronous agreement protocols”. In: *Proceedings of the Second Annual ACM Symposium on Principles of Distributed Computing*. PODC ’83. Montreal, Quebec, Canada: Association for Computing Machinery, 1983, pp. 27–30. ISBN: 0897911105. DOI: [10.1145/800221.806707](https://doi.org/10.1145/800221.806707). URL: <https://doi.org/10.1145/800221.806707>.
- [105] Ran Canetti and Tal Rabin. “Fast asynchronous Byzantine agreement with optimal resilience”. In: *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing*. STOC ’93. San Diego, California, USA: Association for Computing Machinery, 1993, pp. 42–51. ISBN: 0897915917. DOI: [10.1145/167088.167105](https://doi.org/10.1145/167088.167105). URL: <https://doi.org/10.1145/167088.167105>.

- [106] Luciano Freitas, Petr Kuznetsov, and Andrei Tonkikh. “Distributed randomness from approximate agreement”. In: *arXiv preprint arXiv:2205.11878* (2022).
- [107] Cheng Wang. “Asynchronous Byzantine agreement with optimal resilience and linear complexity”. In: *arXiv preprint arXiv:1507.06165* (2015).
- [108] Christian Cachin, Klaus Kursawe, and Victor Shoup. “Random Oracles in Constantipole: Practical Asynchronous Byzantine Agreement Using Cryptography”. In: *Proceedings of the Nineteenth Annual ACM Symposium on Principles of Distributed Computing (PODC)*. PODC '00. Portland, Oregon, USA: Association for Computing Machinery, 2000, pp. 123–132. ISBN: 1581131836. DOI: [10.1145/343477.343531](https://doi.org/10.1145/343477.343531). URL: <https://doi.org/10.1145/343477.343531>.
- [109] Christian Cachin et al. *Secure and Efficient Asynchronous Broadcast Protocols*. Apr. 2001. DOI: [10.1007/3-540-44647-8_31](https://doi.org/10.1007/3-540-44647-8_31).
- [110] Yossi Gilad et al. “Algorand: Scaling byzantine agreements for cryptocurrencies”. In: *Proceedings of the 26th symposium on operating systems principles*. 2017, pp. 51–68.
- [111] Christian Badertscher et al. “Bitcoin as a transaction ledger: A composable treatment”. In: *Advances in Cryptology–CRYPTO 2017: 37th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 20–24, 2017, Proceedings, Part I 37*. Springer. 2017, pp. 324–356.
- [112] Yonatan Sompolinsky, Yoad Lewenberg, and Aviv Zohar. *Spectre: Serialization of proof-of-work events: Confirming transactions via recursive elections*. 2016.
- [113] Yonatan Sompolinsky, Shai Wyborski, and Aviv Zohar. “PHANTOM GHOSTDAG: a scalable generalization of Nakamoto consensus: September 2, 2021”. In: *Proceedings of the 3rd ACM Conference on Advances in Financial Technologies*. AFT '21. Arlington, Virginia: Association for Computing Machinery, 2021, pp. 57–70. ISBN:

9781450390828. DOI: [10.1145/3479722.3480990](https://doi.org/10.1145/3479722.3480990). URL: <https://doi.org/10.1145/3479722.3480990>.
- [114] Kostis Karantias, Aggelos Kiayias, and Dionysis Zindros. “Proof-of-Burn”. In: *International Conference on Financial Cryptography and Data Security*. 2020, pp. 523–540.
 - [115] Juan Garay, Aggelos Kiayias, and Nikos Leonardos. “The Bitcoin Backbone Protocol: Analysis and Applications”. In: *J. ACM* 71.4 (Aug. 2024). ISSN: 0004-5411. DOI: [10.1145/3653445](https://doi.org/10.1145/3653445). URL: <https://doi.org/10.1145/3653445>.
 - [116] Yujin Kwon et al. “Impossibility of Full Decentralization in Permissionless Blockchains”. In: *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*. AFT ’19. Zurich, Switzerland: Association for Computing Machinery, 2019, pp. 110–123. ISBN: 9781450367325. DOI: [10.1145/3318041.3355463](https://doi.org/10.1145/3318041.3355463). URL: <https://doi.org/10.1145/3318041.3355463>.
 - [117] Heinrich von Stackelberg. *Market Structure and Equilibrium*. Springer, 1934.
 - [118] Dmytro Korzhyk, Vincent Conitzer, and Tuomas Sandholm. “Stackelberg Security Games: A Framework for Resource Allocation in Adversarial Environments”. In: *Artificial Intelligence* 172.14 (2008), pp. 1634–1660.
 - [119] Xiaoyu Chen et al. “A Survey of Computational Approaches to Stackelberg Security Games”. In: *IEEE Transactions on Systems, Man, and Cybernetics* (2019).
 - [120] Martin J. Osborne and Ariel Rubinstein. *A Course in Game Theory*. MIT Press, 1994.
 - [121] John Nash. “Equilibrium points in n-person games”. In: *Proceedings of the National Academy of Sciences* 36.1 (1950), pp. 48–49.
 - [122] John C. Harsanyi. “Games with Incomplete Information Played by ”Bayesian” Players, I-III”. In: *Management Science* 14 (1967), pp. 159–182.

- [123] Jingjing Zhang et al. “dBFT: A Byzantine Fault Tolerant Protocol with Graceful Performance Degradation”. In: *Proceedings of the 38th Symposium on Reliable Distributed Systems (SRDS)*. 2019.
- [124] Qin Wang et al. *Security Analysis of the dBFT Protocol of NEO*. Tech. rep. Monsoon Honours Project, Machine Learning Lab, IIT Bombay, 2021.
- [125] Vivek Bagaria et al. “Prism: Deconstructing the Blockchain to Approach Physical Limits”. In: *CoRR* abs/1904.00113 (2019).
- [126] Haodong Yu et al. “OHIE: Blockchain Scaling Made Simple”. In: *Proceedings of the IEEE Symposium on Security and Privacy*. 2020, pp. 1595–1610.
- [127] Lukas Aumayr et al. “Bitcoin-Compatible Virtual Channels”. In: *Proceedings of the IEEE Symposium on Security and Privacy*. 2021, pp. 2045–2062.
- [128] Sri Aravinda Krishnan Thyagarajan and Giulio Malavolta. “Lockable Signatures for Blockchains”. In: *Proceedings of the IEEE Symposium on Security and Privacy*. 2021, pp. 1451–1468.
- [129] Hengrui Jia et al. “Proof of Learning”. In: *Proceedings of the IEEE Symposium on Security and Privacy*. 2021, pp. 606–623.
- [130] Alejandro Ranchal-Pedrosa and Vincent Gramoli. “Rational Agreement in the Presence of Crash Faults”. In: *IEEE International Conference on Blockchain (ICBC)*. 2022, pp. 470–475.
- [131] Yackolley Amoussou-Guenou et al. “Rationals vs Byzantines in Consensus-based Blockchains”. In: *Proceedings of AAMAS*. 2020, pp. 43–51.
- [132] Adam Groce et al. “Byzantine Agreement with a Rational Adversary”. In: *ICALP*. 2012.
- [133] Juan A. Garay et al. “Rational Protocol Design: Cryptography against Incentive-Driven Adversaries”. In: *2013 IEEE 54th Annual Symposium on Foundations of Computer Science* (2013), pp. 648–657.

- [134] Christian Badertscher et al. “A Rational Protocol Design Treatment of Bitcoin”. In: *Eurocrypt* (2018).
- [135] Dimitris Karakostas, Aggelos Kiayias, and Thomas Zacharias. “Blockchain Nash Dynamics and the Pursuit of Compliance”. In: *arXiv preprint arXiv:2201.00000* (2022).
- [136] Alejandro Ranchal-Pedrosa and Vincent Gramoli. “TRAP: The Bait to Rational Players to Solve Byzantine Agreement”. In: *ACM AsiaCCS*. 2022, pp. 123–134.
- [137] Ittay Eyal and Emin Gün Sirer. “Majority Is Not Enough: Bitcoin Mining Is Vulnerable”. In: *Financial Cryptography and Data Security (FC)*. 2014, pp. 436–454.
- [138] Juan Garay, Aggelos Kiayias, and Nikos Leonardos. “Chains of Variable Difficulty: A Theory of the Block-Mining Rate”. In: *EUROCRYPT*. 2016, pp. 516–546.
- [139] M. Pease, R. Shostak, and L. Lamport. “Reaching Agreement in the Presence of Faults”. In: *J. ACM* 27.2 (Apr. 1980), pp. 228–234. ISSN: 0004-5411. DOI: [10.1145/322186.322188](https://doi.org/10.1145/322186.322188). URL: <https://doi.org/10.1145/322186.322188>.
- [140] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. “Impossibility of Distributed Consensus with One Faulty Process”. In: *J. ACM* 32.2 (Apr. 1985), pp. 374–382. ISSN: 0004-5411. DOI: [10.1145/3149.214121](https://doi.org/10.1145/3149.214121). URL: <https://doi.org/10.1145/3149.214121>.
- [141] Gabriel Bracha. “Asynchronous Byzantine agreement protocols”. In: *Information and Computation* 75.2 (1987), pp. 130–143. ISSN: 0890-5401. DOI: [https://doi.org/10.1016/0890-5401\(87\)90054-X](https://doi.org/10.1016/0890-5401(87)90054-X). URL: <https://www.sciencedirect.com/science/article/pii/089054018790054X>.
- [142] Leslie Lamport. “The Part-Time Parliament”. In: *ACM Trans. Comput. Syst.* 16.2 (May 1998), pp. 133–169. ISSN: 0734-2071. DOI: [10.1145/279227.279229](https://doi.org/10.1145/279227.279229). URL: <https://doi.org/10.1145/279227.279229>.

- [143] Leslie Lamport. “Paxos Made Simple”. In: *ACM SIGACT News (Distributed Computing Column)* 32, 4 (Whole Number 121, December 2001) (Dec. 2001), pp. 51–58. URL: <https://www.microsoft.com/en-us/research/publication/paxos-made-simple/>.
- [144] Diego Ongaro and John Ousterhout. “In Search of an Understandable Consensus Algorithm”. In: *Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference*. USENIX ATC’14. Philadelphia, PA: USENIX Association, 2014, pp. 305–320. ISBN: 9781931971102.
- [145] Andrew Miller et al. “The Honey Badger of BFT Protocols”. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’16. Vienna, Austria: Association for Computing Machinery, 2016, pp. 31–42. ISBN: 9781450341394. DOI: [10.1145/2976749.2978399](https://doi.org/10.1145/2976749.2978399). URL: <https://doi.org/10.1145/2976749.2978399>.
- [146] Alejandro Ranchal-Pedrosa and Vincent Gramoli. “TRAP: The Bait of Rational Players to Solve Byzantine Consensus”. In: *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*. ASIA CCS ’22. Nagasaki, Japan: Association for Computing Machinery, 2022, pp. 168–181. ISBN: 9781450391405. DOI: [10.1145/3488932.3517386](https://doi.org/10.1145/3488932.3517386). URL: <https://doi.org/10.1145/3488932.3517386>.
- [147] Thomas C. Schelling. *The strategy of Conflict*. Oxford University Press, 1963.
- [148] Alejandro Ranchal-Pedrosa and Vincent Gramoli. “ZLB: A blockchain to tolerate colluding majorities”. In: *arXiv preprint arXiv:2007.10541* (2020).
- [149] Pierre Civi, Seth Gilbert, and Vincent Gramoli. “Polygraph: Accountable Byzantine Agreement”. In: *2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS)*. 2021, pp. 403–413. DOI: [10.1109/ICDCS51616.2021.00046](https://doi.org/10.1109/ICDCS51616.2021.00046).

- [150] Flaviu Cristian et al. “ATOMIC BROADCAST: FROM SIMPLE MESSAGE DIFFUSION TO BYZANTINE AGREEMENT”. In: *Twenty-Fifth International Symposium on Fault-Tolerant Computing, 1995, ' Highlights from Twenty-Five Years'.* (1995), pp. 431–.
- [151] Erica Blum, Jonathan Katz, and Julian Loss. “Tardigrade: An atomic broadcast protocol for arbitrary network conditions”. In: *Advances in Cryptology–ASIACRYPT 2021: 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part II 27.* Springer. 2021, pp. 547–572.
- [152] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. “Consensus in the Presence of Partial Synchrony”. In: *J. ACM* 35.2 (Apr. 1988), pp. 288–323. ISSN: 0004-5411. DOI: [10.1145/42282.42283](https://doi.org/10.1145/42282.42283). URL: <https://doi.org/10.1145/42282.42283>.
- [153] Kostis Karantias, Aggelos Kiayias, and Dionysis Zindros. “Proof-of-Burn”. In: *Financial Cryptography and Data Security: 24th International Conference, FC 2020.* Kota Kinabalu, Malaysia: Springer-Verlag, 2020, pp. 523–540. ISBN: 978-3-030-51279-8. DOI: [10.1007/978-3-030-51280-4_28](https://doi.org/10.1007/978-3-030-51280-4_28). URL: https://doi.org/10.1007/978-3-030-51280-4_28.
- [154] Tim Roughgarden and Clara Shikhelman. “Ignore the Extra Zeroes: Variance-Optimal Mining Pools”. In: *Financial Cryptography and Data Security.* Ed. by Nikita Borisov and Claudia Diaz. Berlin, Heidelberg: Springer Berlin Heidelberg, 2021, pp. 233–249. ISBN: 978-3-662-64331-0.
- [155] Lucianna Kiffer and Rajmohan Rajaraman. “HaPPY-Mine: Designing a Mining Reward Function”. In: *Financial Cryptography and Data Security.* Ed. by Nikita Borisov and Claudia Diaz. Berlin, Heidelberg: Springer Berlin Heidelberg, 2021, pp. 250–268. ISBN: 978-3-662-64331-0.

- [156] Joe Lao. “A network-dependent rewarding system: proof-of-mining”. In: *arXiv e-prints* (2014), arXiv–1409.
- [157] Danilo Bazzanella and Andrea Gangemi. “Bitcoin: a new proof-of-work system with reduced variance”. In: *Financial Innovation* 9.1 (2023), pp. 1–14.
- [158] Gyu Chol Kim et al. “Single-Tiered Hybrid PoW Consensus Protocol to Encourage Decentralization in Bitcoin”. In: *Sec. and Commun. Netw.* 2023 (July 2023). ISSN: 1939-0114. DOI: [10.1155/2023/6169933](https://doi.org/10.1155/2023/6169933). URL: <https://doi.org/10.1155/2023/6169933>.
- [159] *ziftrCOIN*, “A Cryptocurrency to Enable Commerces,”. 2014.
- [160] Loi Luu et al. “{SmartPool}: Practical Decentralized Pooled Mining”. In: *26th USENIX security symposium (USENIX security 17)*. 2017, pp. 1409–1426.
- [161] George Dean Bissias and Brian Neil Levine. “Bobtail: A Proof-of-Work Target that Minimizes Blockchain Mining Variance (Draft)”. In: *NDSS* (2017).
- [162] Rafael Pass and Elaine Shi. “FruitChains: A Fair Blockchain”. In: *Proceedings of the ACM Symposium on Principles of Distributed Computing*. PODC ’17. Washington, DC, USA: Association for Computing Machinery, 2017, pp. 315–324. ISBN: 9781450349925. DOI: [10.1145/3087801.3087809](https://doi.org/10.1145/3087801.3087809). URL: <https://doi.org/10.1145/3087801.3087809>.
- [163] A. Stouka and T. Zacharias. “On the (De) centralization of FruitChains”. In: *2023 IEEE 36th Computer Security Foundations Symposium (CSF)*. Los Alamitos, CA, USA: IEEE Computer Society, July 2023, pp. 229–244. DOI: [10.1109/CSF57540.2023.00020](https://doi.ieeecomputersociety.org/10.1109/CSF57540.2023.00020). URL: <https://doi.ieeecomputersociety.org/10.1109/CSF57540.2023.00020>.
- [164] Andrew Asmakov. *Bitcoin Hash Rate Hits New All-Time High Amid Stagnating Prices*. 2023.

- [165] Daira Hopwood et al. “Zcash protocol specification”. In: *GitHub: San Francisco, CA, USA* 4.220 (2016), p. 32.
- [166] Christian Badertscher, Yun Lu, and Vassilis Zikas. “A Rational Protocol Treatment of 51% Attacks”. In: *Advances in Cryptology – CRYPTO 2021*. Ed. by Tal Malkin and Chris Peikert. Cham: Springer International Publishing, 2021, pp. 3–32. ISBN: 978-3-030-84252-9.
- [167] Jon Matonis. “The bitcoin mining arms race: Ghash. io and the 51% issue”. In: *New York, NY, USA: CoinDesk, Tech. Rep* (2014).
- [168] Ben Fisch, Rafael Pass, and Abhi Shelat. “Socially optimal mining pools”. In: *Web and Internet Economics: 13th International Conference, WINE 2017, Proceedings 13*. Springer. 2017, pp. 205–218.
- [169] Panagiotis Chatzigiannis et al. “Diversification across mining pools: Optimal mining strategies under pow”. In: *Journal of Cybersecurity* 8.1 (2022), tyab027.
- [170] Axel Cortes-Cubero et al. “Smoothing block rewards: How much should miners pay for mining pools?” In: *arXiv preprint arXiv:2309.02297* (2023).
- [171] Chun-Hung Chiu and Tsan-Ming Choi. “Supply chain risk analysis with mean-variance models: A technical review”. In: *Annals of Operations Research* 240.2 (2016), pp. 489–507.
- [172] *Hashrate Distribution Over Time*. <https://www.blockchain.com/explorer/charts/pools-timeseries>. Accessed: 2023-10-09.
- [173] Lin Chen et al. “A Game Theoretical Analysis of Non-Linear Blockchain System”. In: *Distrib. Ledger Technol.* (July 2023). ISSN: 2769-6472. DOI: [10.1145/3607195](https://doi.org/10.1145/3607195). URL: <https://doi.org/10.1145/3607195>.
- [174] Tim Roughgarden. “Transaction fee mechanism design”. In: *ACM SIGecom Exchanges* 19.1 (2021), pp. 52–55.

- [175] Hao Chung and Elaine Shi. “Foundations of transaction fee mechanism design”. In: *ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 2023, pp. 3856–3899.
- [176] Shoeb Siddiqui, Ganesh Vanahalli, and Sujit Gujar. “BitcoinF: Achieving Fairness For Bitcoin In Transaction Fee Only Model”. In: *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems*. AAMAS ’20. Auckland, New Zealand: International Foundation for Autonomous Agents and Multiagent Systems, 2020, pp. 2008–2010. ISBN: 9781450375184.
- [177] Joshua S. Gans and Hanna Halaburda. “Zero Cost” Majority Attacks on Permissionless Blockchains. Papers. arXiv.org, 2023. URL: <https://EconPapers.repec.org/RePEc:arx:papers:2308.06568>.
- [178] Aggelos Kiayias et al. “Blockchain Mining Games”. In: *Proceedings of the 2016 ACM Conference on Economics and Computation*. EC ’16. Maastricht, The Netherlands: Association for Computing Machinery, 2016, pp. 365–382. ISBN: 9781450339360. DOI: [10.1145/2940716.2940773](https://doi.org/10.1145/2940716.2940773). URL: <https://doi.org/10.1145/2940716.2940773>.
- [179] Marcelo Arenas et al. “Cryptocurrency Mining Games with Economic Discount and Decreasing Rewards”. In: *37th International Symposium on Theoretical Aspects of Computer Science (STACS 2020)*. Ed. by Christophe Paul and Markus Bläser. Vol. 154. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2020, 54:1–54:16. ISBN: 978-3-95977-140-5. DOI: [10.4230/LIPIcs.STACS.2020.54](https://doi.org/10.4230/LIPIcs.STACS.2020.54).
- [180] Itay Tsabary and Ittay Eyal. “The gap game”. In: *Proceedings of the 2018 ACM SIGSAC conference on Computer and Communications Security (CCS)*. 2018, pp. 713–728.
- [181] Shashank Motepalli and Hans-Arno Jacobsen. “Reward Mechanism for Blockchains Using Evolutionary Game Theory”. In: *2021 3rd Conference on Blockchain Research*

- Applications for Innovative Networks and Services (BRAINS)*. 2021, pp. 217–224. DOI: [10.1109/BRAINS52497.2021.9569791](https://doi.org/10.1109/BRAINS52497.2021.9569791).
- [182] Amirheckmat Taghizadeh, Hamed Kebriaei, and Dusit Niyato. “Mean Field Game for Equilibrium Analysis of Mining Computational Power in Blockchains”. In: *IEEE Internet of Things Journal* 7.8 (2020), pp. 7625–7635. DOI: [10.1109/JIOT.2020.2988304](https://doi.org/10.1109/JIOT.2020.2988304).
- [183] Xiaojun Liu et al. “Evolutionary Game for Mining Pool Selection in Blockchain Networks”. In: *IEEE Wireless Communications Letters* 7.5 (2018), pp. 760–763. DOI: [10.1109/LWC.2018.2820009](https://doi.org/10.1109/LWC.2018.2820009).
- [184] Chengzhen Xu et al. “Dynamic Selection of Mining Pool with Different Reward Sharing Strategy in Blockchain Networks”. In: *ICC 2020 - 2020 IEEE International Conference on Communications (ICC)*. 2020, pp. 1–6. DOI: [10.1109/ICC40277.2020.9149279](https://doi.org/10.1109/ICC40277.2020.9149279).
- [185] Eitan Altman et al. “Blockchain Competition Between Miners: A Game Theoretic Perspective”. In: *Frontiers in Blockchain*. 2020.
- [186] Ben Fisch, Rafael Pass, and Abhi Shelat. “Socially optimal mining pools”. In: *Web and Internet Economics: 13th International Conference, WINE 2017, Bangalore, India, December 17–20, 2017, Proceedings 13*. Springer. 2017, pp. 205–218.
- [187] Eugenio Cortesi et al. “A New Approach for Bitcoin Pool-Hopping Detection”. In: *Comput. Netw.* 205.C (Mar. 2022). ISSN: 1389-1286. DOI: [10.1016/j.comnet.2021.108758](https://doi.org/10.1016/j.comnet.2021.108758). URL: <https://doi.org/10.1016/j.comnet.2021.108758>.